



Software Requirements Specification

for

NOESIS

An Operating-System-Inspired Kernel for Autonomous Multi-Agent Intelligence

Version 1.0 · 6 October 2026 · As-built specification derived from the source code

Prepared by

Programme	SAP ID	Name
B.Tech., School of Computer Science	500118979	Dhruv Shah
B.Tech., School of Computer Science	500123471	Manan Nasa

Project Guide: Dr. Archana Kumari

Department of Informatics
School of Computer Science
UNIVERSITY OF PETROLEUM & ENERGY STUDIES (UPES)
Dehradun – 248007, Uttarakhand, India
Academic Year 2026–2027

Document Control

Document identification

Field	Value
Document	Software Requirements Specification (SRS) — NOESIS
Version	1.0 (as-built baseline)
Status	Submitted for mentor review — approval pending
Date	6 October 2026
Authors	Dhruv Shah (500118979), Manan Nasa (500123471)
Project guide / reviewer	Dr. Archana Kumari
Baseline	Repository working tree on 2026-10-06 (branch main, last commit 81414fb3, plus 19 uncommitted modified files and 2 new frontend routes)
Method	Read-only static analysis of the code, isolated execution of the test suites, and measurements on a scratch copy (Appendix G)

Version history

Version	Date	Change	Author(s)
0.1	14 Feb 2026	Team SRS template (HyperAGRI-Spectra.AI project) used as structural reference	D. Shah
0.5	Aug 2026	Synopsis-stage requirements in the NOESIS synopsis report	D. Shah, M. Nasa
1.0	6 October 2026	Complete as-built SRS: every requirement traced to code, tests and measurements	D. Shah, M. Nasa

Revision history (to be completed during review)

Date	Change	Reason for change	Mentor signature

Approval

Role	Name	Decision	Date
Project guide	Dr. Archana Kumari	Pending	
Author	Dhruv Shah	Prepared	6 October 2026
Author	Manan Nasa	Prepared	6 October 2026

Table of Contents

Document Control.....	i
List of Abbreviations	xv
1 Introduction.....	1
1.1 Purpose.....	1
1.2 Document conventions and evidence policy.....	1
1.3 Intended audience and reading suggestions.....	2
1.4 Project scope	2
1.5 Background and problem statement.....	3
1.6 Motivation.....	4
1.7 Target beneficiaries.....	4
1.8 Definitions, acronyms and abbreviations.....	5
1.9 References.....	5
1.10 Document organisation	7
2 Overall Description.....	8
2.1 Product perspective.....	8
2.2 Product functions	8
2.3 Reference algorithm (end-to-end processing).....	9
2.4 Data and data structures	10
2.5 SWOT analysis	10
2.6 Project features.....	10
2.7 User classes and characteristics	11
2.8 Operating environment	11
2.9 Design and implementation constraints	12
2.10 Assumptions and dependencies	12
2.11 System boundaries	12
3 System Architecture.....	13

3.1 High-level architecture.....	13
3.2 Component architecture	14
3.3 Module architecture	15
3.4 Deployment architecture	16
3.5 Data architecture	17
3.6 Communication architecture	17
3.7 Data-flow models.....	18
3.8 Design diagrams.....	20
4 External Interface Requirements.....	24
4.1 User interfaces	24
4.2 Software interfaces.....	24
4.3 Hardware interfaces	25
4.4 Communication interfaces and protocols.....	25
5 Functional Requirements	25
5.1 Authentication and session tokens	27
5.2 Conversations and messages.....	30
5.3 Planning and plan execution	33
5.4 Agent roster and agent behaviour	35
5.5 Documents	39
5.6 Memory records and tiered memory	41
5.7 Kernel, capabilities and scheduling	43
5.8 Tool registry and sandboxed tools	46
5.9 LLM provider integration	48
5.10 Health, observability and metrics	49
5.11 Benchmarking and evaluation.....	51
5.12 Universal Agent Protocol.....	52
5.13 Plugins and command-line interface.....	53

5.14 Dashboard (Noesis Studio)	54
5.15 Retrieval pipeline library	56
6 Non-Functional Requirements	57
6.1 Performance	57
6.2 Scalability	59
6.3 Reliability.....	59
6.4 Availability	59
6.5 Security	60
6.6 Privacy	60
6.7 Maintainability.....	60
6.8 Usability.....	60
6.9 Accessibility.....	61
6.10 Portability.....	61
6.11 Compatibility	61
6.12 Observability.....	61
6.13 Fault tolerance.....	61
6.14 Recoverability.....	62
6.15 Resource utilisation.....	62
6.16 Reproducibility	62
6.17 Software quality attributes	62
7 User Requirements.....	63
7.1 User workflows.....	63
7.2 Access restrictions	63
8 Use Cases.....	64
8.1 UC-01 Log in and obtain tokens.....	64
8.2 UC-02 Refresh access token	65
8.3 UC-03 View own identity.....	65

8.4 UC-04 Start an autonomous run from a goal	65
8.5 UC-05 Browse and search conversations	66
8.6 UC-06 Work with a thread.....	66
8.7 UC-07 Plan a goal.....	67
8.8 UC-08 Execute a plan	67
8.9 UC-09 Inspect run executions.....	67
8.10 UC-10 Invoke a single agent.....	68
8.11 UC-11 Manage documents	68
8.12 UC-12 Query and compact memory records	68
8.13 UC-13 Monitor health and kernel	69
8.14 UC-14 Scrape metrics	69
8.15 UC-15 Measure LLM throughput.....	69
8.16 UC-16 View benchmark results.....	70
8.17 UC-17 Exchange UAP envelopes.....	70
8.18 UC-18 Run offline evaluation and CLI	70
9 Data Requirements.....	71
9.1 Input data	71
9.2 Output data.....	71
9.3 Key domain objects.....	71
9.4 Validation rules.....	72
9.5 Storage requirements and data lifecycle	72
10 API Specification.....	72
10.1 Common conventions	73
10.2 Health.....	74
10.3 Kernel.....	76
10.4 Authentication.....	78
10.5 Observability and metrics	80

10.6	Conversations.....	82
10.7	Agents	87
10.8	Runs	88
10.9	Documents	90
10.10	Memory.....	93
10.11	UAP.....	95
10.12	Benchmarking (capability-gated).....	97
11	Database Design.....	98
11.1	Relationships and referential actions	102
11.2	Transactions and pagination	102
12	AI/ML Specification	102
12.1	Models used	103
12.2	Inference pipeline.....	103
12.3	Evaluation datasets and harness.....	104
12.4	Evaluation metrics and statistics.....	105
12.5	Measured and reported results	105
12.6	Model limitations	106
13	Algorithms	106
13.1	A1 Deterministic goal decomposition (PlannerAgent).....	107
13.2	A2 Topological execution with scoped tokens (AgentSvc).....	107
13.3	A3 Six-tier promotion with hash chain (PromotionController).....	108
13.4	A4 Tri-state acceptance decision (ExecutorAgent)	109
13.5	A5 Heuristic review agents (Reflection, Judge, Critic)	109
13.6	A6 Capability-token verification (HTTP gate)	110
13.7	A7 Weighted model routing (ModelRouter).....	110
13.8	A8 Priority/deadline scheduling (Scheduler).....	110
13.9	A9 BM25 + Reciprocal Rank Fusion retrieval	111

13.10 A10 Sliding-window rate limiting	112
14 UI/UX Requirements	113
14.1 General UI requirements	113
14.2 Dashboard (/)	114
14.3 Conversations (/conversations)	115
14.4 New run (/conversations/new)	116
14.5 Thread (/conversations/[id])	117
14.6 Agents (/agents)	118
14.7 Bench Runner (/agents/benchmark)	119
14.8 Bench Results Hub (/benchmarks)	120
14.9 Execution Timeline (/timeline)	121
14.10 Memory (/memory)	122
14.11 Memory Explorer (/memory/explorer)	123
14.12 Documents (/documents)	124
14.13 Observability (/metrics)	125
14.14 Settings (/settings)	126
15 Security Requirements	126
15.1 Security requirements	127
15.2 Threat considerations	127
16 Error Handling	128
16.1 Recovery behaviour	129
17 Testing Requirements and Traceability	129
17.1 Test strategy	129
17.2 Requirements traceability matrix	131
17.3 Acceptance test cases (selected)	133
18 Deployment Requirements	134
18.1 Build and deployment process (verified path)	134

18.2 Container deployment (not executed).....	134
18.3 Continuous integration.....	135
19 Other Requirements	135
20 Limitations	135
21 Future Scope	136
Appendix A: Glossary.....	137
Appendix B: Analysis Models	137
Appendix C: Issues List.....	137
Appendix D: Configuration Reference	138
Appendix E: Repository Structure	139
Appendix F: Documentation-to-Implementation Reconciliation	139
Appendix G: Audit Evidence Log	140

List of Figures

Figure 2.1: System context: actors, the NOESIS boundary and external services	8
Figure 3.1: Layered architecture of NOESIS (as built)	13
Figure 3.2: Component diagram: backend modules and their principal dependencies	14
Figure 3.3: Deployment architecture: verified laptop configuration and the (unexecuted) container definitions.....	16
Figure 3.4: Data architecture: durable, volatile and interchange data	17
Figure 3.5: Communication architecture and protocol conventions.....	17
Figure 3.6: Data-flow diagram, level 0	18
Figure 3.7: Data-flow diagram, level 1	19
Figure 3.8: Class diagram of the agent subsystem.....	20
Figure 3.9: Class diagram of the kernel subsystem	21
Figure 3.10: Sequence diagram: creating a conversation and running its plan in the background.....	22
Figure 3.11: Sequence diagram: login, identity and refresh	22
Figure 3.12: Sequence diagram: capability-token verification on a gated route	23
Figure 3.13: State diagrams for conversations, plan steps and the kernel	23
Figure 4.1: Dashboard site map and data source of each route	24
Figure 6.1: Measured in-process API latency (deterministic mode)	58
Figure 6.2: Measured local-LLM latency and throughput through GET /llm/benchmark	59
Figure 8.1: Use-case diagram	64
Figure 10.1: Distribution of API operations by route group (from the live OpenAPI schema)	73
Figure 10.2: Interactive API documentation (Swagger UI) served at /docs — screenshot of the running backend.....	73
Figure 11.1: Entity–relationship diagram of the relational schema (as defined in the ORM)	99

Figure 12.1: Agent runtime: per-step execution with LLM enrichment and deterministic fallback.....	103
Figure 12.2: Composition of the Noesis-SE50 corpus	104
Figure 12.3: Syntax-compile pass per task in the W7 run of 6 October 2026.....	106
Figure 13.1: Deterministic goal decomposition (PlannerAgent)	107
Figure 13.2: Six-tier promotion with hash chain (PromotionController)	108
Figure 13.3: Tri-state acceptance decision (ExecutorAgent).....	109
Figure 13.4: Weighted model routing (ModelRouter).....	110
Figure 13.5: Priority/deadline scheduling (Scheduler)	111
Figure 13.6: BM25 + Reciprocal Rank Fusion retrieval	112
Figure 13.7: Measured tier occupancy in a controlled promotion scenario.....	112
Figure 14.1: Dashboard (/) — screenshot of the running application	114
Figure 14.2: Conversations (/conversations) — screenshot of the running application ..	115
Figure 14.3: New run (/conversations/new) — screenshot of the running application ..	116
Figure 14.4: Thread (/conversations/[id]) — screenshot of the running application.....	117
Figure 14.5: Agents (/agents) — screenshot of the running application	118
Figure 14.6: Bench Runner (/agents/benchmark) — screenshot of the running application	119
Figure 14.7: Bench Results Hub (/benchmarks) — screenshot of the running application	120
Figure 14.8: Execution Timeline (/timeline) — screenshot of the running application ..	121
Figure 14.9: Memory (/memory) — screenshot of the running application.....	122
Figure 14.10: Memory Explorer (/memory/explorer) — screenshot of the running application.....	123
Figure 14.11: Documents (/documents) — screenshot of the running application	124
Figure 14.12: Observability (/metrics) — screenshot of the running application	125
Figure 14.13: Settings (/settings) — screenshot of the running application.....	126
Figure 16.1: Exception handling and recovery paths	128

Figure 17.1: Backend test results per module (measured).....	130
Figure 17.2: Backend line coverage per package (measured)	130
Figure 17.3: Frontend Vitest results per file (measured)	131
Figure 18.1: GitHub Actions workflows defined in the repository	135

List of Tables

Table 1.1: Implementation-status vocabulary used throughout this SRS	1
Table 1.2: Audience and reading guide	2
Table 1.3: Project deliverables and status	3
Table 1.4: Target beneficiaries.....	4
Table 2.1: Product function groups and implementation status.....	8
Table 2.2: Principal data structures	10
Table 2.3: SWOT — internal factors	10
Table 2.4: SWOT — external factors	10
Table 2.5: User classes.....	11
Table 2.6: Operating environment	11
Table 2.7: Design and implementation constraints	12
Table 2.8: System boundary.....	12
Table 3.1: Backend packages (measured; 17,072 non-blank, non-comment lines).....	14
Table 3.2: Architecturally significant modules.....	15
Table 4.1: Software interfaces	24
Table 4.2: Communication interfaces	25
Table 5.1: Functional requirements — summary index.....	25
Table 6.1: Performance requirements	57
Table 6.2: Scalability requirements	59
Table 6.3: Reliability requirements.....	59
Table 6.4: Availability requirements	59
Table 6.5: Security requirements	60
Table 6.6: Privacy requirements	60
Table 6.7: Maintainability requirements.....	60
Table 6.8: Usability requirements.....	60
Table 6.9: Accessibility requirements.....	61

Table 6.10: Portability requirements.....	61
Table 6.11: Compatibility requirements	61
Table 6.12: Observability requirements.....	61
Table 6.13: Fault tolerance requirements.....	61
Table 6.14: Recoverability requirements	62
Table 6.15: Resource utilisation requirements.....	62
Table 6.16: Reproducibility requirements	62
Table 6.17: Software quality attributes	62
Table 7.1: User roles, permissions and enforcement	63
Table 9.1: Input data	71
Table 9.2: Output data.....	71
Table 9.3: Principal domain objects.....	71
Table 9.4: Data lifecycle	72
Table 10.1: API conventions.....	73
Table 11.1: Table documents.....	99
Table 11.2: Table memory_tiers_promotions.....	100
Table 11.3: Table users	100
Table 11.4: Table conversations.....	100
Table 11.5: Table memories.....	100
Table 11.6: Table messages.....	101
Table 11.7: Table task_executions	101
Table 11.8: Relationships.....	102
Table 12.1: Language models referenced by the code.....	103
Table 12.2: Per-agent generation settings (services/kernel_services.py)	104
Table 12.3: Evaluation datasets and harness semantics	104
Table 12.4: Metrics and statistical tests	105
Table 12.5: AI-related results.....	105

Table 14.1: General UI requirements.....	113
Table 15.1: Security mechanisms and gaps	126
Table 15.2: Security requirements	127
Table 15.3: Threat analysis	127
Table 16.1: Error classes and handling	128
Table 17.1: Test levels and results	129
Table 17.2: Requirements traceability matrix (functional requirements)	131
Table 17.3: Acceptance test cases executed during the audit	133
Table 18.1: Deployment requirements.....	134
Table 19.1: Other requirements	135
Table 20.1: Known limitations.....	135
Table 21.1: Future enhancements (proposed)	136
Table A.1: Glossary	137
Table B.1: Analysis models in this SRS	137
Table C.1: Open issues identified during the audit.....	137
Table D.1: Environment variables (config.py Settings and frontend)	138
Table F.1: Claims versus implementation	139
Table F.2: Sanskrit codename mappings found in three places.....	140
Table G.1: Evidence collected for this SRS	140

List of Abbreviations

Abbreviation	Meaning
ABC	Abstract Base Class (Python)
API	Application Programming Interface
ASGI	Asynchronous Server Gateway Interface
AST	Abstract Syntax Tree
BM25	Okapi Best Matching 25 ranking function
CI	Continuous Integration
CI95	95 % confidence interval
CLI	Command-Line Interface
CORS	Cross-Origin Resource Sharing
CRUD	Create, Read, Update, Delete
CSV	Comma-Separated Values
DAG	Directed Acyclic Graph
DFD	Data-Flow Diagram
DI	Dependency Injection
DTO	Data Transfer Object
ER	Entity–Relationship
FR	Functional Requirement
GC	Garbage Collection
HMAC	Hash-based Message Authentication Code
HS256	HMAC with SHA-256 (JWT algorithm)
HTTP(S)	Hypertext Transfer Protocol (Secure)
IRQ	Interrupt request (kernel interrupt record)
JSON	JavaScript Object Notation
JWT	JSON Web Token
LLM	Large Language Model
LOC	Lines of Code
LRU	Least Recently Used
MAC	Mandatory Access Control (as used by the project for capability gating)
MBPP	Mostly Basic Python Problems benchmark
NFR	Non-Functional Requirement
OOM	Out of Memory
ORM	Object–Relational Mapping
OTel	OpenTelemetry
RAG	Retrieval-Augmented Generation
REST	Representational State Transfer
RRF	Reciprocal Rank Fusion
RTM	Requirements Traceability Matrix
SE-50	Noesis software-engineering 50-task corpus
SHA-256	Secure Hash Algorithm, 256-bit
SQL	Structured Query Language
SRS	Software Requirements Specification
SSRF	Server-Side Request Forgery
TLV	Tag–Length–Value encoding
TTL	Time To Live

Abbreviation	Meaning
UAP	Universal Agent Protocol (project-defined)
UC	Use Case
UI/UX	User Interface / User Experience
UUID	Universally Unique Identifier
WCAG	Web Content Accessibility Guidelines

1 Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines what the NOESIS system does, how well it does it, and under which constraints it operates. NOESIS is an operating-system-inspired orchestration kernel for multi-agent artificial-intelligence (AI) workflows. A user states a goal in natural language; the system decomposes it into a dependency-ordered plan, runs a roster of twelve specialised agents step by step under capability-scoped permissions, records an audit trail, and presents the result in a web dashboard.

Unlike a forward-looking SRS written before implementation, this document is an **as-built specification**: every requirement was derived from, and checked against, the source code that existed in the repository on 6 October 2026. Where the code, the project README and earlier project documents disagree, the code is treated as authoritative and the difference is recorded explicitly (Appendix F). The SRS follows the structure recommended by IEEE Std 830 and ISO/IEC/IEEE 29148 [37, 38], adapted to the School of Computer Science SRS template used by the team.

1.2 Document conventions and evidence policy

Requirements are identified as FR-*nnn* (functional), NFR-*<category>-nn* (non-functional) and UC-*nn* (use cases). Each requirement carries an **implementation status** drawn from the fixed vocabulary in Table 1.1, so that a reader can always distinguish what runs today from what is designed or planned.

Table 1.1: Implementation-status vocabulary used throughout this SRS

Status label	Meaning
Implemented	Behaviour exists in the code, is reachable through the API/CLI/UI, and was exercised by tests or measurements.
Partially implemented	Behaviour exists but deviates from the stated requirement in a documented way (e.g. a defect or a missing branch).
Implemented as library (not wired to API)	Working, tested library code that no API route or runtime path currently calls.
Stub / simulated	The interface exists but returns simulated, synthetic or placeholder results.
Planned (not implemented)	Described in project documents or required by good practice, but absent from the code.
Not verifiable	Cannot be confirmed from the available implementation (e.g. deployment targets that were not executed).

Every quantitative statement is labelled **measured** (obtained during this audit; method in Appendix G), **derived** (computed from code constants, e.g. routing scores) or **reported** (copied from evaluation artefacts committed in `docs/eval/`). File references use repository-relative paths such as `backend/noesis/kernel/kernel.py`. Code identifiers appear in a monospace font.

1.3 Intended audience and reading suggestions

Table 1.2: Audience and reading guide

Reader	Recommended sections
Project guide / examiners	1, 2, 5 (summary table), 6, 17 (traceability), 20 (limitations)
Developers and maintainers	3, 5, 9–13, 16, 18, Appendices D–E
Testers	5 (acceptance criteria), 6, 8, 17
Operators	4, 15, 18, 19, Appendix D
Researchers	12, 13, 17.6 and the companion thesis

1.4 Project scope

NOESIS provides a self-hostable backend kernel and a web dashboard for running and auditing multi-agent workflows. In scope are: goal intake and conversation management; deterministic planning; dependency-ordered execution by twelve typed agents; capability-based permission checks for agents, tools and selected HTTP routes; optional enrichment of agent output by a local or hosted LLM; a six-tier memory promotion model with a hash-chained provenance log; a lexical retrieval library; health, metrics and kernel introspection endpoints; a benchmark harness with statistics; and a Next.js dashboard.

1.4.1 System objectives

1. Provide a typed agent runtime in which each step of a plan is executed by a named agent under an explicitly minted, least-privilege capability set.
2. Make planning reproducible: identical goal and seed inputs must produce bit-identical plans and audit hashes.
3. Separate decision logic (deterministic agents) from language generation (LLM), so that the system keeps working when no LLM is available.
4. Record an auditable trail of every executed step (TaskExecution rows, conversation messages, kernel spans, promotion hash chain).
5. Offer a dashboard and REST API for creating runs, inspecting threads and monitoring the kernel.

6. Provide an evaluation harness for software-engineering goal sets and code benchmarks with statistical aggregation.

1.4.2 System deliverables

Table 1.3: Project deliverables and status

Deliverable	Location	Status
Backend package <code>noesis</code> (FastAPI kernel, agents, memory, tools, LLM adapters)	backend/noesis/	Implemented
Dashboard (Next.js 14, 13 routes)	frontend/app/	Partially implemented (6 pages show placeholder data)
Automated tests (312 backend, 105 frontend)	backend/tests/, frontend/src/**/*.*.test.ts(x)	Implemented
Evaluation harness and SE-50 corpus	backend/scripts/, backend/benchmarks/	Partially implemented
Container definitions	backend/Dockerfile, frontend/Dockerfile, docker-compose*.yaml	Not verifiable (not executed)
Wheel build	backend/dist_rc2/noesis-0.1.0-py3-none-any.whl	Implemented
Documentation package (this SRS, thesis, presentation)	final_documentation/	Implemented

1.4.3 Out of scope

- Multi-tenant hosting, billing or user self-registration.
- Distributed (multi-node) kernels; the scheduler and all in-memory state are single-process.
- Training or fine-tuning of models; NOESIS consumes pre-trained models through HTTP APIs.
- Edge-hardware deployment (NVIDIA Jetson) described in the synopsis — no Jetson-specific code exists.

1.5 Background and problem statement

Large language models can follow instructions, call tools and draft code, and frameworks such as LangChain/LangGraph, AutoGen, MetaGPT and CrewAI [46, 13, 14, 47] compose them into multi-agent systems. These frameworks are primarily *libraries for wiring prompts and tools*. They typically leave four system-level concerns to the application developer:

1. **Permission boundaries** — any agent can usually call any registered tool; there is no kernel that checks a scoped capability before a side effect occurs.
2. **Reproducibility** — sampling-based generation makes the same goal produce different plans, which hinders debugging, regression testing and audit.

3. **Memory lifecycle** — memory is commonly a flat vector store; promotion, retention and provenance are ad hoc.
4. **Operational visibility** — health, metrics, per-step audit records and a human-facing console are not part of the agent framework itself.

Operating systems solved analogous problems for processes decades ago with capability-based protection [2, 3, 4], scheduling and process lifecycle [5], and memory hierarchies. Recent research proposes treating the LLM itself or the agent runtime as an operating system [7, 8]. **Problem statement:** design and implement an orchestration kernel that applies operating-system primitives — least-privilege capabilities, deterministic scheduling of plan steps, tiered memory with provenance, and observable system calls — to multi-agent LLM workflows, while remaining usable on a single developer laptop without paid API keys.

1.6 Motivation

- **Safety of side effects.** Agents that write files, run shell commands or fetch URLs need an explicit, auditable permission model; prompt injection makes model output untrusted [27, 39].
- **Engineering discipline.** Deterministic, seed-controlled behaviour allows unit tests, snapshot tests and exact replay — standard practice for software, rare for agent systems.
- **Cost and privacy.** Local models (e.g. Qwen2.5-Coder 7B via Ollama) remove per-token fees and keep data on the machine; a kernel must therefore work with or without hosted APIs.
- **Teaching value.** Mapping agents, tools and memory onto OS concepts (syscalls, capability tables, schedulers, interrupts) makes the architecture explainable and testable.

1.7 Target beneficiaries

Table 1.4: Target beneficiaries

Beneficiary	Benefit
Software developers and small teams	Run goal-driven multi-agent workflows locally with an audit trail and permission checks.
AI-systems researchers and students	A transparent, test-covered reference implementation of capability-gated agents, deterministic planning and tiered memory.
Platform/security engineers	A concrete model of least-privilege tool access and signed capability tokens for agent APIs.
Course instructors	Executable examples of OS concepts (syscalls, schedulers, interrupts) applied to a modern workload.

1.8 Definitions, acronyms and abbreviations

Abbreviations are listed in the front matter; a glossary of domain terms (agent, capability, syscall, tier, promotion, envelope, run, step, etc.) is given in Appendix A.

1.9 References

The documents, standards, research papers and software referenced by this SRS are listed below. The numbering is shared with the companion thesis.

- [1] R. C. Atkinson and R. M. Shiffrin, "Human memory: A proposed system and its control processes," in *The Psychology of Learning and Motivation*, vol. 2, K. W. Spence and J. T. Spence, Eds. New York: Academic Press, 1968, pp. 89–195, doi: 10.1016/S0079-7421(08)60422-3.
- [2] J. B. Dennis and E. C. Van Horn, "Programming semantics for multiprogrammed computations," *Communications of the ACM*, vol. 9, no. 3, pp. 143–155, 1966, doi: 10.1145/365230.365252.
- [3] J. H. Saltzer and M. D. Schroeder, "The protection of information in computer systems," *Proceedings of the IEEE*, vol. 63, no. 9, pp. 1278–1308, 1975, doi: 10.1109/PROC.1975.9939.
- [4] M. S. Miller, K.-P. Yee, and J. Shapiro, "Capability myths demolished," Systems Research Laboratory, Johns Hopkins University, Tech. Rep. SRL2003-02, 2003.
- [5] D. M. Ritchie and K. Thompson, "The UNIX time-sharing system," *Communications of the ACM*, vol. 17, no. 7, pp. 365–375, 1974, doi: 10.1145/361011.361061.
- [6] A. B. Kahn, "Topological sorting of large networks," *Communications of the ACM*, vol. 5, no. 11, pp. 558–562, 1962, doi: 10.1145/368996.369025.
- [7] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez, "MemGPT: Towards LLMs as operating systems," arXiv:2310.08560, 2023.
- [8] K. Mei, Z. Li, S. Xu, R. Ye, Y. Ge, and Y. Zhang, "AIOs: LLM agent operating system," arXiv:2403.16971, 2024.
- [9] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2023, arXiv:2210.03629.
- [10] J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, arXiv:2201.11903.
- [11] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.11366.
- [12] T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2302.04761.
- [13] Q. Wu *et al.*, "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," arXiv:2308.08155, 2023.

- [14] S. Hong *et al.*, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024, arXiv:2308.00352.
- [15] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for 'mind' exploration of large language model society," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023, arXiv:2303.17760.
- [16] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proc. ACM Symp. User Interface Software and Technology (UIST)*, 2023, arXiv:2304.03442.
- [17] L. Wang *et al.*, "A survey on large language model based autonomous agents," *Frontiers of Computer Science*, vol. 18, no. 6, 2024, arXiv:2308.11432.
- [18] Z. Xi *et al.*, "The rise and potential of large language model based agents: A survey," arXiv:2309.07864, 2023.
- [19] P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, arXiv:2005.11401.
- [20] S. Robertson and H. Zaragoza, "The probabilistic relevance framework: BM25 and beyond," *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009, doi: 10.1561/15000000019.
- [21] G. V. Cormack, C. L. A. Clarke, and S. Büttcher, "Reciprocal rank fusion outperforms Condorcet and individual rank learning methods," in *Proc. 32nd Int. ACM SIGIR Conf.*, 2009, pp. 758–759, doi: 10.1145/1571941.1572114.
- [22] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, vol. 12, pp. 157–173, 2024, doi: 10.1162/tacl_a_00638.
- [23] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.
- [24] J. Austin *et al.*, "Program synthesis with large language models," arXiv:2108.07732, 2021.
- [25] C. E. Jimenez *et al.*, "SWE-bench: Can language models resolve real-world GitHub issues?," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024, arXiv:2310.06770.
- [26] B. Hui *et al.*, "Qwen2.5-Coder technical report," arXiv:2409.12186, 2024.
- [27] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, arXiv:2302.12173.
- [28] M. S. Charikar, "Similarity estimation techniques from rounding algorithms," in *Proc. 34th ACM Symp. Theory of Computing (STOC)*, 2002, pp. 380–388, doi: 10.1145/509907.509965.
- [29] S. Haber and W. S. Stornetta, "How to time-stamp a digital document," *Journal of Cryptology*, vol. 3, no. 2, pp. 99–111, 1991, doi: 10.1007/BF00196791.
- [30] H. Krawczyk, M. Bellare, and R. Canetti, "HMAC: Keyed-hashing for message authentication," IETF RFC 2104, 1997.
- [31] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, 2015.

- [32] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979, doi: 10.1214/aos/1176344552.
- [33] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947, doi: 10.1007/BF02295996.
- [34] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [35] J. Pineau *et al.*, "Improving reproducibility in machine learning research (a report from the NeurIPS 2019 reproducibility program)," *Journal of Machine Learning Research*, vol. 22, no. 164, pp. 1–20, 2021.
- [36] A. Cockburn, "Hexagonal architecture (ports and adapters)," 2005. [Online]. Available: <https://alistair.cockburn.us/hexagonal-architecture/>
- [37] "IEEE recommended practice for software requirements specifications," IEEE Std 830-1998, 1998.
- [38] "Systems and software engineering — Life cycle processes — Requirements engineering," ISO/IEC/IEEE 29148:2018, 2018.
- [39] OWASP Foundation, "OWASP Top 10 for Large Language Model Applications," 2023. [Online]. Available: <https://owasp.org/www-project-top-10-for-large-language-model-applications/>
- [40] S. Ramírez, "FastAPI," software framework. [Online]. Available: <https://fastapi.tiangolo.com/>
- [41] Pydantic Services, "Pydantic v2 documentation." [Online]. Available: <https://docs.pydantic.dev/>
- [42] SQLAlchemy authors, "SQLAlchemy 2.0 documentation." [Online]. Available: <https://docs.sqlalchemy.org/>
- [43] Vercel, "Next.js documentation (App Router)." [Online]. Available: <https://nextjs.org/docs>
- [44] Ollama, "Ollama API documentation." [Online]. Available: <https://github.com/ollama/ollama/blob/main/docs/api.md>
- [45] Prometheus authors, "Exposition formats." [Online]. Available: https://prometheus.io/docs/instrumenting/exposition_formats/
- [46] LangChain, "LangChain and LangGraph documentation." [Online]. Available: <https://python.langchain.com/>
- [47] CrewAI, "CrewAI documentation." [Online]. Available: <https://docs.crewai.com/>

Project sources: repository README.md, ARCHITECTURE.md, AUDIT.md, NOESIS synopsis report (DOCX, 2026), synopsis presentation (PPTX), and the team's SRS template (PDF, 14 Feb 2026). The PDF copy of the synopsis supplied with the documentation request was empty (0 bytes); the DOCX version in the repository was used instead.

1.10 Document organisation

Chapter 2 describes the product in its environment; Chapter 3 its architecture; Chapter 4 its external interfaces. Chapters 5–8 state functional, non-functional and user requirements and the use cases. Chapters 9–14 specify data, API, database, the AI/agent subsystem, algorithms and the user interface. Chapters 15–19 cover security, error handling, testing, deployment and other requirements. Chapters 20–21 record limitations

and future scope. Appendices contain the glossary, analysis models, issue list, configuration reference, repository map, documentation reconciliation and the audit evidence log.

2 Overall Description

2.1 Product perspective

NOESIS is a new, self-contained product consisting of a Python backend (package `noesis`, served by Uvicorn/FastAPI) and a TypeScript dashboard (Next.js 14). It depends on a relational database (SQLite by default), optionally on a local Ollama server or hosted LLM APIs, and optionally on Qdrant and Redis, which are only probed by the health endpoint in the current code. Figure 2.1 shows the system in its environment.

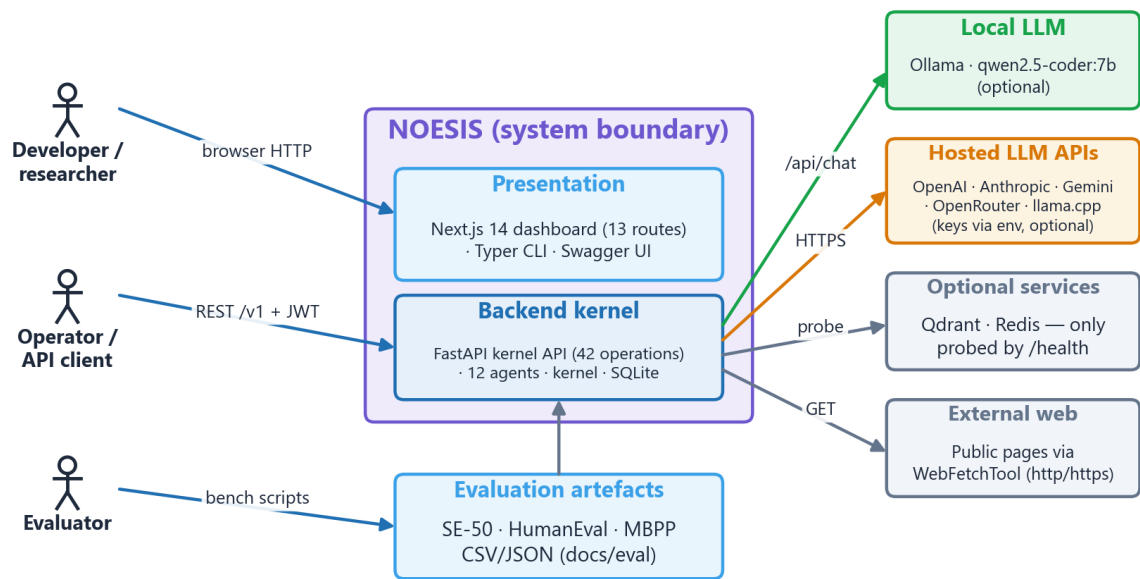


Figure 2.1: System context: actors, the NOESIS boundary and external services

Three human actors interact with NOESIS: dashboard users (browser), API clients/operators (REST and CLI) and evaluators (benchmark scripts). The backend optionally calls a local Ollama server or hosted LLM APIs over HTTP, probes Qdrant/Redis for readiness, fetches public web pages through the `web_fetch` tool, and serves evaluation tables written by the harness.

2.2 Product functions

Table 2.1: Product function groups and implementation status

Function group	Requirements	#	Status mix
----------------	--------------	---	------------

Function group	Requirements	#	Status mix
Authentication and session tokens	FR-001–FR-005	5	Implemented; Planned (not implemented)
Conversations and messages	FR-006–FR-012	7	Implemented
Planning and plan execution	FR-013–FR-017	5	Implemented; Partially implemented
Agent roster and agent behaviour	FR-018–FR-026	9	Implemented; Stub / simulated
Documents	FR-027–FR-029	3	Implemented; Stub / simulated
Memory records and tiered memory	FR-030–FR-035	6	Implemented; Implemented as library (not wired to API); Partially implemented; Planned (not implemented)
Kernel, capabilities and scheduling	FR-036–FR-041	6	Implemented; Implemented as library (not wired to API)
Tool registry and sandboxed tools	FR-042–FR-044	3	Implemented; Implemented as library (not wired to API)
LLM provider integration	FR-045–FR-047	3	Implemented; Implemented as library (not wired to API)
Health, observability and metrics	FR-048–FR-051	4	Implemented; Partially implemented; Stub / simulated
Benchmarking and evaluation	FR-052–FR-054	3	Implemented; Partially implemented
Universal Agent Protocol	FR-055–FR-056	2	Implemented as library (not wired to API); Stub / simulated
Plugins and command-line interface	FR-057–FR-058	2	Implemented
Dashboard (Noesis Studio)	FR-059–FR-063	5	Implemented; Partially implemented; Stub / simulated
Retrieval pipeline library	FR-064–FR-065	2	Implemented as library (not wired to API)

2.3 Reference algorithm (end-to-end processing)

The core processing chain, triggered when a user submits a goal, is summarised below; the detailed algorithms are specified in Chapter 13.

1. **Goal intake** — the goal text becomes a conversation title and the first user message (FR-006).
2. **Deterministic decomposition** — PlannerAgent tokenises the goal, derives a seed string from the seed, goal length and SHA-256 of the goal, chooses 3–12 worker steps by keyword classification, links each step to earlier steps, and appends a final reflection step (Algorithm A1).
3. **Topological execution** — AgentSvc orders steps with Kahn's algorithm [6] and, for each step, mints a capability token limited to the agent's declared capabilities plus its tool hints (A2).
4. **Agent work** — the assigned rule-based agent produces a typed report; when an LLM is reachable its response to the agent's system prompt is attached as rationale (FR-040).
5. **Audit** — every step yields a TaskExecution row and an assistant message; the conversation receives a run summary and a final status.

6. **Memory** — the plan and every step are inserted as T1 memory entries and promotion passes run, producing a provenance hash (A3).
7. **Decision gate** — when the executor agent is part of a plan it returns SIGNOFF, REPLAN or REJECT from weighted acceptance criteria (A4).

2.4 Data and data structures

Table 2.2: Principal data structures

Data structure	Purpose	Defined in
Pydantic v2 models (<code>ExecutionPlan</code> , <code>PlanStep</code> , <code>Conversation</code> , <code>Message</code> , <code>TaskExecution</code> , ...)	Validated DTOs at every trust boundary	<code>types.py</code>
Agent report models (12 types)	Typed output of each agent	<code>agents/core.py</code>
Capability (op, target-glob) / <code>CapabilityToken</code>	Permission set and opaque token identity	<code>kernel/capabilities.py</code>
Binary heap of (−priority, −deadline, seq)	Scheduler run-queue	<code>kernel/scheduler.py</code>
OrderedDict (LRU) / dict tiers	Memory tiers T1–T6 and scope tiers	<code>memory/tiers.py</code> , <code>promotion.py</code>
Inverted index {term → {fragment → tf}}	BM25 retrieval	<code>rag/pipeline.py</code>
SHA-256 hash chain	Promotion provenance, content hashes, plan audit	<code>memory/promotion.py</code>
SQL tables (7)	Durable conversations, messages, documents, memories, executions, users, tier snapshots	<code>database/models.py</code>

2.5 SWOT analysis

Table 2.3: SWOT — internal factors

Strengths	Weaknesses
• Clear OS-style architecture (ports/adapters, syscalls, capability table) • Deterministic planner — bit-identical plans (measured 50/50) • Works without any LLM; LLM adds rationale only • 310 passing backend tests, 70 % coverage	• Agents are rule-based heuristics; output quality is limited • Resource routes are unauthenticated • Several features are stubs (document ingestion, RAG agent, UAP send) • Six dashboard pages render placeholder data

Table 2.4: SWOT — external factors

Opportunities	Threats
• Wire the existing BM25 library and memory bus into the API • Replace heuristics with LLM-backed agents behind the same typed reports • Functional (test-executing) code benchmarks • Use the capability gate on all routes	• Mature frameworks (LangGraph, AutoGen) evolve quickly • Local 7B models are slow on CPU-class hardware (≈ 23 tok/s measured) • Prompt-injection risk grows once LLM output drives actions • Inflated claims in the README can undermine credibility

2.6 Project features

- Goal-driven runs with conversation threads and per-step messages
- Deterministic, seed-controlled planner and topological runtime
- Twelve typed agents with spawn-time capability checks

- Kernel with 27 syscall identifiers, capability table, span ring buffer, scheduler, allocator, model router and interrupt model
- HMAC-signed capability tokens on benchmark routes
- JWT login with bcrypt/scrypt hashing and sliding-window rate limiting
- Six-tier memory promotion controller with SHA-256 provenance chain
- BM25 + RRF retrieval library with citations
- Adapters for five LLM provider families with retry and failover
- Prometheus metrics, health probes, structured logs with secret redaction
- Benchmark harness and statistics module
- Next.js dashboard with conversation workflow and monitoring pages
- Plugin manager and Typer CLI

2.7 User classes and characteristics

Table 2.5: User classes

User class	Characteristics	Typical activities
Dashboard user (developer / researcher)	Comfortable with web apps; may not know the API	Start runs, read threads, browse pages
API client / operator	Uses curl, scripts or the CLI; reads JSON	Plan/execute runs, manage documents, monitor health
Administrator	Holds the <code>admin</code> role in the built-in user directory	Same as operator; no admin-only routes are enforced today
Evaluator	Runs Python scripts; interprets statistics	Benchmark runs, throughput tests, result review
Maintainer	Python/TypeScript developer	Extend agents, tools, providers, plugins

2.8 Operating environment

Table 2.6: Operating environment

Component	Requirement (declared)	Verified during audit
Operating system	Windows, Linux, macOS (portable Python/Node)	Windows 11 Home 10.0.26200
Python	≥ 3.10 , < 3.13 (pyproject); CI uses 3.10–3.12	3.13.2 (all tests passed)
Node.js	≥ 20.17	20.18.1 (bundled in frontend/.node)
Database	SQLite via aiosqlite (default); PostgreSQL by URL	SQLite
LLM runtime (optional)	Ollama $\geq$ any version exposing <code>/api/chat</code>	Ollama with qwen2.5-coder:7b-instruct-q4_K_M
Browser	Modern evergreen browser	Microsoft Edge 154 (headless)
Containers (optional)	Docker Compose v2	Not executed

2.9 Design and implementation constraints

Table 2.7: Design and implementation constraints

Category	Constraint
Hardware	Runs on a single laptop; LLM throughput depends on local CPU/GPU (measured ≈ 23 completion tok/s for a 7B Q4 model).
Software	Python async stack (FastAPI, SQLAlchemy 2 async); Pydantic v2 everywhere; Next.js 14 App Router with TypeScript strict mode.
Technology	LLM access only through HTTP APIs; no in-process model loading. Optional services must not be required for the core path.
Communication	JSON over HTTP with a common envelope; dashboard proxies <code>/api/backend/*</code> to the backend; bench pages call port 8000 directly.
Security	Secrets only via environment variables; staging/production start-up rejects the development secret, DEBUG and wildcard CORS.
Determinism	No wall-clock or Python <code>hash()</code> inside plan identity; UUIDv5 with a fixed namespace and a fixed epoch timestamp.
Programming standards	Ruff lint/format (line length 150), mypy configuration, ports-and-adapters layering, typed DTOs, ESLint with zero-warning gate.

2.10 Assumptions and dependencies

2.10.1 Assumptions

- The system is operated by trusted users on a private network or localhost (resource routes are unauthenticated).
- A single backend process owns the database and in-memory state.
- Goals are written in English; the planner's keyword tables are English.
- Users accept that agent outputs are heuristic; LLM text is advisory.

2.10.2 Dependencies

- Python packages declared in `backend/pyproject.toml` (FastAPI, Pydantic, SQLAlchemy, aiosqlite, httpx, tenacity, structlog, passlib, prometheus-client, qdrant-client, redis, typer, alembic, ...). LangChain/LangGraph and the official OpenAI/Anthropic/Google SDKs are declared but **not imported** by any module — providers are implemented with httpx.
- npm packages in `frontend/package.json` (Next 14.2.18, React 18.3.1, TanStack Query 5, Zod 3, Recharts 2, sonner, Tailwind 3.4).
- Ollama for local inference; API keys for hosted providers.
- Benchmark corpora downloaded at run time (HumanEval/MBPP), with bundled stubs as fallback.

2.11 System boundaries

Table 2.8: System boundary

Inside the boundary	Outside the boundary
---------------------	----------------------

Inside the boundary	Outside the boundary
FastAPI application, kernel, agents, memory, tools, LLM adapters, SQL schema, dashboard, CLI, benchmark scripts	LLM inference engines, hosted model APIs, Qdrant/Redis servers, web sites fetched by tools, Prometheus server, container runtime, CI runners

3 System Architecture

3.1 High-level architecture

NOESIS is organised in five layers over a storage layer (Figure 3.1). Dependencies point downwards only: the API layer calls services, services call agents and the kernel, and adapters isolate persistence and network access behind abstract ports [36].

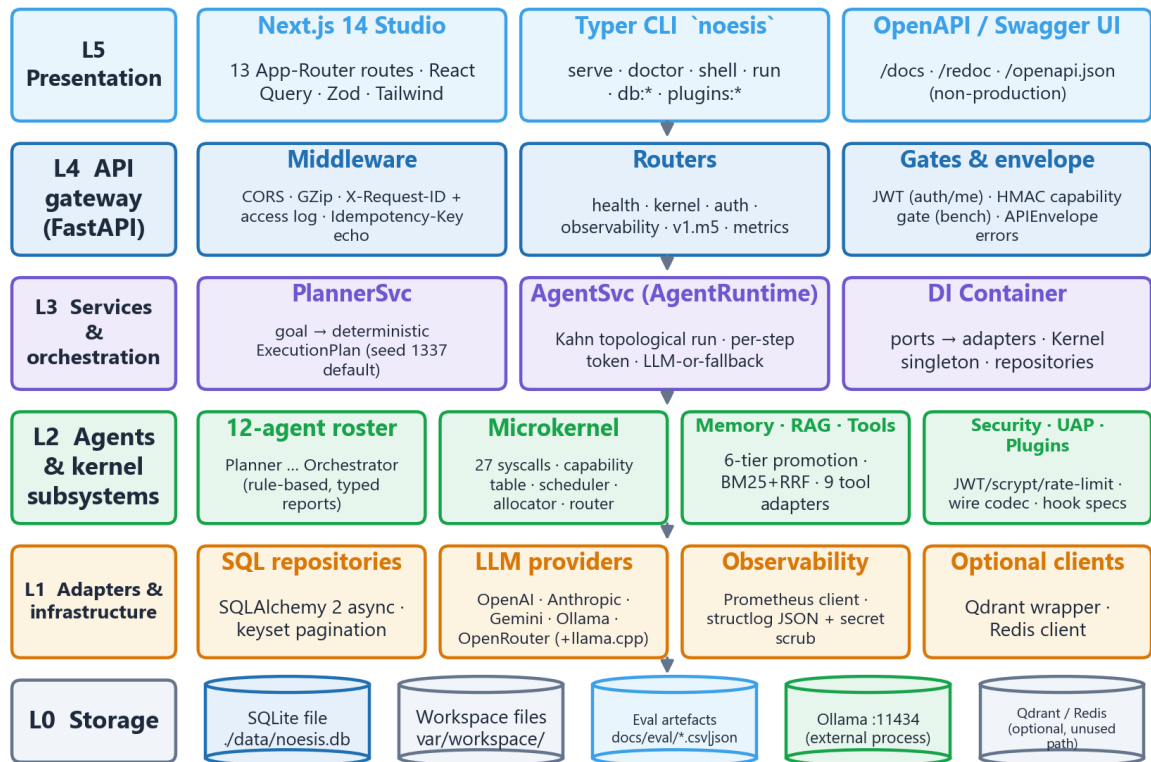


Figure 3.1: Layered architecture of NOESIS (as built)

L5 contains the user-facing entry points. L4 is the FastAPI application created by `create_app()` with its middleware, routers, exception handlers and two security gates. L3 holds the services that implement planning and execution plus the dependency-injection container. L2 contains the domain subsystems: agents, kernel, memory, retrieval, tools, security, protocol and plugins. L1 contains the adapters to SQL, LLM APIs, metrics and optional stores. L0 lists the stores actually touched in the verified configuration.

3.2 Component architecture

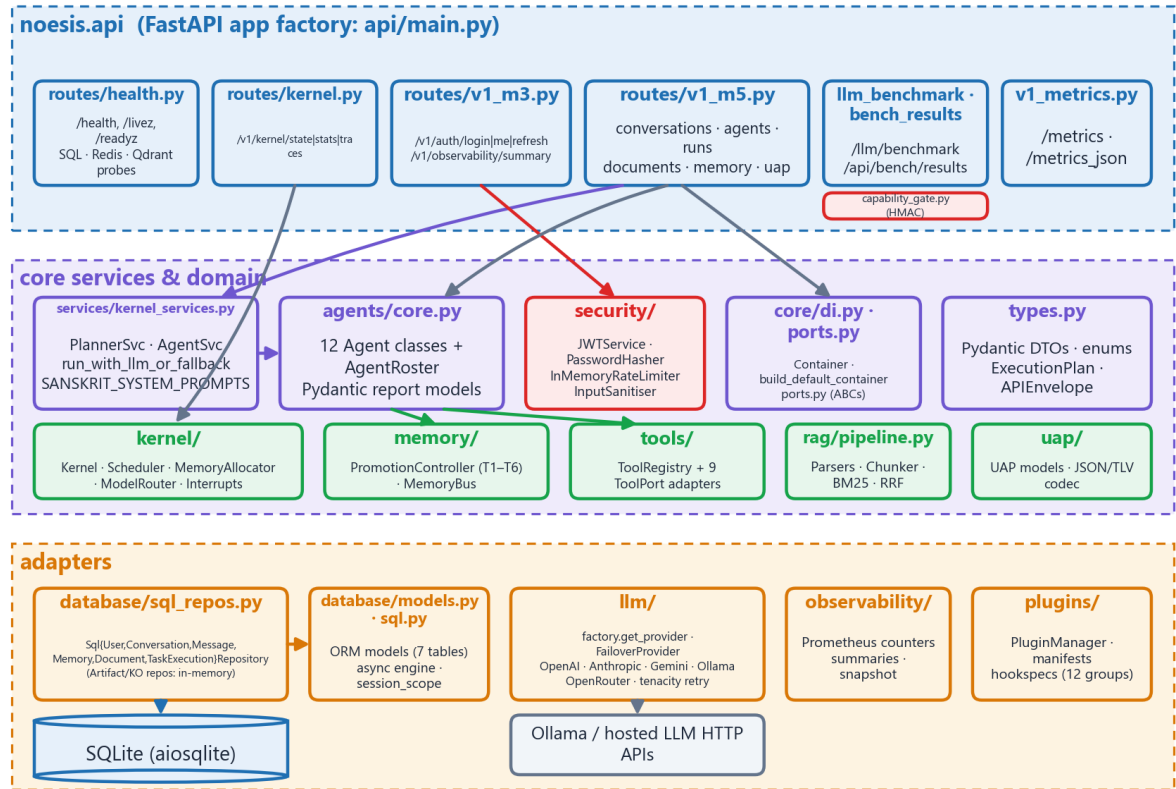


Figure 3.2: Component diagram: backend modules and their principal dependencies

Routers in `noesis.api.routes` call `build_services_from_container`, the agent classes, the security services or the kernel directly. The `capability_gate` dependency protects the two benchmark routers. Services depend on agents, which depend on memory and tools. Repositories map DTOs to ORM rows; LLM adapters wrap HTTP APIs.

Table 3.1: Backend packages (measured; 17,072 non-blank, non-comment lines)

Package	Files	LOC	Classes	Functions	Responsibility
api	14	2,341	30	93	FastAPI factory, middleware, routers, dependency helpers, capability gate
agents	2	1,850	32	48	Twelve agents, report models, AgentRoster
(root)	5	1,740	42	44	types.py, config.py, logging.py, cli.py, __init__.py
kernel	7	1,729	34	73	Kernel, syscalls, capabilities, scheduler, allocator, model router, interrupts
llm	11	1,650	15	76	Provider base, factory/failover, five adapters, retry, prompt registry, structured-output parser
database	6	1,190	19	94	Async engine/session, ORM models, repositories, Qdrant and Redis clients
benchmarks	2	1,083	4	21	pass@k, McNemar, Wilcoxon, bootstrap/Wilson CIs, aggregation to JSON tables
memory	4	1,068	19	66	Tier port + adapters, MemoryBus (RRF), PromotionController
tools	1	1,062	13	65	ToolRegistry + nine tool adapters
uap	2	802	13	59	Protocol models, JSON/TLV codecs
services	2	600	3	15	PlannerSvc, AgentSvc, LLM-or-fallback
plugins	5	585	8	55	Manifest, hookspecs, manager, capability bridge
security	1	422	10	24	JWT, password hashing, rate limiters, sanitiser

Package	Files	LOC	Classes	Functions	Responsibility
core	3	388	20	55	Ports (ABCs) and DI container
rag	2	357	13	20	Parsers, chunker, BM25 store, hybrid search
observability	2	205	1	9	Prometheus counters/summaries and JSON snapshot

3.3 Module architecture

The repository is a monorepo with a Python backend, a TypeScript frontend, documentation and evaluation artefacts. Table 3.2 lists the architecturally significant modules; a full map is in Appendix E.

Table 3.2: Architecturally significant modules

Module / file	Key elements	Notes
api/main.py	create_app(), lifespan(), 4 exception handlers, request-id middleware	Boots DI → Kernel → plugins → DB; in-place SQLite column migrations
api/routes/v1_m5.py	23 operations: conversations, agents, runs, documents, memory, UAP	Background run task inside create conversation
api/routes/v1_m3.py	login/me/refresh, observability summary, require_auth()	Built-in user directory (admin, demo)
services/kernel_services.py	PlannerSvc, AgentSvc, run_with_llm or fallback, SANSKRIT_SYSTEM_PROMPTS	Sync agent loop off-loaded with asyncio.to_thread
agents/core.py	12 Agent subclasses, 12 report models, AgentRoster	2 105 lines; all rule-based
kernel/kernel.py	Kernel, SysCall (27), AgentContext, AgentHandle, span buffer	6 default syscall handlers
memory/promotion.py	PromotionController, thresholds, SHA-256 chain, SQL save/load	save/load hooks not wired
tools/__init__.py	ToolRegistry, Shell, PythonSandbox, Files, GitHub, Weather, Calendar, Email, DB, WebFetch	4 tools in the default registry
llm/factory.py	get_provider, FailoverProvider, get_default_local_ollama	Ollama probe per services build
database/sql_repos.py	8 repository adapters, keyset pagination	Artifact/KnowledgeObject repos in memory
frontend/src/lib/api/*.ts	requestJson, endpoints, bench client, Zod schemas	Envelope mismatch for several list endpoints

3.4 Deployment architecture

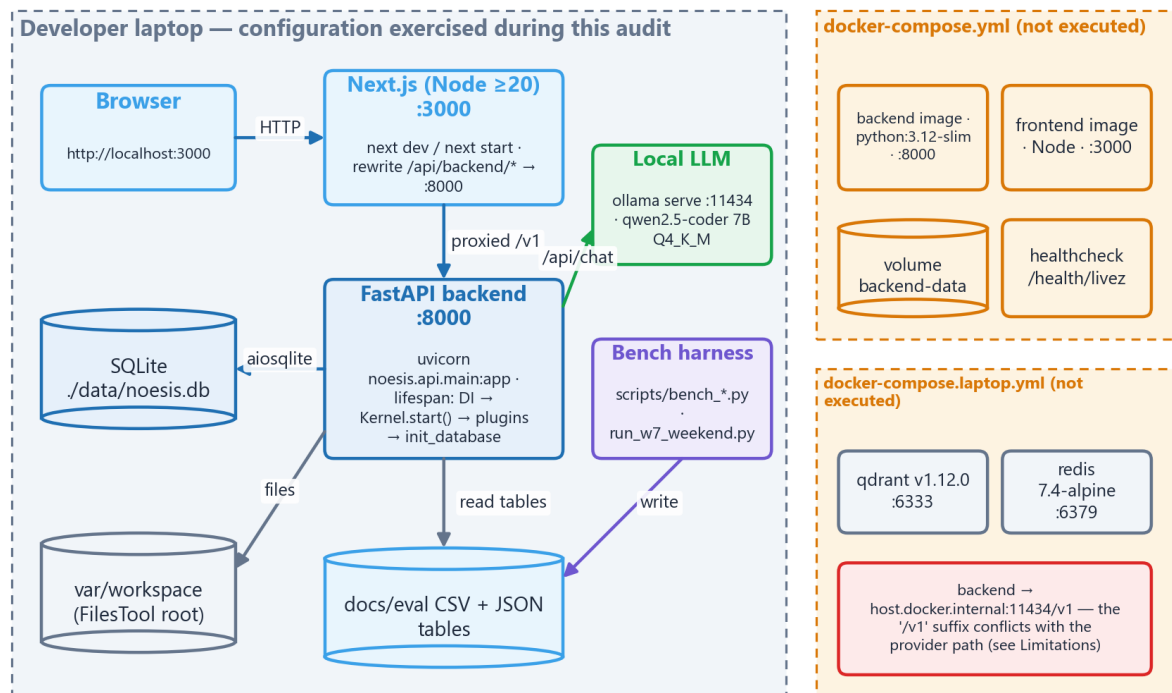


Figure 3.3: Deployment architecture: verified laptop configuration and the (unexecuted) container definitions

The verified configuration runs the Next.js server on port 3000, the FastAPI backend on port 8000 with a SQLite file, a workspace directory for the files tool, and optionally Ollama on port 11434. Two Compose files describe container deployments; they were not executed during this audit, and configuration analysis found two issues highlighted in red (Chapter 18).

3.5 Data architecture

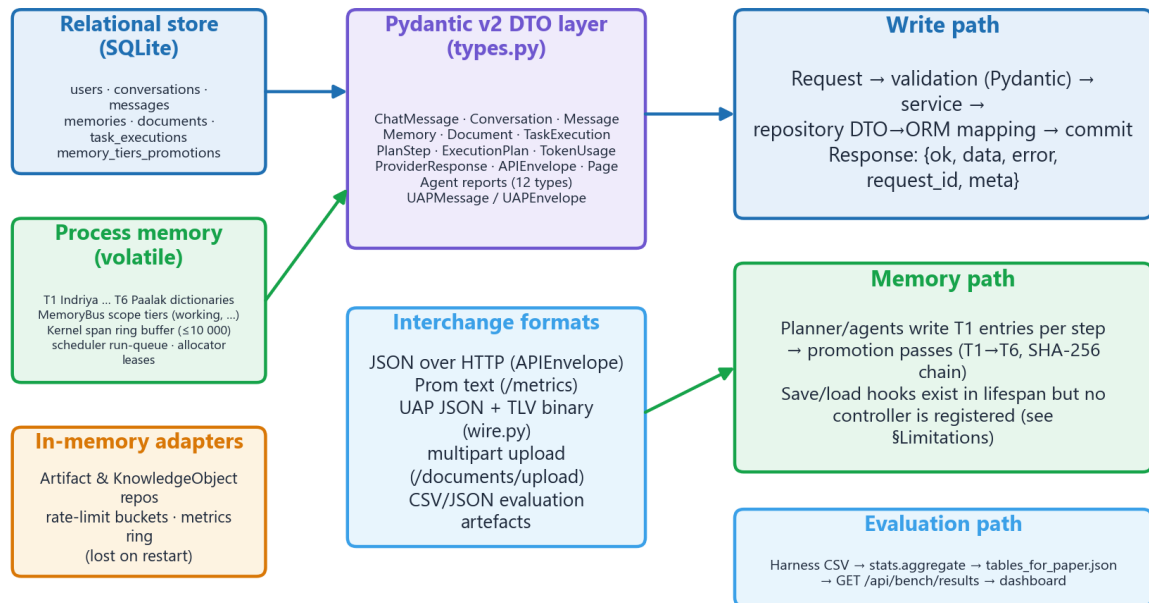


Figure 3.4: Data architecture: durable, volatile and interchange data

Durable state lives in SQLite. Kernel spans, memory tiers, rate-limit buckets, metric rings and two repositories live only in process memory and are lost on restart. All boundaries use Pydantic DTOs; interchange formats are JSON envelopes, Prometheus text, UAP JSON/TLV, multipart uploads and evaluation CSV/JSON.

3.6 Communication architecture

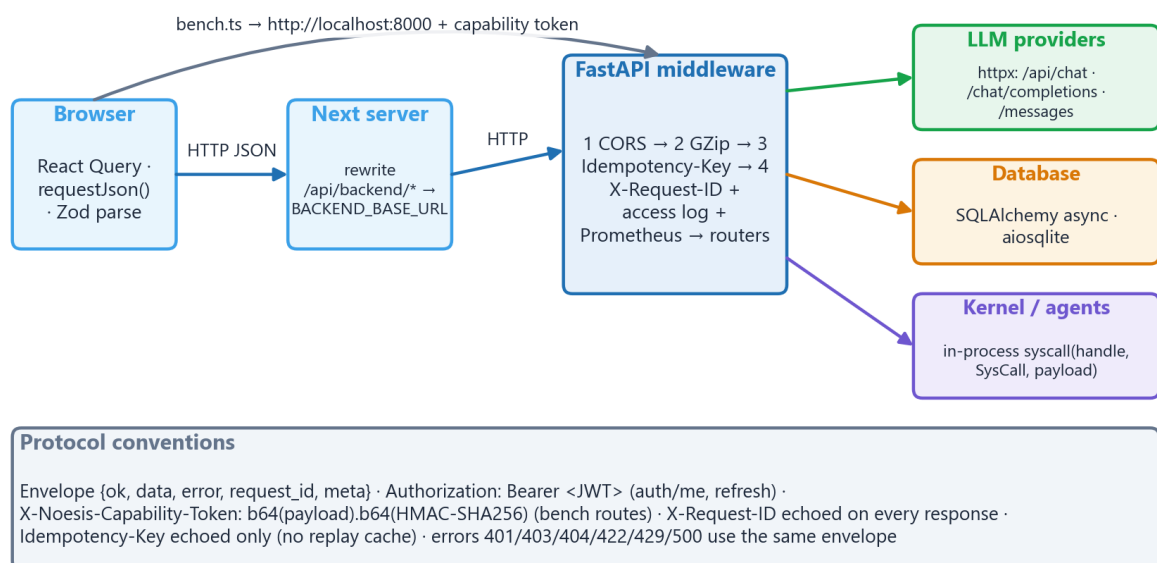


Figure 3.5: Communication architecture and protocol conventions

Browser requests flow through the Next.js rewrite to the backend middleware chain. Benchmark pages bypass the proxy and call the backend directly with a development capability token. The backend talks to LLM providers over HTTPS/HTTP with httpx and to the database through SQLAlchemy; agents talk to the kernel in-process.

3.7 Data-flow models

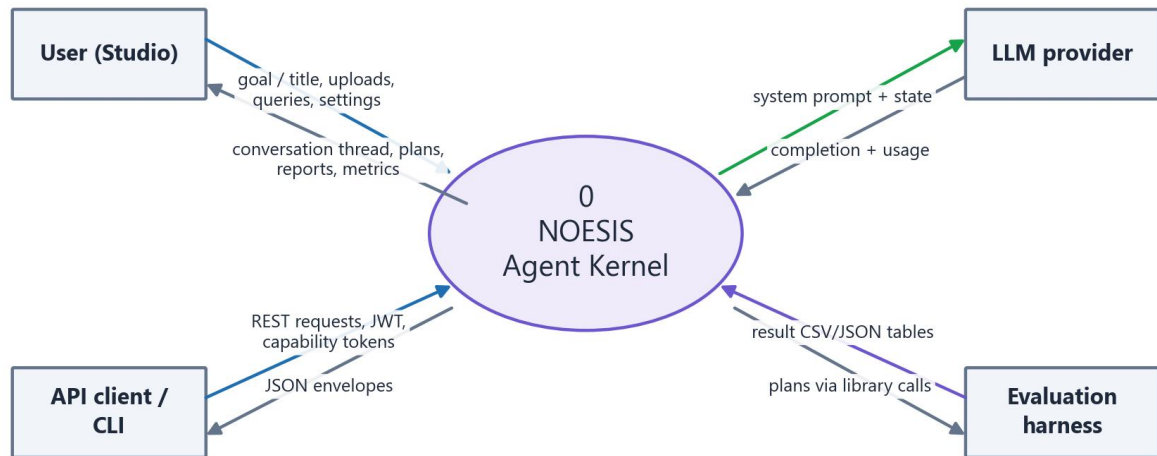


Figure 3.6: Data-flow diagram, level 0

At the highest level NOESIS exchanges goals, queries and tokens with users and clients, prompts and completions with LLM providers, and result tables with the evaluation harness.

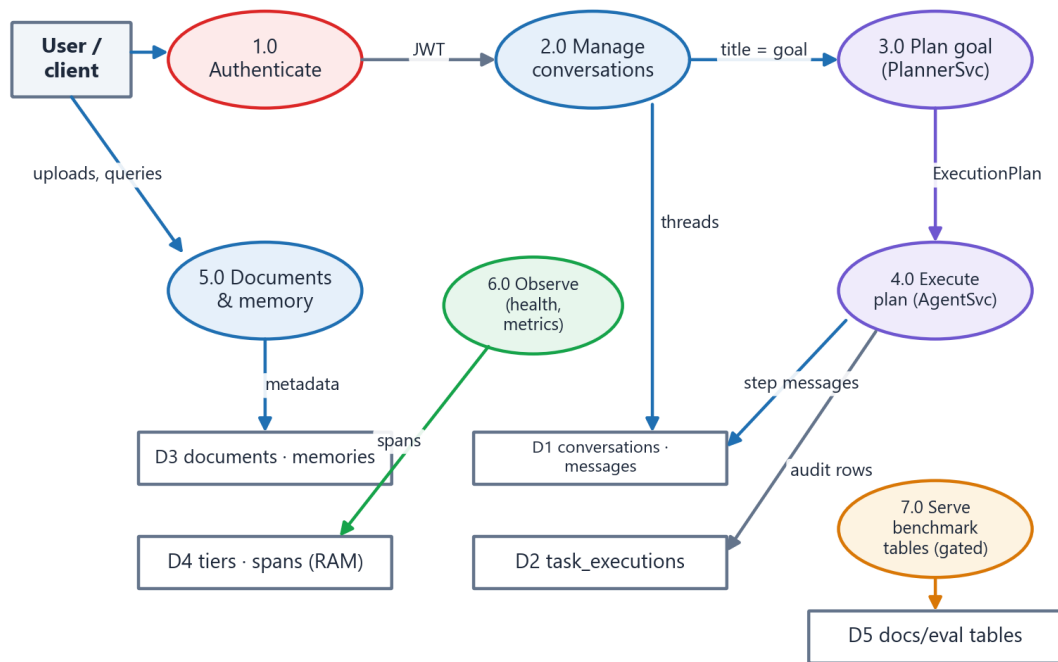


Figure 3.7: Data-flow diagram, level 1

Seven processes decompose the system: authentication, conversation management, planning, execution, document/memory management, observation and benchmark serving. Data stores D1–D5 correspond to SQL tables, in-memory structures and evaluation files.

3.8 Design diagrams

3.8.1 Class diagrams

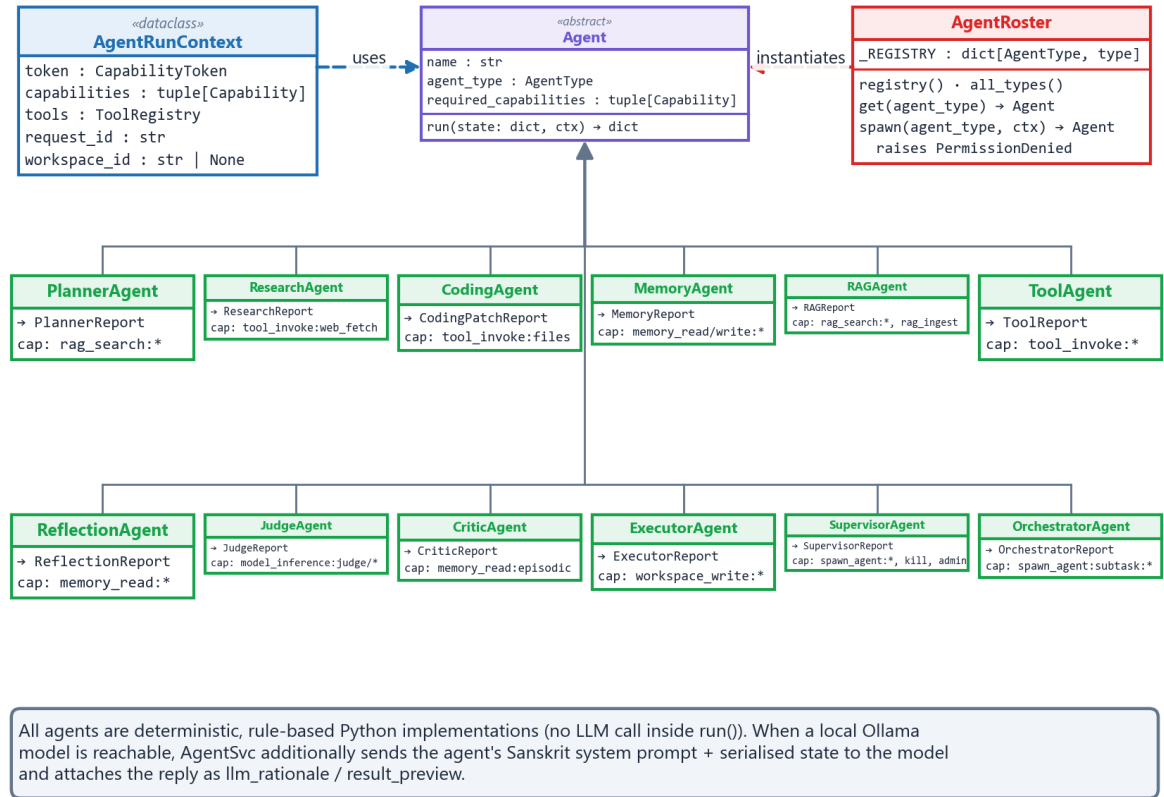


Figure 3.8: Class diagram of the agent subsystem

Every agent derives from the abstract `Agent`, declares `required_capabilities`, and implements `run(state, ctx)`. `AgentRunContext` carries the token, capability tuple and tool registry. `AgentRoster` maps each `AgentType` to a class and performs the spawn-time capability check.

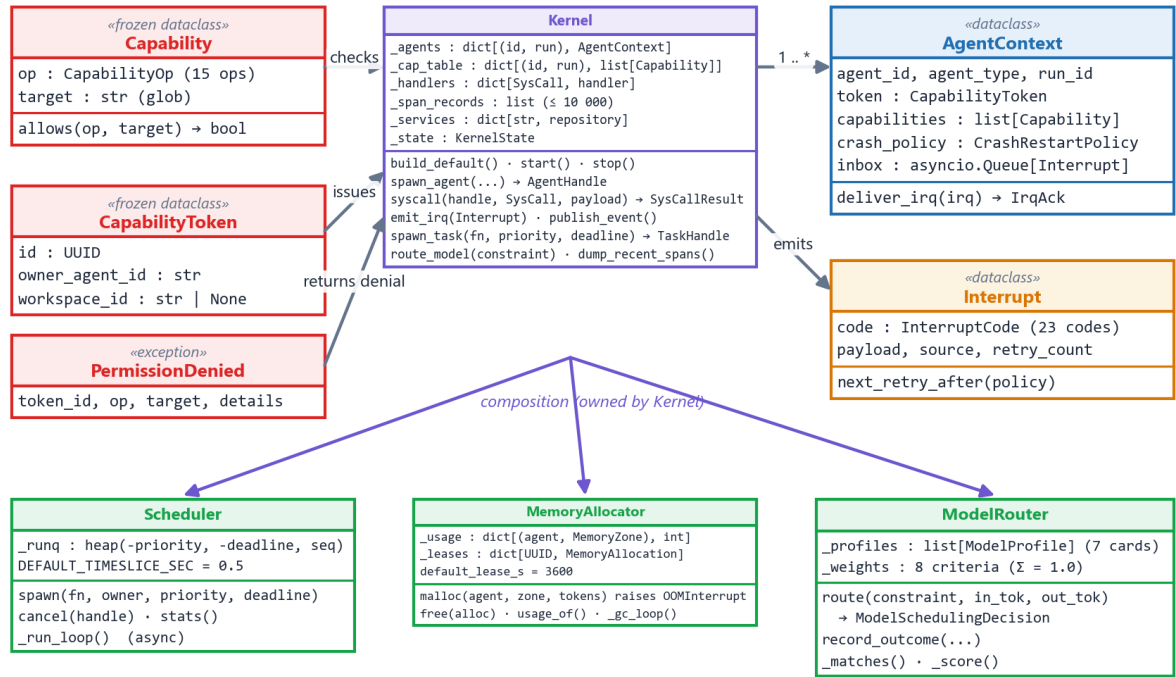


Figure 3.9: Class diagram of the kernel subsystem

The `Kernel` owns the capability table, agent contexts, syscall handlers and span buffer, and composes the `Scheduler`, `MemoryAllocator` and `ModelRouter`. Denials are represented by `PermissionDenied`; asynchronous events by `Interrupt` records delivered to agent inboxes.

3.8.2 Sequence diagrams

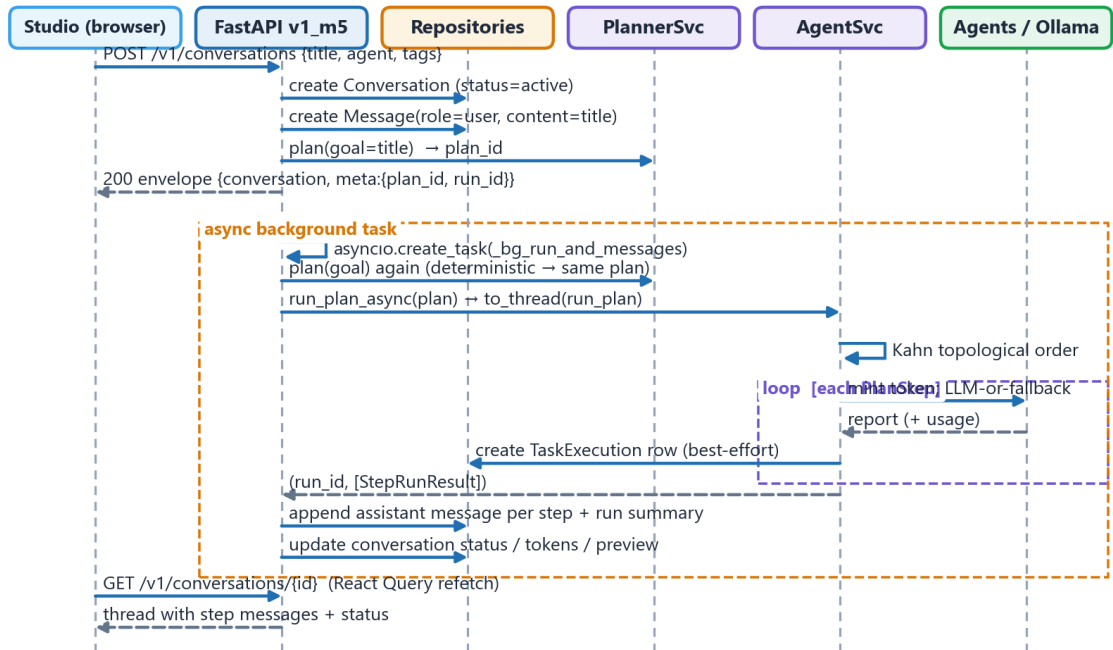


Figure 3.10: Sequence diagram: creating a conversation and running its plan in the background

The HTTP request returns as soon as the conversation and goal message are stored and a plan id is computed. A background task re-plans (deterministically), executes the plan in a worker thread, persists audit rows and messages, and finally updates the conversation status. The dashboard polls the thread to show progress.

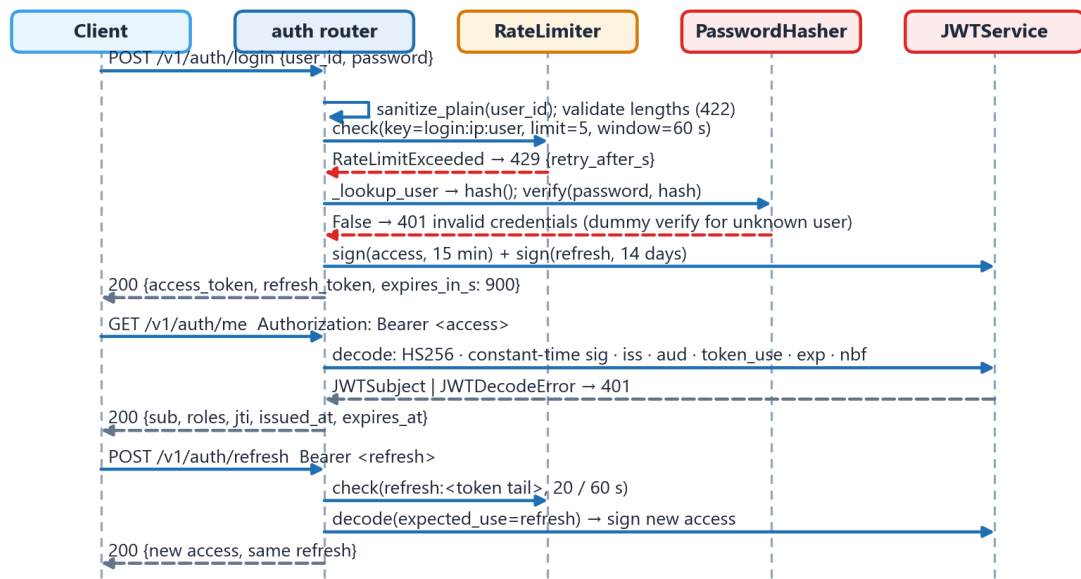


Figure 3.11: Sequence diagram: login, identity and refresh

Login applies the rate limit before password verification; tokens are verified with constant-time signature comparison and claim checks.

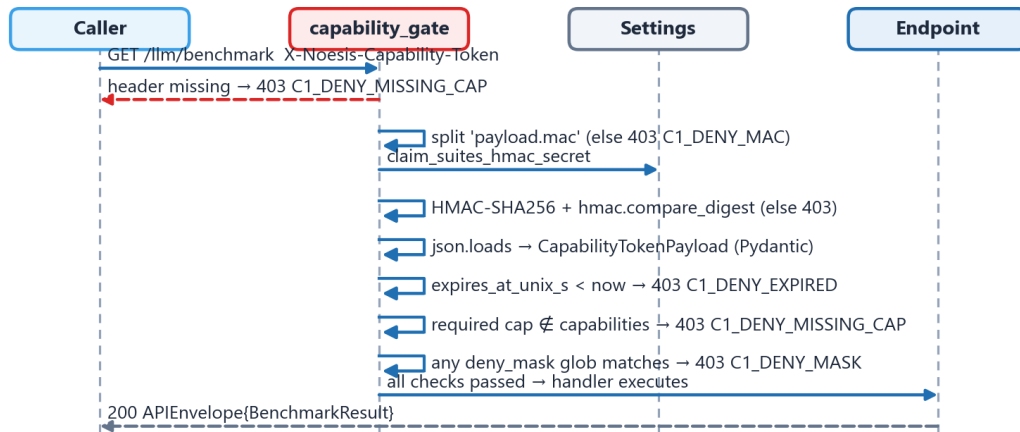
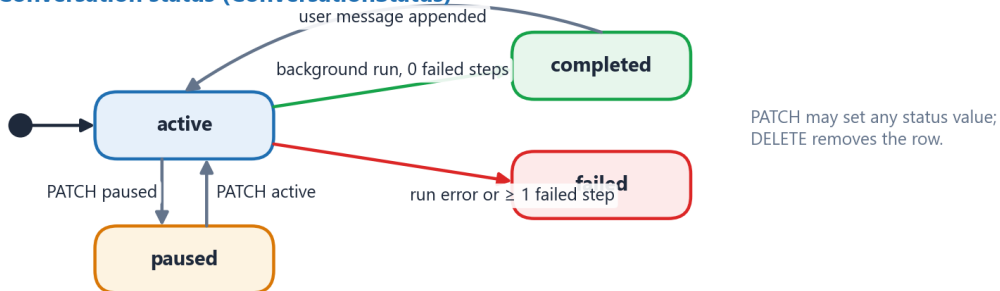


Figure 3.12: Sequence diagram: capability-token verification on a gated route

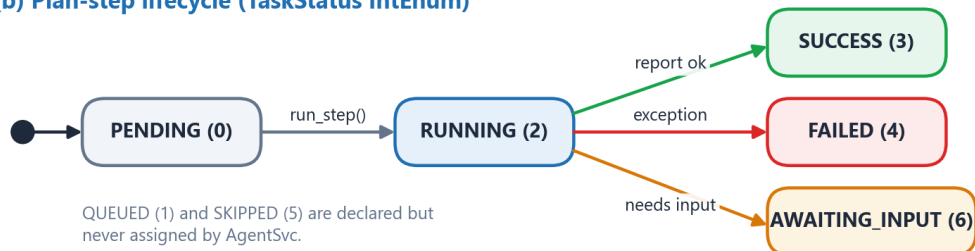
Each of the five checks maps to a distinct 403 reason code; only a request passing all checks reaches the handler.

3.8.3 State diagrams

(a) Conversation status (ConversationStatus)



(b) Plan-step lifecycle (TaskStatus IntEnum)



(c) Kernel lifecycle (KernelState)

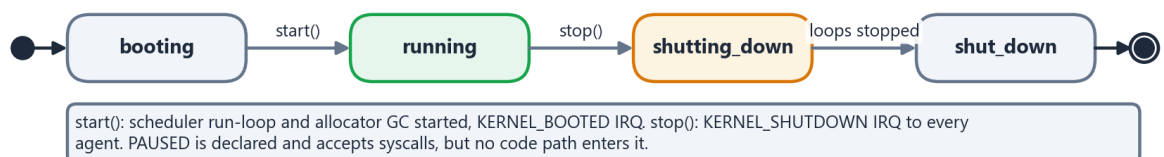


Figure 3.13: State diagrams for conversations, plan steps and the kernel

Conversation status is driven by the background run and by PATCH requests; step status by AgentSvc; kernel state by the application lifespan. Declared-but-unused states are noted.

4 External Interface Requirements

4.1 User interfaces

The dashboard (Noesis Studio) is a dark-themed single-page application with a persistent sidebar, a top bar and a content area. Chapter 14 specifies each screen with real screenshots. Figure 4.1 summarises the routes and whether they show live backend data.

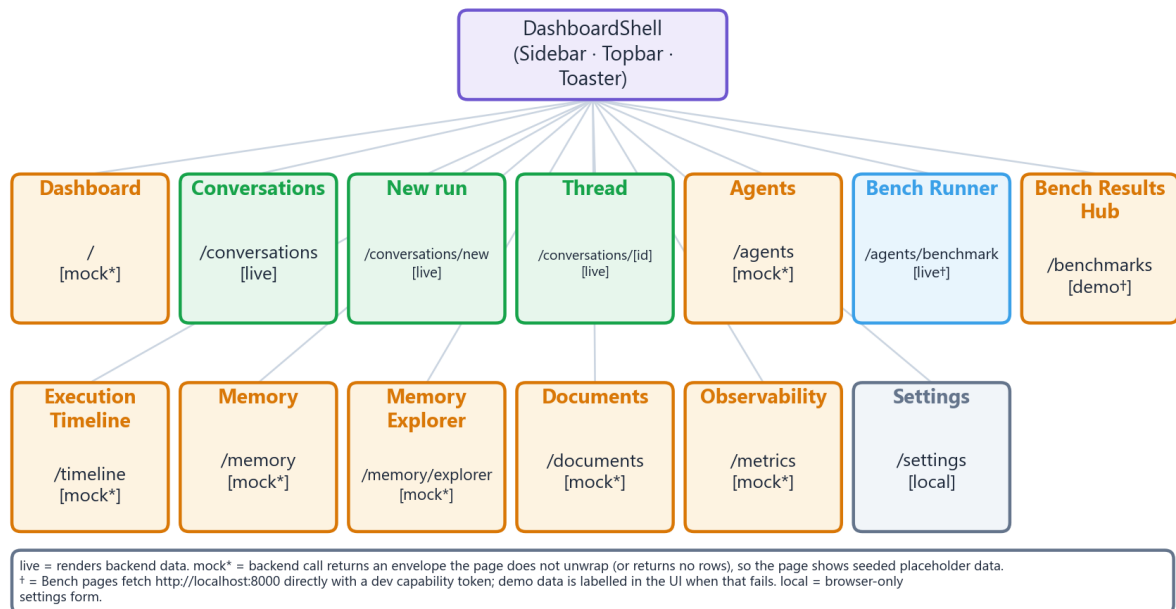


Figure 4.1: Dashboard site map and data source of each route

Only the three conversation routes render live data in the current build. Six monitoring pages request backend data but receive envelope-wrapped responses that their helpers do not unwrap, so they show seeded placeholder data. The two benchmark pages call port 8000 directly and label any fallback data in the UI.

4.2 Software interfaces

Table 4.1: Software interfaces

Interface	Direction	Protocol / format	Purpose
NOESIS REST API	Clients → backend	HTTP/1.1, JSON, OpenAPI 3.1	42 operations on 36 paths (Chapter 10)
Ollama	Backend → Ollama	HTTP JSON: GET /api/tags, POST /api/chat, /api/embeddings	Local chat completions and embeddings
OpenAI-compatible	Backend → provider	HTTPS JSON: POST /chat/completions, /embeddings	OpenAI and llama.cpp server

Interface	Direction	Protocol / format	Purpose
Anthropic	Backend → provider	HTTPS JSON: POST /v1/messages	Claude models
Google Gemini	Backend → provider	HTTPS JSON: POST v1beta/models/{model};generateContent	Gemini models
OpenRouter	Backend → provider	OpenAI-compatible HTTPS	Model aggregator
SQL database	Backend → DB	SQLAlchemy 2 async (aiosqlite / asyncpg-ready URL)	Durable state
Qdrant / Redis	Backend → service	qdrant-client / redis-py	Readiness probes only
Prometheus	Prometheus → backend	GET /metrics text exposition v0.0.4	Metrics scrape
Benchmark artefacts	Harness ↔ backend	CSV and JSON files under EVAL ROOT	Results publication

4.3 Hardware interfaces

NOESIS has no direct hardware interfaces. GPU acceleration, when present, is used transparently by the external LLM runtime (Ollama). The synopsis-stage target of an NVIDIA Jetson Orin Nano deployment is **not implemented** and was not verified.

4.4 Communication interfaces and protocols

Table 4.2: Communication interfaces

Protocol element	Specification
Transport	HTTP on localhost in the verified configuration; HTTPS is expected to be provided by a reverse proxy (not part of the code).
Response envelope	{ok: bool, data: T null, error: str null, request_id: str null, meta: {}} for all routes except /auth/*, /observability/summary and /metrics.
Correlation	X-Request-ID accepted or generated and echoed; Idempotency-Key echoed (no replay store).
Compression	GZip for responses ≥ 1 KiB.
CORS	Explicit origin list (default http://localhost:3000), credentials allowed, wildcard rejected at start-up.
User authentication	JWT HS256 [31] in Authorization: Bearer header.
Route capability token	X-Noesis-Capability-Token: b64url(payload).b64url(HMAC-SHA256) [30].
Inter-agent protocol	UAP v1 models with JSON (sorted keys) and TLV binary encodings mirroring schemas/uap/v1/uap.proto.
Data volumes	Typical JSON responses 0.3–10 KB; plan responses 3–8 KB; uploads unbounded (read fully into memory).

5 Functional Requirements

This chapter states 65 functional requirements grouped by function area. Each requirement records its actor, preconditions, inputs, processing, outputs, postconditions, alternate and exception flows, dependencies, priority, acceptance criteria, implementation status and the code and tests that realise it. Table 5.1 is the summary index.

Table 5.1: Functional requirements — summary index

ID	Requirement	Priority	Status
----	-------------	----------	--------

ID	Requirement	Priority	Status
FR-001	User login and token issuance	High	Implemented
FR-002	Identity introspection (/auth/me)	High	Implemented
FR-003	Access-token refresh	Medium	Implemented
FR-004	Production secret enforcement	High	Implemented
FR-005	Role-based authorisation on resource routes	High	Planned (not implemented)
FR-006	Create conversation from a goal	High	Implemented
FR-007	List, filter and search conversations	High	Implemented
FR-008	Read a conversation thread	High	Implemented
FR-009	Append message to a conversation	Medium	Implemented
FR-010	Update conversation (title, status, tags)	Medium	Implemented
FR-011	Delete conversation	Medium	Implemented
FR-012	List messages of a conversation	Low	Implemented
FR-013	Deterministic goal decomposition	High	Implemented
FR-014	Topological plan execution	High	Implemented
FR-015	Execution audit trail	Medium	Implemented
FR-016	List executions of a run	Medium	Partially implemented
FR-017	Background run completion messages	High	Implemented
FR-018	Agent catalogue	Medium	Implemented
FR-019	Direct single-agent invocation	Medium	Implemented
FR-020	Spawn-time capability gating of agents	High	Implemented
FR-021	Twelve typed agents	High	Implemented
FR-022	Research agent	Medium	Implemented
FR-023	Coding agent	Medium	Implemented
FR-024	Reflection, judge and critic agents	Medium	Implemented
FR-025	Executor tri-state decision	High	Implemented
FR-026	Memory, RAG, tool, supervisor and orchestrator agents	Low	Stub / simulated
FR-027	Register document metadata	Medium	Implemented
FR-028	Upload document file	Medium	Stub / simulated
FR-029	List, get and delete documents	Medium	Implemented
FR-030	List memory records	Medium	Partially implemented
FR-031	Query memory records	Low	Partially implemented
FR-032	Compress stale memory	Low	Implemented
FR-033	Six-tier memory promotion with provenance	High	Partially implemented
FR-034	Tier persistence across restarts	Medium	Planned (not implemented)
FR-035	Multi-tier recall bus	Low	Implemented as library (not wired to API)
FR-036	Kernel lifecycle and state	Medium	Implemented
FR-037	Capability-checked system calls	High	Implemented
FR-038	HMAC capability gate on HTTP routes	High	Implemented
FR-039	Kernel statistics and traces	Low	Implemented
FR-040	LLM enrichment with deterministic fallback	High	Implemented
FR-041	Scheduler, allocator, router and interrupts	Medium	Implemented as library (not wired to API)
FR-042	Capability-gated tool registry	High	Implemented
FR-043	Default sandboxed tools	High	Implemented
FR-044	Additional tool adapters	Low	Implemented as library (not wired to API)
FR-045	Pluggable LLM providers	High	Implemented
FR-046	Provider failover	Medium	Implemented
FR-047	Embedding providers	Low	Implemented as library (not wired to API)

ID	Requirement	Priority	Status
FR-048	Liveness and readiness probes	High	Implemented
FR-049	Prometheus and JSON metrics	Medium	Partially implemented
FR-050	Observability summary	Low	Stub / simulated
FR-051	Request correlation and structured logs	Medium	Implemented
FR-052	LLM throughput benchmark endpoint	Medium	Implemented
FR-053	Benchmark results API	Medium	Implemented
FR-054	Offline evaluation harness	Medium	Partially implemented
FR-055	UAP message model and wire codecs	Low	Implemented as library (not wired to API)
FR-056	UAP HTTP endpoints	Low	Stub / simulated
FR-057	Plugin discovery and lifecycle	Low	Implemented
FR-058	Command-line interface	Low	Implemented
FR-059	Dashboard shell and navigation	Medium	Implemented
FR-060	Conversation workflow in the UI	High	Implemented
FR-061	Monitoring pages (dashboard, agents, timeline, memory, documents, observability)	Medium	Stub / simulated
FR-062	Benchmark hub and runner pages	Medium	Partially implemented
FR-063	Settings page	Low	Stub / simulated
FR-064	Document parsing and chunking	Medium	Implemented as library (not wired to API)
FR-065	Hybrid lexical search	Medium	Implemented as library (not wired to API)

5.1 Authentication and session tokens

5.1.1 FR-001 User login and token issuance

Requirement ID	FR-001
Name	User login and token issuance
Description	The system shall authenticate a user identifier and password and return a signed HS256 access token and refresh token pair.
Actor	Dashboard user, API client
Preconditions	Backend running; user exists in the built-in directory (<i>admin</i> or <i>demo</i>).
Inputs	JSON body { <i>user_id</i> (2–128 chars), <i>password</i> (8–4096 chars)}; client IP (or X-Forwarded-For).
Processing	1. Strip whitespace and reject unknown fields (Pydantic <i>extra=forbid</i>). 2. Sanitise <i>user_id</i> with <code>InputSanitiser.sanitize_plain</code>. 3. Apply sliding-window rate limit key <code>login:<client>:<user></code> (5 attempts / 60 s). 4. Look up the user in <code>_lookup_user</code>; for unknown users perform a dummy hash verification to equalise timing. 5. Verify the password with <code>PasswordHasher</code> (bcrypt via <code>passlib</code>, <code>scrypt</code> fallback). 6. Sign access (15 min) and refresh (14 days) JWTs with roles, <code>jti</code>, <code>iss noesis:<env></code>, <code>aud noesis:api</code>.
Outputs	200 TokenPair { <i>access_token</i> , <i>refresh_token</i> , <i>token_type</i> 'Bearer', <i>expires_in_s</i> 900}.
Postconditions	Tokens can be presented to <code>/v1/auth/me</code> and <code>/v1/auth/refresh</code> .
Alternate flow	Admin login uses <code>ADMIN_PASSWORD</code> when the attribute exists on Settings; otherwise the built-in default password applies (see Security).
Exception flow	422 validation error; 401 invalid credentials (with WWW-Authenticate: Bearer); 429 with <code>meta.retry_after_s</code> when the limit is exceeded.
Dependencies	<code>security.JWTService</code> , <code>PasswordHasher</code> , <code>InMemoryRateLimiter</code>
Priority	High
Acceptance criteria	1. Valid demo credentials return 200 with two three-segment JWTs. 2. Wrong password returns 401 'invalid credentials'. 3. Sixth attempt in 60 s from the same client/user returns 429 (measured: 401×5 then 429).

Implementation status	Implemented
Realised in	api/routes/v1_m3.py: login(); security/__init__.py
Verified by	test_m3::test_auth_login_demo_user_returns_token_pair, test_auth_login_bad_password_is_401, test_auth_login_rate_limit_blocks_same_ip_user

5.1.2 FR-002 Identity introspection (/auth/me)

Requirement ID	FR-002
Name	Identity introspection (/auth/me)
Description	The system shall return the subject, roles, token id and validity window of a presented access token.
Actor	Authenticated client
Preconditions	Client holds an access token.
Inputs	Header <i>Authorization</i> : Bearer <access JWT>.
Processing	1. Extract bearer token (else 401 'missing bearer token'). 2. Decode: header alg must be HS256; constant-time signature compare; check iss, aud, token_use='access', exp, nbf, sub type.
Outputs	200 {sub, roles[], jti, issued_at, expires_at, token_use}.
Postconditions	None (read-only).
Alternate flow	—
Exception flow	401 for missing, malformed, tampered, expired or wrong-use (refresh) tokens.
Dependencies	FR-001
Priority	High
Acceptance criteria	1. Valid token returns its subject. 2. Tampered signature returns 401 'jwt signature mismatch' (measured). 3. Refresh token returns 401 "jwt token_use must be 'access'" (measured).
Implementation status	Implemented
Realised in	api/routes/v1_m3.py: me(), require_auth()
Verified by	test_m3::test_auth_me_returns_subject, test_auth_me_missing_bearer_returns_401, test_jwt_tampered_sig_fails, test_jwt_wrong_use_fails

5.1.3 FR-003 Access-token refresh

Requirement ID	FR-003
Name	Access-token refresh
Description	The system shall issue a new access token in exchange for a valid refresh token.
Actor	Authenticated client
Preconditions	Client holds a refresh token.
Inputs	Header <i>Authorization</i> : Bearer <refresh JWT>.
Processing	1. Rate-limit on the last 10 characters of the token (20 / 60 s). 2. Decode with expected_use='refresh'. 3. Sign a new access token carrying refresh_jti; return the same refresh token.
Outputs	200 TokenPair with new access_token and unchanged refresh_token.
Postconditions	Old access tokens remain valid until expiry (no revocation list).
Alternate flow	—
Exception flow	401 invalid token; 429 rate limit.
Dependencies	FR-001
Priority	Medium
Acceptance criteria	Refresh returns a different access token for the same subject.
Implementation status	Implemented
Realised in	api/routes/v1_m3.py: refresh()

Verified by	test_m3::test_auth_refresh_issues_new_access
-------------	--

5.1.4 FR-004 Production secret enforcement

Requirement ID	FR-004
Name	Production secret enforcement
Description	The system shall refuse to start in staging/production with the built-in development SECRET_KEY, with DEBUG enabled, or with a wildcard CORS origin.
Actor	Operator
Preconditions	Process start-up.
Inputs	Environment variables APP_ENV, SECRET_KEY, DEBUG, CORS_ORIGINS.
Processing	Pydantic Settings model validator <code>_debug_for_dev_only</code> runs on <code>Settings()</code> construction.
Outputs	Validation error at start-up (process exits) or a valid Settings object.
Postconditions	Application only starts with acceptable configuration.
Alternate flow	—
Exception flow	ValueError describing the violated rule.
Dependencies	config.py
Priority	High
Acceptance criteria	1. APP_ENV=staging with default secret raises. 2. CORS '*' raises.
Implementation status	Implemented
Realised in	config.py: Settings._debug_for_dev_only
Verified by	test_config::test_debug_forbidden_in_production, test_config_edges::test_cors_wildcard_rejected, test_staging_requires_explicit_secret

5.1.5 FR-005 Role-based authorisation on resource routes

Requirement ID	FR-005
Name	Role-based authorisation on resource routes
Description	Resource routes (/v1/conversations, /v1/runs, /v1/documents, /v1/memory, /v1/agents, /v1/kernel, /v1/observability) should require an authenticated principal and appropriate role.
Actor	All
Preconditions	—
Inputs	—
Processing	require_auth(require_any_role=...) exists but is applied only to /v1/auth/me.
Outputs	—
Postconditions	—
Alternate flow	—
Exception flow	—
Dependencies	FR-001
Priority	High
Acceptance criteria	Unauthenticated calls to resource routes are rejected (NOT met: they currently succeed).
Implementation status	Planned (not implemented)
Realised in	api/routes/v1_m3.py: require_auth()
Verified by	test_m3::test_observability_summary_requires_admin_role (marked xfail: guard intentionally removed)

5.2 Conversations and messages

5.2.1 FR-006 Create conversation from a goal

Requirement ID	FR-006
Name	Create conversation from a goal
Description	The system shall create a conversation whose title is the user's goal, persist the goal as the first user message, compute a deterministic plan id, and start the multi-agent run in the background.
Actor	Dashboard user, API client
Preconditions	Database initialised.
Inputs	JSON {title (1–200), agent='orchestrator', tags[], user_id?}.
Processing	1. Create Conversation with metadata {status: active, agent, tags, message_count 1, token_count 0, cost_usd 0, preview}. 2. Insert Message(role=user, content=title, metadata.kind='user_goal'). 3. PlannerSvc.plan(goal=title) → plan_id (random UUID on planner failure). 4. Store plan_id and a new run_id in metadata. 5. Schedule _bg_run_and_messages() with asyncio.create_task and return immediately.
Outputs	200 envelope {data: ConversationEnriched, meta: {plan_id, run_id}}.
Postconditions	Background task runs FR-014 and appends messages (FR-009).
Alternate flow	If message insertion fails the error is logged and creation still succeeds.
Exception flow	422 when title is missing/too long; 500 on database errors.
Dependencies	FR-013, FR-014
Priority	High
Acceptance criteria	1. Response returns within ~1 s while the run continues asynchronously (measured 0.82 s with LLM, 1.73 s deterministic incl. provider probe). 2. Thread reaches status completed/failed and contains one assistant message per step plus a run summary (measured 7 messages for a 5-step plan).
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: create_conversation(), _bg_run_and_messages()
Verified by	test_m5::test_conversations_create_and_list, test_conversations_crud_lifecycle

5.2.2 FR-007 List, filter and search conversations

Requirement ID	FR-007
Name	List, filter and search conversations
Description	The system shall return a keyset-paginated list of conversations enriched with status, agent, preview, tags and counters, filterable by status, agent and free-text search.
Actor	Dashboard user, API client
Preconditions	—
Inputs	Query: cursor, limit (1–200, default 25), status, agent, search.
Processing	1. Page conversations via repository (id > cursor). 2. Load up to 20 messages per conversation for preview/token count. 3. Apply status/agent/search filters in memory (search over title, preview, tags).
Outputs	200 envelope {data: [ConversationEnriched], meta: {limit, next_cursor, total}}.
Postconditions	—
Alternate flow	—
Exception flow	422 on invalid limit.
Dependencies	FR-006
Priority	High
Acceptance criteria	Filters reduce the returned list; meta.total reflects the unfiltered page total.
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: list_conversations()
Verified by	test_m5::test_v1_conversations_list_has_agent_field_in_envelope

5.2.3 FR-008 Read a conversation thread

Requirement ID	FR-008
Name	Read a conversation thread
Description	The system shall return one conversation and (optionally) its first N messages.
Actor	Dashboard user, API client
Preconditions	Conversation exists.
Inputs	Path conversation_id (UUID); query include_messages (bool, default true), limit_messages (1–500, default 50).
Processing	Fetch conversation; fetch messages ordered by keyset; enrich.
Outputs	200 envelope with conversation fields and messages[].
Postconditions	—
Alternate flow	—
Exception flow	404 'Conversation <id> not found'; 422 invalid UUID.
Dependencies	FR-006
Priority	High
Acceptance criteria	Unknown id returns 404 in the standard envelope (measured).
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: get_conversation()
Verified by	test_m5::test_conversations_crud_lifecycle

5.2.4 FR-009 Append message to a conversation

Requirement ID	FR-009
Name	Append message to a conversation
Description	The system shall append a message with a role in {system, user, assistant, tool, observation, reflection} and update the conversation preview, counters and status.
Actor	Dashboard user, API client
Preconditions	Conversation exists.
Inputs	JSON {role, content (≥1 char), name?, tool_call_id?}.
Processing	1. Validate role literal. 2. Persist Message. 3. Increment message_count, set preview (first 160 chars), set status active.
Outputs	200 envelope with the stored Message.
Postconditions	Conversation status becomes active.
Alternate flow	—
Exception flow	404 unknown conversation; 422 invalid role/content.
Dependencies	FR-006
Priority	Medium
Acceptance criteria	Appending to a completed conversation sets status back to active.
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: append_message()
Verified by	test_m5::test_conversations_crud_lifecycle

5.2.5 FR-010 Update conversation (title, status, tags)

Requirement ID	FR-010
Name	Update conversation (title, status, tags)
Description	The system shall update title, status or tags of a conversation.

Actor	Dashboard user, API client
Preconditions	Conversation exists.
Inputs	JSON {title?, status?, tags?} (extra fields rejected).
Processing	Patch provided fields; set updated_at.
Outputs	200 envelope with the enriched conversation.
Postconditions	—
Alternate flow	—
Exception flow	404; 422.
Dependencies	FR-006
Priority	Medium
Acceptance criteria	Renamed title is returned (measured).
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: patch_conversation()
Verified by	test_m5::test_conversations_crud_lifecycle

5.2.6 FR-011 Delete conversation

Requirement ID	FR-011
Name	Delete conversation
Description	The system shall delete a conversation and cascade-delete its messages.
Actor	Dashboard user, API client
Preconditions	Conversation exists.
Inputs	Path conversation_id.
Processing	Repository delete (ORM cascade on messages; task_executions.conversation_id set NULL).
Outputs	200 {deleted: true}.
Postconditions	Conversation removed (hard delete; the route docstring's 'soft delete' is not what the code does).
Alternate flow	—
Exception flow	404 when absent.
Dependencies	FR-006
Priority	Medium
Acceptance criteria	Second delete returns 404.
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: delete_conversation()
Verified by	test_m5::test_conversations_crud_lifecycle

5.2.7 FR-012 List messages of a conversation

Requirement ID	FR-012
Name	List messages of a conversation
Description	The system shall page through messages of a conversation.
Actor	Dashboard user, API client
Preconditions	—
Inputs	Path conversation_id; cursor; limit.
Processing	Keyset pagination on message id filtered by conversation.
Outputs	200 envelope list + meta.
Postconditions	—
Alternate flow	—
Exception flow	422.
Dependencies	FR-009

Priority	Low
Acceptance criteria	limit=2 returns at most 2 messages (measured).
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: list_messages()
Verified by	test_repositories::test_message_list_for_conversation

5.3 Planning and plan execution

5.3.1 FR-013 Deterministic goal decomposition

Requirement ID	FR-013
Name	Deterministic goal decomposition
Description	The system shall decompose a goal (≥ 5 characters) into an ExecutionPlan of 3–12 worker steps plus a final reflection step, deterministically for a given (goal, seed).
Actor	API client, Conversation flow
Preconditions	—
Inputs	JSON {goal (≥ 5), seed?, conversation_id?}.
Processing	PlannerSvc mints a planner token and calls PlannerAgent.run (see Algorithm A1).
Outputs	200 envelope PlannerReport {goal, plan{id, steps[], reasoning, created_at, metadata.promotion_report}, seeds, required_capabilities}.
Postconditions	—
Alternate flow	Goals shorter than 5 characters yield a FAILED report with a single 'restate goal' step when invoked via the agent; the HTTP schema rejects them with 422.
Exception flow	422 for goal < 5 chars.
Dependencies	—
Priority	High
Acceptance criteria	1. Same goal and seed give byte-identical plan JSON (measured 50/50 SE-50 goals $\times$ 3 runs). 2. Planner time median 1.05 ms in-process (measured).
Implementation status	Implemented
Realised in	agents/core.py: PlannerAgent; services/kernel_services.py: PlannerSvc
Verified by	test_m5::test_planner_same_seed_produces_byte_identical_plan, test_planner_run_always_appends_reflection_final_step, test_plannersvc_plan_seed_9999_many_steps

5.3.2 FR-014 Topological plan execution

Requirement ID	FR-014
Name	Topological plan execution
Description	The system shall execute plan steps in dependency order (Kahn's algorithm, index tie-break), running the agent assigned to each step with a per-step capability token.
Actor	API client, Conversation flow
Preconditions	Plan is valid (contiguous indices, ≥ 1 step).
Inputs	JSON {plan: ExecutionPlan, conversation_id?, user_id?}.
Processing	1. Validate plan (ExecutionPlan.model_validate). 2. Topologically order steps; detect cycles. 3. For each step: instantiate agent, compute capabilities, mint token (TTL 900 s), shape state, call run_with_llm_or_fallback (FR-040). 4. Record StepRunResult (status, duration, preview, tokens, cost, report).
Outputs	200 envelope {run_id, steps[{step_id, agent_type, status, status_code, duration_ms, tokens_in, tokens_out, cost_usd, result_preview, report_type, report, error}]}
Postconditions	TaskExecution rows written (FR-015).
Alternate flow	Unregistered agent type $\rightarrow$ step AWAITING_INPUT.

Exception flow	Circular dependencies → ValueError (500); invalid plan payload → 500 'Internal data validation error' (measured; 422 would be expected).
Dependencies	FR-013, FR-021, FR-040
Priority	High
Acceptance criteria	1. Linear and diamond dependency graphs execute in order; cycles raise. 2. 5-step plan runs in 1.96 s deterministic / 60.4 s with qwen2.5-coder 7B (measured).
Implementation status	Implemented
Realised in	services/kernel_services.py: AgentSvc.run_plan(), _topological_order()
Verified by	test_m5::test_agentsvc_topological_linear_chain, test_agentsvc_topological_diamond, test_agentsvc_topological_circular_dep_raises_valueerror, test_runs_plan_and_execute

5.3.3 FR-015 Execution audit trail

Requirement ID	FR-015
Name	Execution audit trail
Description	The system shall persist one TaskExecution row per executed step with agent type, description, status, tokens, cost, duration, error, preview and metadata (run id, step index, dependencies).
Actor	System
Preconditions	FR-014 running.
Inputs	StepRunResult.
Processing	Create TaskExecution DTO; repository.create in a private event loop; failures are swallowed.
Outputs	task_executions rows.
Postconditions	Audit rows available to FR-016.
Alternate flow	—
Exception flow	Persistence errors are ignored (run continues).
Dependencies	FR-014
Priority	Medium
Acceptance criteria	Rows exist for every step of a run (verified via conversation id).
Implementation status	Implemented
Realised in	services/kernel_services.py: AgentSvc.run_plan()
Verified by	test_m5::test_agentsvc_run_plan_persists_task_execution_rows_and_returns_ordered

5.3.4 FR-016 List executions of a run

Requirement ID	FR-016
Name	List executions of a run
Description	The system shall list the TaskExecution rows that belong to a run identifier.
Actor	API client
Preconditions	—
Inputs	Path run_id (UUID); cursor; limit.
Processing	Repository list_for_run filters on conversation_id = run_id because the table has no run_id column.
Outputs	200 envelope list.
Postconditions	—
Alternate flow	—
Exception flow	On repository error an empty page is returned.
Dependencies	FR-015
Priority	Medium
Acceptance criteria	Querying with a real run_id returns its rows (NOT met: returns [] — measured); querying with the conversation id returns the rows.

Implementation status	Partially implemented
Realised in	database/sql_repos.py: SqlTaskExecutionRepository.list_for_run()
Verified by	test_repositories::test_task_execution_run_filter

5.3.5 FR-017 Background run completion messages

Requirement ID	FR-017
Name	Background run completion messages
Description	After a background run the system shall append one assistant message per reported step (prefixed [AGENT:ReportType]) and a run summary with token totals, cost and success/failure counts, then mark the conversation completed or failed.
Actor	System
Preconditions	FR-006 started a run.
Inputs	StepRunResults.
Processing	Compose messages; persist; update conversation metadata (status, token_count, cost_usd, message_count, preview).
Outputs	Assistant messages; updated conversation.
Postconditions	Status completed if no step failed, else failed.
Alternate flow	—
Exception flow	Any exception → 'Run failed — <type>: <msg>' message and status failed.
Dependencies	FR-006, FR-014
Priority	High
Acceptance criteria	LLM-mode run recorded 2 594 tokens in the summary (measured).
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: _bg_run_and_messages()
Verified by	test_m5::test_agentsvc_run_plan_writes_assistant_messages_for_report_types

5.4 Agent roster and agent behaviour

5.4.1 FR-018 Agent catalogue

Requirement ID	FR-018
Name	Agent catalogue
Description	The system shall list the agents exposed for direct invocation with description, required capabilities and report type.
Actor	Dashboard user, API client
Preconditions	—
Inputs	—
Processing	Return the static <code>_BUILTIN_AGENTS</code> list (7 manifests: planner, research, coding, reflection, memory, rag, tool).
Outputs	200 envelope list of AgentManifest.
Postconditions	—
Alternate flow	—
Exception flow	—
Dependencies	—
Priority	Medium
Acceptance criteria	Response contains manifest fields (name, agent_type, description, required_capabilities, report_type).
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: list_agents()

Verified by	test_m5::test_v1_agents_list_contains_manifest_fields
-------------	---

5.4.2 FR-019 Direct single-agent invocation

Requirement ID	FR-019
Name	Direct single-agent invocation
Description	The system shall run one of planner, research, coding or reflection on a caller-supplied state dictionary and return the resulting state.
Actor	API client
Preconditions	—
Inputs	JSON {agent, state{}}.
Processing	Instantiate agent; mint token with its required capabilities (TTL 600 s); build AgentRunContext with the default tool registry; run in a worker thread.
Outputs	200 envelope with output state (status, status_history, report, report_type).
Postconditions	—
Alternate flow	—
Exception flow	400 'Unknown agent' for other names (e.g. judge — measured).
Dependencies	FR-021
Priority	Medium
Acceptance criteria	Reflection on a self-contradicting answer reports ≥ 1 mistake (measured 2, confidence 0.62).
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: invoke_agent()
Verified by	test_m5::test_agents_list_and_invoke

5.4.3 FR-020 Spawn-time capability gating of agents

Requirement ID	FR-020
Name	Spawn-time capability gating of agents
Description	The roster shall refuse to spawn an agent whose context lacks any capability listed in the agent's <code>required_capabilities</code> .
Actor	Kernel / runtime
Preconditions	—
Inputs	AgentType and AgentRunContext.capabilities.
Processing	For each required (op, target) check any(c.allows(op, target)); collect missing pairs.
Outputs	Agent instance, or PermissionDenied(op=spawn_agent, target=<type>, details listing missing caps).
Postconditions	—
Alternate flow	—
Exception flow	PermissionDenied.
Dependencies	FR-033
Priority	High
Acceptance criteria	All 12 agent types denied with empty capabilities and allowed with their required set (measured 12/12).
Implementation status	Implemented
Realised in	agents/core.py: AgentRoster.spawn()
Verified by	test_m3::test_agent_roster_spawn_gate_rejects_insufficient_token, test_agent_roster_spawn_accepts_allow_all_for_every_agent

5.4.4 FR-021 Twelve typed agents

Requirement ID	FR-021
Name	Twelve typed agents
Description	The system shall provide twelve agent implementations, one per AgentType, each returning a typed Pydantic report: Planner, Research, Coding, Memory, RAG, Tool, Reflection, Judge, Critic, Executor, Supervisor, Orchestrator.
Actor	Runtime
Preconditions	—
Inputs	state dict + AgentRunContext.
Processing	See §AI/Algorithm specification for the rule set of each agent.
Outputs	Updated state with status and report.
Postconditions	—
Alternate flow	—
Exception flow	Agent-specific FAILED / AWAITING_INPUT results.
Dependencies	—
Priority	High
Acceptance criteria	AgentRoster.registry() has exactly 12 entries matching the AgentType enum.
Implementation status	Implemented
Realised in	agents/core.py
Verified by	test_m3::test_agent_roster_exactly_12_matches_enum

5.4.5 FR-022 Research agent

Requirement ID	FR-022
Name	Research agent
Description	Given a query and optional seed URLs, the research agent shall fetch up to 6 sources through the web_fetch tool, extract query-relevant sentences, list crude conflicts and score confidence.
Actor	Runtime
Preconditions	TOOL_INVOKE:web_fetch capability.
Inputs	state {query, seed_urls[]}
Processing	1. Plan 5 research steps. 2. Fetch each URL ($\leq 12\,000$ chars). 3. Rank sentences by query-term hits; build citations. 4. Detect opposing keyword pairs across sources; confidence = $\min(0.9, 0.35 + 0.15 \cdot n - 0.2 \cdot \text{conflicts})$.
Outputs	ResearchReport.
Postconditions	—
Alternate flow	No seed URLs → AWAITING_INPUT with confidence 0.2 (the default path in plan execution).
Exception flow	Missing query → FAILED.
Dependencies	FR-043
Priority	Medium
Acceptance criteria	With no seed URLs the report asks for seed_urls (measured).
Implementation status	Implemented
Realised in	agents/core.py: ResearchAgent
Verified by	test_m3::test_research_agent_synthesises_multiple_sources, test_research_agent_plan_only_when_no_seed_urls

5.4.6 FR-023 Coding agent

Requirement ID	FR-023
Name	Coding agent

Description	The coding agent shall read a workspace file, parse 'replace X with Y' instructions into patches, apply them in dry-run, compute a unified diff and SHA-256 of the result, and write new files when the target does not exist.
Actor	Runtime
Preconditions	TOOL_INVOKE:files.
Inputs	state {target_file, instructions}.
Processing	1. Read via FilesTool (create from extracted code if missing). 2. Parse replace instructions (double/single/unquoted). 3. apply_patch requires exactly one match per hunk. 4. Diff + sha256; confidence 0.85 with patches, 0.35 without.
Outputs	CodingPatchReport.
Postconditions	Files under var/workspace may be created or rewritten.
Alternate flow	No parsable instruction → zero patches, low confidence.
Exception flow	Ambiguous match → 'dry-run failed' (confidence 0.15).
Dependencies	FR-044
Priority	Medium
Acceptance criteria	Ambiguous original snippet raises ValueError in apply_patch.
Implementation status	Implemented
Realised in	agents/core.py: CodingAgent
Verified by	test_m3::test_coding_agent_returns_plan_and_patches, test_coding_agent_patch_ambiguous_match_raises

5.4.7 FR-024 Reflection, judge and critic agents

Requirement ID	FR-024
Name	Reflection, judge and critic agents
Description	The system shall audit candidate answers: Reflection detects missing citations, tool failures and contradictions; Judge ranks candidates by lexical density and citation markers; Critic scores five quality axes.
Actor	Runtime
Preconditions	—
Inputs	state {answer candidates draft}.
Processing	Rule-based heuristics described in §Algorithms (A5).
Outputs	ReflectionReport / JudgeReport / CriticReport.
Postconditions	—
Alternate flow	—
Exception flow	Missing inputs → FAILED.
Dependencies	—
Priority	Medium
Acceptance criteria	Contradiction 'is safe ... is not safe' is flagged (measured).
Implementation status	Implemented
Realised in	agents/core.py
Verified by	test_m3::test_reflection_reports_contradiction, test_judge_ranks_candidates_and_picks_winner, test_critic_scores_quality_and_produces_rubric

5.4.8 FR-025 Executor tri-state decision

Requirement ID	FR-025
Name	Executor tri-state decision
Description	The executor agent shall return SIGNOFF (confidence ≥ 0.90), REPLAN (0.50–0.90) or REJECT (< 0.50) from severity-weighted acceptance criteria minus plan-health penalties.
Actor	Runtime

Preconditions	—
Inputs	state {plan?, accept_criteria?, final_answer?, report?}.
Processing	Algorithm A4.
Outputs	ExecutorReport with ExecutorTriStateDecision.
Postconditions	—
Alternate flow	Criteria synthesised from answer length / critic score when none supplied.
Exception flow	—
Dependencies	—
Priority	High
Acceptance criteria	Decision grid reproduces thresholds exactly (computed 16-cell grid).
Implementation status	Implemented
Realised in	agents/core.py: ExecutorAgent.evaluate_decision()
Verified by	test_executor_tristate (4 tests), test_m3::test_executor_signs_off_good_answer

5.4.9 FR-026 Memory, RAG, tool, supervisor and orchestrator agents

Requirement ID	FR-026
Name	Memory, RAG, tool, supervisor and orchestrator agents
Description	The remaining agents shall provide typed reports for memory operations, retrieval, batched tool calls, task-tree planning and fan-out/join policy selection.
Actor	Runtime
Preconditions	Capabilities per agent.
Inputs	state.
Processing	1. MemoryAgent recall/store/compress return summaries (no repository I/O). 2. RAGAgent returns k deterministic synthetic chunks (no index lookup). 3. ToolAgent invokes registry tools. 4. SupervisorAgent computes a hypothetical task tree from an MD5 of the goal. 5. OrchestratorAgent returns a worker list and join policy (all/any/majority) without dispatching.
Outputs	MemoryReport / RAGReport / ToolReport / SupervisorReport / OrchestratorReport.
Postconditions	—
Alternate flow	—
Exception flow	Missing inputs → FAILED/AWAITING_INPUT.
Dependencies	FR-043
Priority	Low
Acceptance criteria	Reports validate against their Pydantic models.
Implementation status	Stub / simulated
Realised in	agents/core.py
Verified by	test_m3::test_memory_agent_*, test_rag_agent_*, test_tool_agent_*, test_supervisor_*, test_orchestrator_*

5.5 Documents

5.5.1 FR-027 Register document metadata

Requirement ID	FR-027
Name	Register document metadata
Description	The system shall store document metadata (title, source URI, MIME type, user, metadata) with ingestion_status 'pending'.
Actor	Dashboard user, API client
Preconditions	—

Inputs	JSON {title, source_uri, mime_type?, user_id?, metadata?}.
Processing	Create Document row.
Outputs	200 envelope Document.
Postconditions	—
Alternate flow	—
Exception flow	422.
Dependencies	—
Priority	Medium
Acceptance criteria	Created document is retrievable by id (measured).
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: create_document()
Verified by	test_m5::test_documents_lifecycle_create_list_get_upload_delete

5.5.2 FR-028 Upload document file

Requirement ID	FR-028
Name	Upload document file
Description	The system shall accept a multipart file upload and record byte size, MIME type and a chunk count.
Actor	Dashboard user, API client
Preconditions	—
Inputs	multipart file; form fields user_id?, title?, source_uri?.
Processing	1. Read the whole file into memory; create Document(status processing). 2. Simulate ingestion: set status 'ready' and chunk_count = max(1, bytes // 2048). The bytes are not stored, parsed, chunked or indexed.
Outputs	200 envelope Document (status ready).
Postconditions	Only metadata persists.
Alternate flow	—
Exception flow	422 when file missing.
Dependencies	—
Priority	Medium
Acceptance criteria	6 300-byte upload reports chunk_count 3 (measured).
Implementation status	Stub / simulated
Realised in	api/routes/v1_m5.py: upload_document()
Verified by	test_m5::test_documents_lifecycle_create_list_get_upload_delete

5.5.3 FR-029 List, get and delete documents

Requirement ID	FR-029
Name	List, get and delete documents
Description	The system shall list (with status and search filters), fetch and delete document records.
Actor	Dashboard user, API client
Preconditions	—
Inputs	Query/path parameters.
Processing	Keyset page; filter by ingestion_status and title/URI substring.
Outputs	200 envelopes.
Postconditions	—
Alternate flow	—
Exception flow	404 for unknown id (measured on second delete).

Dependencies	FR-027
Priority	Medium
Acceptance criteria	Delete then get returns 404.
Implementation status	Implemented
Realised in	api/routes/v1_m5.py
Verified by	test_m5::test_documents_lifecycle_create_list_get_upload_delete

5.6 Memory records and tiered memory

5.6.1 FR-030 List memory records

Requirement ID	FR-030
Name	List memory records
Description	The system shall list stored long-term memory rows filtered by memory_type and minimum importance.
Actor	Dashboard user, API client
Preconditions	—
Inputs	Query cursor, limit, memory_type, min_importance (0–1).
Processing	Keyset page; in-memory filters.
Outputs	200 envelope list of Memory.
Postconditions	—
Alternate flow	—
Exception flow	422.
Dependencies	—
Priority	Medium
Acceptance criteria	Returns [] on a fresh database (no API writes to memories exist).
Implementation status	Partially implemented
Realised in	api/routes/v1_m5.py: list_memory()
Verified by	test_m5::test_memory_list_query_compress

5.6.2 FR-031 Query memory records

Requirement ID	FR-031
Name	Query memory records
Description	The system shall rank memory rows against a text query using token-set Jaccard overlap plus $0.1 \times \text{importance}$.
Actor	API client
Preconditions	—
Inputs	JSON {content, tiers[], top_k (1–200), min_importance}.
Processing	Load up to $\max(100, 5 \cdot \text{top_k})$ rows; tokens ≥ 3 chars; score; sort; truncate.
Outputs	200 envelope list with score.
Postconditions	—
Alternate flow	—
Exception flow	422.
Dependencies	FR-030
Priority	Low
Acceptance criteria	Results are ordered by descending score.
Implementation status	Partially implemented
Realised in	api/routes/v1_m5.py: query_memory()

Verified by	test_m5::test_memory_list_query_compress
-------------	--

5.6.3 FR-032 Compress stale memory

Requirement ID	FR-032
Name	Compress stale memory
Description	The system shall mark memory rows older than N days as compressed and set a 120-character summary.
Actor	API client
Preconditions	—
Inputs	JSON {older_than_days (1–3650), tiers[]}
Processing	Scan ≤ 200 rows; update eligible ones.
Outputs	200 {compressed_count, ids[]}
Postconditions	Rows flagged is_compressed.
Alternate flow	—
Exception flow	Per-row update errors ignored.
Dependencies	FR-030
Priority	Low
Acceptance criteria	Fresh database returns compressed_count 0 (measured).
Implementation status	Implemented
Realised in	api/routes/v1_m5.py: compress_memory()
Verified by	test_m5::test_memory_list_query_compress

5.6.4 FR-033 Six-tier memory promotion with provenance

Requirement ID	FR-033
Name	Six-tier memory promotion with provenance
Description	The memory subsystem shall promote entries T1 Indriya $\rightarrow$ T2 Smriti $\rightarrow$ T3 Gyān $\rightarrow$ T4 Yojanā $\rightarrow$ T5 Sāadhanā $\rightarrow$ T6 Paalak on explicit triggers and record a SHA-256 hash chain of every migration.
Actor	MemoryAgent (via PlannerAgent)
Preconditions	Entries inserted into T1.
Inputs	MemoryEntry objects, plan references, Gyān corpus fingerprints, clock, human-review marks.
Processing	Algorithm A3: bottom-up edge evaluation per pass; T1 LRU cap 1000; T6 refuses overwrite.
Outputs	PromotionReport {tier_counts, provenance_tail_sha, total_events}.
Postconditions	Entries reside in their new tier; chain tail updated.
Alternate flow	Thresholds configurable via PromotionThresholds.
Exception flow	T6 overwrite $\rightarrow$ ValueError.
Dependencies	—
Priority	High
Acceptance criteria	1. Identical inputs give an identical provenance tail (measured 5/5 repeats). 2. In live planning only T1 is populated (measured tier_counts T1=5, others 0).
Implementation status	Partially implemented
Realised in	memory/promotion.py: PromotionController
Verified by	test_memory_promotion (4 tests), test_memory_promotion_wiring (2 tests)

5.6.5 FR-034 Tier persistence across restarts

Requirement ID	FR-034
Name	Tier persistence across restarts

Description	Tier contents should be saved to <code>memory_tiers_promotions</code> on shutdown and reloaded on start-up.
Actor	System
Preconditions	—
Inputs	—
Processing	Hooks exist in <code>api/main.py</code> lifespan, but no <code>PromotionController</code> is registered on the kernel (<code>_memory_promotion_ctrl1</code> / service 'memory_promotion' are never set), so both hooks are no-ops.
Outputs	—
Postconditions	—
Alternate flow	—
Exception flow	—
Dependencies	FR-033
Priority	Medium
Acceptance criteria	Tier entries survive a restart (NOT met).
Implementation status	Planned (not implemented)
Realised in	<code>api/main.py</code> lifespan; <code>memory/promotion.py</code> <code>save_to_sql/load_from_sql</code>
Verified by	—

5.6.6 FR-035 Multi-tier recall bus

Requirement ID	FR-035
Name	Multi-tier recall bus
Description	The memory bus shall write entries to scope-derived tiers (working, conversation, episodic, user, project, semantic) and recall across tiers with Reciprocal Rank Fusion ($k = 60$).
Actor	Library caller
Preconditions	—
Inputs	<code>MemoryEntry</code> ; query; scope.
Processing	remember → <code>SCOPE_TO_TIERS</code> ; recall → per-tier search → RRF → normalised scores; retain → prune per tier.
Outputs	<code>RecallHit</code> list.
Postconditions	—
Alternate flow	—
Exception flow	<code>KeyError/ValueError</code> for unknown tiers or scopes without mapping.
Dependencies	—
Priority	Low
Acceptance criteria	Recall returns tier identity and merged score.
Implementation status	Implemented as library (not wired to API)
Realised in	<code>memory/bus.py</code> : <code>MemoryBus</code>
Verified by	<code>test_memory_rag::test_bus_*</code> (7 tests)

5.7 Kernel, capabilities and scheduling

5.7.1 FR-036 Kernel lifecycle and state

Requirement ID	FR-036
Name	Kernel lifecycle and state
Description	The kernel shall boot during application start-up (scheduler loop, allocator GC, <code>KERNEL_BOOTED</code> interrupt) and expose its state, uptime, registered services and handler set.
Actor	Operator

Preconditions	Application lifespan.
Inputs	GET /v1/kernel/state.
Processing	Return state, uptime_s, started_at, agents_registered, services_registered, handlers_registered, spans_captured, capacity 10 000.
Outputs	200 envelope.
Postconditions	—
Alternate flow	—
Exception flow	RuntimeError when the lifespan has not run.
Dependencies	—
Priority	Medium
Acceptance criteria	State 'running' with 8 repository services and 6 handlers (measured).
Implementation status	Implemented
Realised in	kernel/kernel.py; api/routes/kernel.py
Verified by	test_kernel_api::test_kernel_state_shows_uptime_after_start, integration::test_lifespan_boots_kernel_state_running

5.7.2 FR-037 Capability-checked system calls

Requirement ID	FR-037
Name	Capability-checked system calls
Description	Every kernel syscall shall be checked against the caller's capability table before dispatch, denied calls shall not execute, and every call shall produce a trace span.
Actor	Agents / plugins
Preconditions	Agent spawned via Kernel.spawn_agent.
Inputs	AgentHandle, SysCall (27 values), payload.
Processing	1. Reject when kernel not RUNNING/PAUSED. 2. Map syscall → (CapabilityOp, target). 3. Deny unless any capability allows (glob, case-insensitive). 4. Dispatch to handler (6 default: MEMORY_READ, MEMORY_WRITE, MODEL_CHAT, MODEL_ROUTE, TOOL_INVOKE, GET_CLOCK). 5. Record span ($\leq 10\,000$ kept).
Outputs	SysCallResult {success, payload, error, latency_ms, trace_id, span_id}.
Postconditions	Span ring buffer updated.
Alternate flow	Unregistered syscall → 'No handler for syscall ...'.
Exception flow	PermissionDenied text in error; handler exceptions captured.
Dependencies	—
Priority	High
Acceptance criteria	0-capability TOOL_INVOKE is denied; matching capability succeeds (measured 9 kernel cases).
Implementation status	Implemented
Realised in	kernel/kernel.py: Kernel.syscall()
Verified by	test_kernel_api (10 tests); scripts/audit_mac_spawn.py

5.7.3 FR-038 HMAC capability gate on HTTP routes

Requirement ID	FR-038
Name	HMAC capability gate on HTTP routes
Description	Gated HTTP routes shall require X-Noesis-Capability-Token = b64url(JSON payload).b64url(HMAC-SHA256), verify signature in constant time, schema, expiry, required capability and deny masks (deny wins).
Actor	Evaluator, dashboard bench pages
Preconditions	CLAIM_SUITES_HMAC_SECRET configured (dev default otherwise).

Inputs	Header token.
Processing	Algorithm A6 (five checks).
Outputs	Request proceeds or 403 with C1_DENY_MISSING_CAP / C1_DENY_MAC / C1_DENY_EXPIRED / C1_DENY_MASK.
Postconditions	—
Alternate flow	Empty <code>requires_capabilities()</code> = authentication only.
Exception flow	403 with reason code.
Dependencies	—
Priority	High
Acceptance criteria	Six deny scenarios and one allow scenario behave as specified (measured).
Implementation status	Implemented
Realised in	<code>api/middleware/capability_gate.py</code>
Verified by	<code>claim_suites</code> (8 tests)

5.7.4 FR-039 Kernel statistics and traces

Requirement ID	FR-039
Name	Kernel statistics and traces
Description	The system shall expose scheduler, allocator and model-router aggregates and the most recent N syscall spans (1–5000).
Actor	Operator, dashboard
Preconditions	—
Inputs	GET /v1/kernel/stats; GET /v1/kernel/traces?limit.
Processing	Aggregate counters; slice span buffer.
Outputs	200 envelopes.
Postconditions	—
Alternate flow	—
Exception flow	422 for limit outside 1–5000.
Dependencies	FR-037
Priority	Low
Acceptance criteria	Traces are empty unless syscalls were issued (HTTP requests do not create spans — measured returned 0).
Implementation status	Implemented
Realised in	<code>api/routes/kernel.py</code>
Verified by	<code>test_kernel_api::test_kernel_traces_*</code>

5.7.5 FR-040 LLM enrichment with deterministic fallback

Requirement ID	FR-040
Name	LLM enrichment with deterministic fallback
Description	When a local Ollama model is reachable, each step shall send the agent's system prompt and serialised state to the model and attach the reply; otherwise the deterministic agent output is used unchanged.
Actor	AgentSvc
Preconditions	AGENTS_OLLAMA_MODEL pulled in Ollama (optional).
Inputs	Agent type, state.
Processing	Algorithm A2.
Outputs	Report (+ <code>llm_rationale</code>), token usage, mode.
Postconditions	—
Alternate flow	Provider absent → deterministic.

Exception flow	Any provider exception → deterministic (warning once per agent type).
Dependencies	FR-048
Priority	High
Acceptance criteria	With Ollama: 3/5 steps recorded tokens (2 594 total); without: 0 tokens and identical reports (measured).
Implementation status	Implemented
Realised in	services/kernel_services.py: run_with_llm_or_fallback()
Verified by	test_m5::test_agentsvc_run_step_research_agent_runs_and_returns_success

5.7.6 FR-041 Scheduler, allocator, router and interrupts

Requirement ID	FR-041
Name	Scheduler, allocator, router and interrupts
Description	The kernel shall provide a priority/deadline scheduler, per-agent token quotas with OOM interrupts, a weighted model router and an interrupt vocabulary with restart back-off policies.
Actor	Kernel API callers
Preconditions	—
Inputs	Programmatic calls.
Processing	See §Algorithms A7–A8.
Outputs	TaskHandle, MemoryAllocation, ModelSchedulingDecision, Interrupt.
Postconditions	—
Alternate flow	—
Exception flow	OOMInterrupt on hard-quota breach.
Dependencies	—
Priority	Medium
Acceptance criteria	Router returns gpt-4o as primary for the default constraint and falls back to gpt-4o-mini when all candidates are filtered (computed).
Implementation status	Implemented as library (not wired to API)
Realised in	kernel/scheduler.py, allocators.py, model_router.py, interrupts.py
Verified by	test_kernel_api::test_kernel_allocator_zones_populated_after_malloc, test_kernel_model_profiles_include_gpu_flag

5.8 Tool registry and sandboxed tools

5.8.1 FR-042 Capability-gated tool registry

Requirement ID	FR-042
Name	Capability-gated tool registry
Description	The tool registry shall refuse an invocation unless the caller presents TOOL_INVOKE for the tool's name, and shall normalise every outcome to ToolResult {success, exit_code, stdout, stderr, duration_ms, artifacts, structured}.
Actor	Agents
Preconditions	Tool registered.
Inputs	name, args, token, capabilities.
Processing	Unknown tool → failure result; check capability; invoke adapter; convert exceptions to exit_code -2.
Outputs	ToolResult.
Postconditions	—
Alternate flow	—
Exception flow	PermissionDenied when capability missing.

Dependencies	—
Priority	High
Acceptance criteria	files tool with only web_fetch capability is denied (measured).
Implementation status	Implemented
Realised in	tools/_init_.py: ToolRegistry
Verified by	test_m3::test_registry_capability_gate_blocks_without_permission

5.8.2 FR-043 Default sandboxed tools

Requirement ID	FR-043
Name	Default sandboxed tools
Description	The default registry shall provide shell (dry-run unless allow-listed), python_sandbox (AST allow-list, restricted builtins, 5 s timeout), files (rooted at var/workspace, traversal-safe) and web_fetch (http/https only).
Actor	Agents
Preconditions	—
Inputs	Tool arguments.
Processing	See §Security for the containment rules.
Outputs	ToolResult.
Postconditions	Workspace files may change (files write/delete).
Alternate flow	—
Exception flow	Exit codes 13 (path escape), 65/66 (syntax/AST), 70 (runtime), 124 (timeout).
Dependencies	FR-042
Priority	High
Acceptance criteria	rm -rf / is only described (dry-run); import os and open() are blocked; ../../etc/passwd is refused (measured).
Implementation status	Implemented
Realised in	tools/_init_.py; services/kernel_services.py: _default_tool_registry()
Verified by	test_m3::test_shell_tool_*, test_python_sandbox_*, test_files_tool_*, test_web_fetch_tool_*

5.8.3 FR-044 Additional tool adapters

Requirement ID	FR-044
Name	Additional tool adapters
Description	The code base shall provide GitHub, weather, calendar (.ics), e-mail (SMTP/IMAP) and read-only database tools for registration by integrators.
Actor	Integrator
Preconditions	Credentials where needed.
Inputs	Tool arguments.
Processing	GitHub/weather/e-mail run in dry-run without credentials; DBTool accepts only SELECT/WITH/EXPLAIN/PRAGMA/SHOW/VALUES and always rolls back.
Outputs	ToolResult.
Postconditions	—
Alternate flow	—
Exception flow	Non-read SQL refused.
Dependencies	FR-042
Priority	Low
Acceptance criteria	INSERT is refused by DBTool (unit test).
Implementation status	Implemented as library (not wired to API)
Realised in	tools/_init_.py

Verified by	test_m3::test_github_tool_dry_run_repo, test_db_tool_blocks_insert, test_calendar_tool_add_and_list, test_email_tool_send_dry_run
-------------	---

5.9 LLM provider integration

5.9.1 FR-045 Pluggable LLM providers

Requirement ID	FR-045
Name	Pluggable LLM providers
Description	The system shall provide chat adapters for OpenAI (and OpenAI-compatible llama.cpp), Anthropic, Google Gemini, Ollama and OpenRouter that return a unified ProviderResponse with token usage.
Actor	Runtime, integrators
Preconditions	Credentials in environment (except Ollama/llama.cpp).
Inputs	ChatMessage list, tools, tool_choice, system prompt.
Processing	Factory_build_raw_provider checks credentials; provider-specific message/tool translation; httpx call; tenacity retry on 408/425/429/5xx (4 attempts).
Outputs	ProviderResponse.
Postconditions	Usage accumulated per provider instance.
Alternate flow	—
Exception flow	RuntimeError when a required key is missing.
Dependencies	—
Priority	High
Acceptance criteria	Ollama path verified live (measured); hosted providers verified only with unit-level mocks.
Implementation status	Implemented
Realised in	llm/*.py
Verified by	test_llm_provider (7), test_llm_factory_audit (2)

5.9.2 FR-046 Provider failover

Requirement ID	FR-046
Name	Provider failover
Description	get_provider shall wrap primary and fallback providers and switch to the fallback on connection errors or HTTP 5xx.
Actor	Runtime
Preconditions	Fallback differs from primary and is buildable.
Inputs	—
Processing	FailoverProvider catches ConnectionError, HTTPRetryableError, HTTP 5xx; records fallback_triggered and reason.
Outputs	ProviderResponse from fallback.
Postconditions	—
Alternate flow	—
Exception flow	Non-failover errors propagate.
Dependencies	FR-045
Priority	Medium
Acceptance criteria	Fallback is used when the primary raises (unit test).
Implementation status	Implemented
Realised in	llm/factory.py: FailoverProvider
Verified by	test_llm_factory_audit::test_factory_fallback_kicks_in_when_primary_raises

5.9.3 FR-047 Embedding providers

Requirement ID	FR-047
Name	Embedding providers
Description	The system shall provide OpenAI and Ollama embedding adapters.
Actor	Integrators
Preconditions	—
Inputs	texts[]
Processing	HTTP embedding calls.
Outputs	Embedding list.
Postconditions	—
Alternate flow	—
Exception flow	ValueError for other providers.
Dependencies	FR-045
Priority	Low
Acceptance criteria	—
Implementation status	Implemented as library (not wired to API)
Realised in	llm/openai_provider.py, ollama_provider.py
Verified by	—

5.10 Health, observability and metrics

5.10.1 FR-048 Liveness and readiness probes

Requirement ID	FR-048
Name	Liveness and readiness probes
Description	The system shall expose liveness (always pass) and readiness that probes SQL (hard), Redis and Qdrant (soft, 'warn').
Actor	Operator, container orchestrator
Preconditions	—
Inputs	GET /health/livez, /health/readyz, /health (and /v1/health/*).
Processing	Run three probes concurrently; overall fail only if SQL fails.
Outputs	200 envelope {status, checks{sql, redis, qdrant}} or error envelope when SQL fails.
Postconditions	—
Alternate flow	—
Exception flow	—
Dependencies	—
Priority	High
Acceptance criteria	Without Redis/Qdrant the status is 'warn' with ok=true (measured).
Implementation status	Implemented
Realised in	api/routes/health.py
Verified by	test_api_smoke::test_livez_always_pass, test_m5::test_v1_root_envelope_ok_true_for_health

5.10.2 FR-049 Prometheus and JSON metrics

Requirement ID	FR-049
Name	Prometheus and JSON metrics
Description	The system shall expose HTTP request counters and duration summaries in Prometheus text format at /metrics and /v1/metrics and a JSON snapshot at /metrics_json and /v1/metrics_json.

Actor	Monitoring
Preconditions	prometheus_client installed (no-op shim otherwise).
Inputs	—
Processing	Request middleware calls <code>observe_http_request(method, path, status, seconds)</code> .
Outputs	text/plain; version=0.0.4 or JSON envelope.
Postconditions	—
Alternate flow	—
Exception flow	—
Dependencies	—
Priority	Medium
Acceptance criteria	1. Counters <code>backend_http_requests_total</code> increase per request. 2. Step counters exist but <code>observe_step_run</code> is never called from the runtime.
Implementation status	Partially implemented
Realised in	observability/metrics.py; api/routes/v1_metrics.py
Verified by	test_m5::test_metrics_prometheus_scrape_returns_text, test_metrics_json_snapshot_envelope_ok_and_contains_expected_metric_names

5.10.3 FR-050 Observability summary

Requirement ID	FR-050
Name	Observability summary
Description	The system shall summarise request latency percentiles, token cost and per-tool latency for a time window.
Actor	Dashboard
Preconditions	—
Inputs	GET /v1/observability/summary?window_s.
Processing	Filter in-memory rings by monotonic time; compute p50/p95.
Outputs	ObservabilitySummary.
Postconditions	—
Alternate flow	—
Exception flow	—
Dependencies	—
Priority	Low
Acceptance criteria	Counters stay zero because <code>record_request/record_tool_trace</code> are never called (measured all zeros).
Implementation status	Stub / simulated
Realised in	api/routes/v1_m3.py
Verified by	test_m3::test_observability_summary_admin

5.10.4 FR-051 Request correlation and structured logs

Requirement ID	FR-051
Name	Request correlation and structured logs
Description	Every response shall carry X-Request-ID (generated when absent) and produce one structured access-log line; secrets in log events shall be redacted.
Actor	Operator
Preconditions	—
Inputs	Any request.
Processing	Middleware assigns request id; logs method, path, status, latency; structlog processor redacts keys containing <code>key/token/secret/password/jwt/authorization/auth/credential</code> and bearer values.

Outputs	Header + log line.
Postconditions	—
Alternate flow	—
Exception flow	—
Dependencies	—
Priority	Medium
Acceptance criteria	Every captured response envelope contains request_id (measured).
Implementation status	Implemented
Realised in	api/main.py; logging.py
Verified by	test_api_smoke

5.11 Benchmarking and evaluation

5.11.1 FR-052 LLM throughput benchmark endpoint

Requirement ID	FR-052
Name	LLM throughput benchmark endpoint
Description	GET /llm/benchmark shall send one prompt of approximately N tokens to Ollama (or a mock when unreachable) and report tokens per second and latency.
Actor	Evaluator
Preconditions	Capability <code>llm.benchmark.read</code> .
Inputs	Query prompt_tokens (8–32768), model?.
Processing	Probe /api/tags; build deterministic prompt; call provider with timeout max(settings.timeout, 60 s); fall back to mock on error.
Outputs	BenchmarkResult {provider, model, reachable, used_mock, tokens, latency_ms, first_token_latency_ms (estimated), tokens/s, fallback_provider, error}.
Postconditions	—
Alternate flow	Mock values when Ollama unreachable.
Exception flow	403 from the gate.
Dependencies	FR-038
Priority	Medium
Acceptance criteria	Live run: 191→150 tokens in 6.4 s (≈23 completion tok/s); 1024-token prompts timed out at 120 s (measured).
Implementation status	Implemented
Realised in	api/routes/llm_benchmark.py
Verified by	test_routes_bench_results::test_llm_benchmark_endpoint_still_works, claim_suites

5.11.2 FR-053 Benchmark results API

Requirement ID	FR-053
Name	Benchmark results API
Description	GET /api/bench/results shall serve the newest tables_for_paper.json under EVAL_ROOT (optionally re-aggregating CSVs) in the dashboard schema, or a flagged demo payload when none exists.
Actor	Dashboard, evaluator
Preconditions	Capability <code>bench.results.read</code> .
Inputs	Query refresh (bool).
Processing	Find newest tables by mtime; validate into BenchmarkResults (extra=forbid); refresh runs stats.aggregate_explicit_csvs_and_write_json.
Outputs	200 envelope + meta {demo, source, generated_at}.

Postconditions	refresh=true may rewrite tables_for_paper.json.
Alternate flow	Demo payload (meta.demo=true) when no tables or parse error.
Exception flow	403 from the gate.
Dependencies	FR-038, FR-054
Priority	Medium
Acceptance criteria	Returned the Oct-06 tables with meta.demo=false (measured).
Implementation status	Implemented
Realised in	api/routes/bench_results.py
Verified by	test_routes_bench_results (5 tests)

5.11.3 FR-054 Offline evaluation harness

Requirement ID	FR-054
Name	Offline evaluation harness
Description	Scripts shall run SE-50 (pipeline sign-off), HumanEval and MBPP (syntax-compile of generated code) and aggregate pass rates with bootstrap CIs, McNemar and Wilcoxon helpers.
Actor	Evaluator
Preconditions	Ollama for adaptive mode; network for full corpora.
Inputs	CLI flags (corpus, seeds, runs, mode seeded adaptive, outdir).
Processing	See §AI/ML Specification.
Outputs	CSV + JSON summaries + tables_for_paper.json.
Postconditions	—
Alternate flow	Bundled stubs used when downloads fail (Oct-06 run).
Exception flow	SystemExit when adaptive model missing.
Dependencies	—
Priority	Medium
Acceptance criteria	Oct-06 run artefacts exist in docs/eval/w7_weekend_real_Oct06_real.
Implementation status	Partially implemented
Realised in	scripts/bench_*.py, scripts/run_w7_weekend.py, benchmarks/stats.py
Verified by	test_bench_stats (8), test_stats_fallbacks (3), test_determinism_manifest (1)

5.12 Universal Agent Protocol

5.12.1 FR-055 UAP message model and wire codecs

Requirement ID	FR-055
Name	UAP message model and wire codecs
Description	The system shall define typed UAP messages (22 intents, evidence, goals, artifacts, spans, envelopes, syscalls) with lossless deterministic JSON and tag-length-value binary encodings.
Actor	Integrators
Preconditions	—
Inputs	UAP model instances / bytes.
Processing	encode_json (sorted keys, UUID hex, RFC 3339 Z) / encode_binary (proto tag numbers).
Outputs	bytes / model.
Postconditions	—
Alternate flow	—
Exception flow	TypeError for unregistered models.
Dependencies	—
Priority	Low

Acceptance criteria	JSON → binary → JSON round-trips are equal (unit tests).
Implementation status	Implemented as library (not wired to API)
Realised in	uap/ __init__.py, uap/wire.py, schemas/uap/v1/uap.proto
Verified by	test_uap_idl (13 tests)

5.12.2 FR-056 UAP HTTP endpoints

Requirement ID	FR-056
Name	UAP HTTP endpoints
Description	The API shall list transports, issue an in-process binding token and accept an envelope.
Actor	API client
Preconditions	—
Inputs	JSON bodies.
Processing	Transports list is static; binding returns a random <code>uap.<hex>.<ts></code> string; envelope/send hashes repr(payload) and echoes routing — no message is delivered.
Outputs	200 envelopes.
Postconditions	—
Alternate flow	—
Exception flow	422.
Dependencies	FR-055
Priority	Low
Acceptance criteria	Envelope send returns <code>accepted=true</code> with <code>payload_sha256</code> (measured).
Implementation status	Stub / simulated
Realised in	api/routes/v1_m5.py
Verified by	test_m5::test_uap_transports_bind_and_envelope

5.13 Plugins and command-line interface

5.13.1 FR-057 Plugin discovery and lifecycle

Requirement ID	FR-057
Name	Plugin discovery and lifecycle
Description	The plugin manager shall discover manifests in <code>./plugins</code> and Python entry points, verify manifests (including SHA checks), load, boot, call hooks with timeouts and shut down plugins.
Actor	Integrator
Preconditions	—
Inputs	Plugin directories / entry points.
Processing	discover → load_from_manifest → boot_all; call_hook with per-hook timeout (30 s).
Outputs	Plugin registry.
Postconditions	—
Alternate flow	No <code>./plugins</code> directory → nothing loaded.
Exception flow	PluginLoadError.
Dependencies	—
Priority	Low
Acceptance criteria	<code>plugins verify</code> reports SHA mismatch (CLI tests).
Implementation status	Implemented
Realised in	plugins/*.py; api/main.py lifespan
Verified by	test_plugins_cli (13 tests)

5.13.2 FR-058 Command-line interface

Requirement ID	FR-058
Name	Command-line interface
Description	The <code>noesis</code> CLI shall provide <code>serve</code> , <code>doctor</code> , <code>shell</code> , <code>run</code> (deterministic or adaptive goal run), <code>db:upgrade/downgrade/makemigrations</code> and <code>plugins list/verify/load/unload</code> .
Actor	Operator, developer
Preconditions	Package installed (entry point <code>noesis.cli:main</code>).
Inputs	Command arguments.
Processing	Typical commands.
Outputs	Console output (table/json).
Postconditions	—
Alternate flow	—
Exception flow	Non-zero exit on failure.
Dependencies	—
Priority	Low
Acceptance criteria	<code>noesis --version</code> prints the version (CLI test).
Implementation status	Implemented
Realised in	<code>cli.py</code>
Verified by	<code>test_plugins_cli::test_noesis_version</code>

5.14 Dashboard (Noesis Studio)

5.14.1 FR-059 Dashboard shell and navigation

Requirement ID	FR-059
Name	Dashboard shell and navigation
Description	The dashboard shall provide a sidebar with ten destinations, a top bar with search field and theme toggle, and toast notifications.
Actor	Dashboard user
Preconditions	Frontend running.
Inputs	Navigation clicks.
Processing	Next.js App Router layout; React Query provider; sonner toaster.
Outputs	Rendered pages.
Postconditions	—
Alternate flow	—
Exception flow	HTTP error toasts for 401/403/429/5xx.
Dependencies	—
Priority	Medium
Acceptance criteria	13 routes compile and render (measured: all returned HTTP 200).
Implementation status	Implemented
Realised in	<code>frontend/app/layout.tsx</code> , <code>src/components/layout/*</code>
Verified by	<code>Sidebar.test.tsx</code> (3)

5.14.2 FR-060 Conversation workflow in the UI

Requirement ID	FR-060
Name	Conversation workflow in the UI
Description	Users shall create a run from a goal, see it in the list with status, open the thread and watch step messages appear.

Actor	Dashboard user
Preconditions	Backend reachable via /api/backend rewrite.
Inputs	Goal text, lead agent, tags.
Processing	POST /v1/conversations; navigate to /conversations/{id}; React Query refetch of GET /v1/conversations/{id}.
Outputs	Thread view with codename badges.
Postconditions	—
Alternate flow	Mock list shown when backend list is empty/unreachable.
Exception flow	Toast on failure.
Dependencies	FR-006
Priority	High
Acceptance criteria	Screenshot shows live thread with step messages (captured).
Implementation status	Implemented
Realised in	frontend/app/conversations/*
Verified by	—

5.14.3 FR-061 Monitoring pages (dashboard, agents, timeline, memory, documents, observability)

Requirement ID	FR-061
Name	Monitoring pages (dashboard, agents, timeline, memory, documents, observability)
Description	These pages should render live backend data.
Actor	Dashboard user
Preconditions	—
Inputs	—
Processing	Endpoint helpers expect bare arrays but the API returns envelopes, so listAgents/listDocuments/listMemory/listTraces return [] and the pages fall back to seeded mock data (buildMock*).
Outputs	Pages with placeholder data and a 'Backend unreachable — falling back to local mock data' banner on error.
Postconditions	—
Alternate flow	—
Exception flow	—
Dependencies	FR-018, FR-027, FR-030, FR-039
Priority	Medium
Acceptance criteria	Pages show backend records (NOT met for these six pages).
Implementation status	Stub / simulated
Realised in	frontend/src/lib/api/endpoints.ts; app/*/page.tsx
Verified by	mocks.test.ts, schemas.test.ts

5.14.4 FR-062 Benchmark hub and runner pages

Requirement ID	FR-062
Name	Benchmark hub and runner pages
Description	The Bench Hub shall show SE-50/HumanEval/MBPP/ablation results; the Bench Runner shall trigger GET /llm/benchmark; both label demo data explicitly.
Actor	Evaluator
Preconditions	Backend at http://localhost:8000.
Inputs	Tab selection; run button.

Processing	fetch with DEV_LOCAL_CAPABILITY_TOKEN (attached only for localhost bases); Zod validation; demo fallback.
Outputs	Charts, tables, KPI tiles; toasts.
Postconditions	—
Alternate flow	Demo/seeded data with a visible banner.
Exception flow	Toast with error.
Dependencies	FR-052, FR-053
Priority	Medium
Acceptance criteria	Demo banner visible when backend fetch fails (screenshot).
Implementation status	Partially implemented
Realised in	frontend/app/benchmarks/page.tsx, app/agents/benchmark/page.tsx, src/lib/api/bench.ts
Verified by	bench_api_bridge.test.ts (13), bench_runner_page.test.tsx (3), bench_tabs_rotate.test.tsx (2), bench_sankey_palette.test.tsx (42)

5.14.5 FR-063 Settings page

Requirement ID	FR-063
Name	Settings page
Description	The settings page shall present API keys, model providers and appearance options.
Actor	Dashboard user
Preconditions	—
Inputs	Form input.
Processing	Local React state only; nothing is sent to the backend.
Outputs	Rendered form.
Postconditions	—
Alternate flow	—
Exception flow	—
Dependencies	—
Priority	Low
Acceptance criteria	Changes do not persist (by design of current code).
Implementation status	Stub / simulated
Realised in	frontend/app/settings/page.tsx
Verified by	—

5.15 Retrieval pipeline library

5.15.1 FR-064 Document parsing and chunking

Requirement ID	FR-064
Name	Document parsing and chunking
Description	The retrieval library shall parse text/Markdown, HTML and CSV/TSV, hash content (SHA-256) and split it into overlapping fragments with citations.
Actor	Library caller
Preconditions	—
Inputs	bytes, MIME type or filename.
Processing	ParserRegistry.pick; Chunker(800, 200) by default.
Outputs	ParsedDocument + Fragment list.
Postconditions	Fragments indexed in FragmentStore.
Alternate flow	Unknown types parsed as text.

Exception flow	ValueError for invalid chunker arguments.
Dependencies	—
Priority	Medium
Acceptance criteria	ARCHITECTURE.md (35 KB) → 57 chunks in 7.2 ms (measured).
Implementation status	Implemented as library (not wired to API)
Realised in	rag/pipeline.py
Verified by	test_memory_rag::test_parser_*, test_chunker_*, test_ingest_*

5.15.2 FR-065 Hybrid lexical search

Requirement ID	FR-065
Name	Hybrid lexical search
Description	The retrieval library shall rank fragments with Okapi BM25 and fuse rankings (BM25 and an optional vector callback) with RRF.
Actor	Library caller
Preconditions	Documents ingested.
Inputs	query, top_k.
Processing	BM25 (k1 1.5, b 0.75); RRF k = 60; normalise.
Outputs	SearchResult list with provenance flags.
Postconditions	—
Alternate flow	Vector half empty unless set_vector_search is configured.
Exception flow	—
Dependencies	FR-064
Priority	Medium
Acceptance criteria	Median query latency 0.11–0.19 ms over 141 fragments (measured).
Implementation status	Implemented as library (not wired to API)
Realised in	rag/pipeline.py: FragmentStore, HybridSearch
Verified by	test_memory_rag::test_fragment_store_bm25_single_term_hits_the_relevant_chunk, test_hybrid_search_rrf_merges_both_halves

6 Non-Functional Requirements

Non-functional requirements are stated with a measurable criterion wherever the code or the measurements allow it. Performance figures were measured in-process with the FastAPI TestClient on the audit machine (Windows 11 Home (10.0.26200), Python 3.13.2, Node.js 20.18.1, FastAPI 0.139.2, Pydantic 2.13.4, SQLAlchemy 2.0.51, Ollama with qwen2.5-coder:7b-instruct-q4_K_M (7.6 B parameters, Q4_K_M, 4.68 GB)); they exclude network latency and are indicative of a single-user laptop deployment, not of a production server.

6.1 Performance

Table 6.1: Performance requirements

ID	Requirement	Measure / criterion	Evidence	Status
----	-------------	---------------------	----------	--------

ID	Requirement	Measure / criterion	Evidence	Status
NFR-PERF-01	Lightweight read endpoints shall respond quickly in-process.	p95 $\leq$ 10 ms for /health/livez, /v1/kernel/state, /v1/agents, /v1/conversations (25 rows), /v1/documents, /metrics.	Measured p95: 1.6, 1.6, 1.6, 7.9, 5.9, 5.2 ms (TestClient, n = 100–200).	Met
NFR-PERF-02	Planning shall add negligible latency compared with LLM inference.	PlannerSvc.plan median $\leq$ 5 ms.	Measured median 1.05 ms, p95 1.64 ms (150 plans).	Met
NFR-PERF-03	Conversation creation shall return before the run completes.	POST /v1/conversations $\leq$ 2 s.	Measured 0.82 s (Ollama up) and 1.73 s (Ollama down, includes ~1.5–2 s provider probe).	Met
NFR-PERF-04	Plan/execute endpoints should not pay a provider-probe cost on every request.	Probe once per process.	build services from container probes Ollama per request; /v1/runs/plan p50 1.71 s when Ollama is down.	Not met
NFR-PERF-05	Local LLM throughput shall be characterised.	Report tokens/s for representative prompts.	Measured $\approx$ 23–24 completion tokens/s (qwen2.5-coder 7B Q4_K_M); 1024-token prompt exceeded 120 s.	Met (characterised)

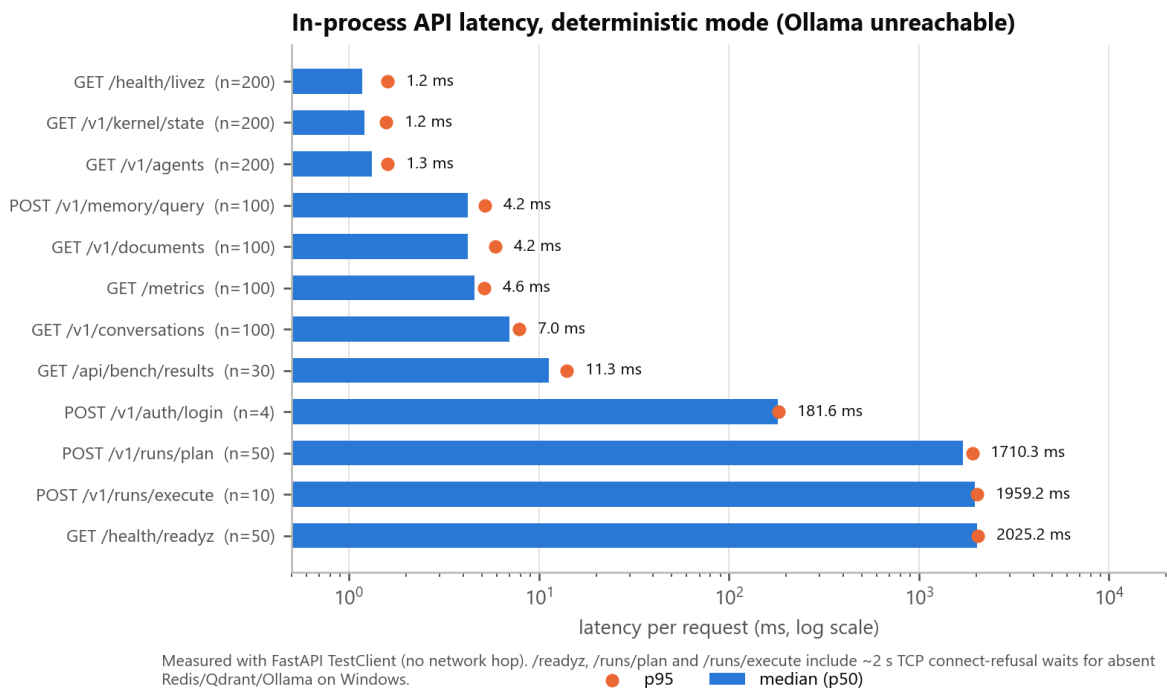
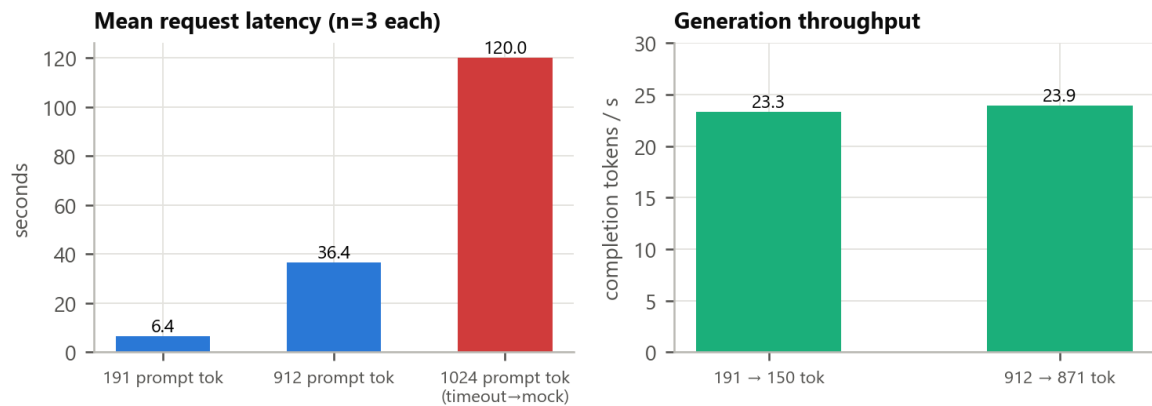


Figure 6.1: Measured in-process API latency (deterministic mode)

Read endpoints answer in 1–11 ms at the median. Login costs $\approx$ 180 ms because a password hash is computed and verified per request. Plan, execute and readiness requests take about two seconds in this measurement because each builds a fresh service pair that probes the (deliberately unreachable) Ollama port, and readiness probes absent Redis and Qdrant; on Windows a refused TCP connection takes $\approx$ 2 s to fail. With Ollama running, the same plan request took 0.64 s (measured).

GET /llm/benchmark against local Ollama (qwen2.5-coder:7b-instruct-q4_K_M)

Measured 2026-10-06 on the author's laptop (CPU/GPU not profiled). The 1024-token requests exceeded the 120 s wait_for timeout and the endpoint returned mock values.

Figure 6.2: Measured local-LLM latency and throughput through GET /llm/benchmark

Prompts of 191 and 912 tokens completed in 6.4 s and 36.4 s respectively, i.e. about 23–24 generated tokens per second. Prompts of 1 024 tokens exceeded the endpoint's 120-second timeout and the endpoint returned mock values with `used_mock=true`, demonstrating the documented fallback.

6.2 Scalability

Table 6.2: Scalability requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-SCAL-01	List endpoints shall paginate.	Keyset pagination, limit ≤ 200 .	PageParams + <code>_paginate_query</code> (id > cursor).	Met
NFR-SCAL-02	The backend should support horizontal scaling.	Shared state externalised.	Rate limiter, metrics rings, memory tiers, kernel spans and artifact repos are process-local.	Not met

6.3 Reliability

Table 6.3: Reliability requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-REL-01	A failure in one plan step shall not abort the run.	Step exceptions mark the step FAILED; remaining steps execute.	AgentSvc.run_step try/except; summary reports failed count.	Met
NFR-REL-02	LLM unavailability shall not break execution.	Deterministic fallback.	Measured identical reports with Ollama unreachable.	Met
NFR-REL-03	Transient provider errors shall be retried.	≤ 4 attempts with jittered back-off on 408/425/429/5xx.	llm/_retry.py with <code>_provider_retry</code> .	Met

6.4 Availability

Table 6.4: Availability requirements

ID	Requirement	Measure / criterion	Evidence	Status
----	-------------	---------------------	----------	--------

ID	Requirement	Measure / criterion	Evidence	Status
NFR-AVL-01	Optional dependencies shall not make the service unhealthy.	Redis/Qdrant down → readiness 'warn', HTTP 200.	Measured readyz status 'warn', ok=true.	Met
NFR-AVL-02	Container health checks shall be defined.	Compose healthcheck on /health/livez.	docker-compose.yml (not executed in this audit).	Not verified

6.5 Security

Table 6.5: Security requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-SEC-01	Credentials shall be stored as salted slow hashes.	bcrypt (12 rounds) or scrypt ($N = 2^{15}$).	security.PasswordHasher; login $\approx$ 180 ms per request (measured).	Met
NFR-SEC-02	Tokens shall be integrity-protected and short-lived.	HS256 JWT, constant-time compare, access 15 min, refresh 14 days; HMAC-SHA256 capability tokens with expiry.	security.JWTService; capability_gate.py; measured tamper → 401/403.	Met
NFR-SEC-03	Resource APIs shall require authentication.	All /v1 resource routes authenticated.	Only /v1/auth/me requires a JWT; bench routes require capability tokens.	Not met
NFR-SEC-04	Brute-force login shall be throttled.	≤ 5 login attempts / 60 s per client+user.	Measured 5×401 then 429 retry after s 60.	Met
NFR-SEC-05	Untrusted code/tool execution shall be contained.	Shell dry-run by default; AST/builtins allow-list; path-root confinement; http(s)-only fetch.	Measured tool containment cases (8).	Partially met (no SSRF filter; sandbox thread cannot be killed)

6.6 Privacy

Table 6.6: Privacy requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-PRIV-01	Secrets shall not appear in logs.	Redaction of sensitive keys and bearer values.	logging._scrub_secrets.	Met
NFR-PRIV-02	Users shall only see their own data.	Per-user filtering.	No ownership checks on conversations/documents.	Not met

6.7 Maintainability

Table 6.7: Maintainability requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-MNT-01	Code shall follow a ports-and-adapters structure with typed DTOs.	Domain types in types.py; ports in core/ports.py; DI container.	Code inspection.	Met
NFR-MNT-02	Automated tests shall gate quality.	≥ 40 % line coverage (pyproject gate); lint clean.	Measured 70.01 % coverage, 310 passed / 2 xfail; frontend tsc 0, ESLint 0.	Met

6.8 Usability

Table 6.8: Usability requirements

ID	Requirement	Measure / criterion	Evidence	Status
----	-------------	---------------------	----------	--------

ID	Requirement	Measure / criterion	Evidence	Status
NFR-USA-01	The UI shall give feedback for long-running work and errors.	Toasts and loading states; demo-data banners.	Screenshots; http.ts emitHttpToast.	Met

6.9 Accessibility

Table 6.9: Accessibility requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-ACC-01	The UI should meet WCAG 2.1 AA.	axe audit 0 critical issues.	scripts/axe_audit.mjs exists in CI; not executed in this audit.	Not verified

6.10 Portability

Table 6.10: Portability requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-PORT-01	The backend shall run on Windows, Linux and macOS with SQLite by default.	No OS-specific code paths in core.	Measured on Windows 11; CI targets ubuntu-latest.	Met (Windows measured)
NFR-PORT-02	The relational store should be switchable to PostgreSQL by URL.	DATABASE_URL change only.	JSONB-with-SQLite-variant columns; start-up migrations use SQLite PRAGMA statements.	Partially met

6.11 Compatibility

Table 6.11: Compatibility requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-COMP-01	Python and Node versions shall be declared.	requires-python ≥ 3.10 , < 3.13 ; Node ≥ 20.17 .	pyproject.toml; package.json. Tests also pass on Python 3.13.2 (measured) despite the < 3.13 bound.	Met

6.12 Observability

Table 6.12: Observability requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-OBS-01	Requests shall be traceable.	X-Request-ID on every response; structured logs; Prometheus counters.	Measured.	Met
NFR-OBS-02	Agent steps shall be measurable.	Step and token counters exported.	Counters defined but never incremented by the runtime.	Not met

6.13 Fault tolerance

Table 6.13: Fault tolerance requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-FT-01	Unhandled exceptions shall not leak internals.	Generic 500 envelope; exception type only in development.	api/main.py _generic_exception_handler.	Met

6.14 Recoverability

Table 6.14: Recoverability requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-REC-01	In-memory state should be recoverable after restart.	Memory tiers persisted.	Persistence hooks not wired (FR-034).	Not met

6.15 Resource utilisation

Table 6.15: Resource utilisation requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-RES-01	Kernel buffers shall be bounded.	Spans $\leq 10\,000$; metrics rings $\leq 10\,000/20\,000$; T1 $\leq 1\,000$.	Code inspection.	Met
NFR-RES-02	Uploads should be bounded.	Maximum request size.	upload_document reads the full file into memory; no size limit.	Not met

6.16 Reproducibility

Table 6.16: Reproducibility requirements

ID	Requirement	Measure / criterion	Evidence	Status
NFR-DET-01	Equal (goal, seed) inputs shall produce bit-identical plans.	SHA-256 equality across runs.	Measured 50/50 goals $\times$ 3 runs; 5 seeds $\rightarrow$ 5 distinct hashes.	Met

6.17 Software quality attributes

Table 6.17: Software quality attributes

Attribute	How NOESIS addresses it	Assessment
Adaptability	New agents, tools, providers and plugins are added through registries (<code>_AGENT_REGISTRY</code> , <code>ToolRegistry</code> , <code>factory</code> , <code>PluginManager</code>).	Good
Correctness	Pydantic validation at boundaries; 310 passing tests; deterministic planner. Agent outputs are heuristic, so <i>task</i> correctness is not guaranteed.	Partial
Flexibility	Ports-and-adapters: repositories, chat and embedding ports are injectable.	Good
Interoperability	OpenAPI 3.1 schema; Prometheus format; UAP with a .proto mirror; OpenAI-compatible provider path.	Good
Maintainability	Consistent module boundaries; ruff configuration; 70 % coverage; but large files (agents/core.py 2 105 lines, v1_m5.py 1 016 lines) and duplicated route groups.	Fair
Portability	Pure-Python and Node; SQLite default. Start-up migrations use SQLite-specific PRAGMA statements.	Fair
Reliability	Fallbacks for LLM, planner and persistence errors; failures are logged rather than crashing the run.	Good
Reusability	Memory bus, RAG pipeline, tool registry and stats module are framework-independent libraries.	Good
Robustness	Generic exception handler; schema validation; but invalid plans surface as HTTP 500 instead of 422.	Fair
Testability	Deterministic agents and injectable ports make unit tests straightforward (312 backend, 105 frontend tests).	Good
Usability	Dashboard with toasts, demo banners and codename badges; several pages show placeholder data.	Fair

7 User Requirements

NOESIS has no persisted user accounts in the current build: the `users` table exists but the login route authenticates against a two-entry built-in directory (`admin`, `demo`). Roles are carried in the JWT but only `/v1/auth/me` checks a token, and no route checks a role. The tables below therefore distinguish the *intended* role model from what is *enforced*.

Table 7.1: User roles, permissions and enforcement

Role	Characteristics	Responsibilities	Permissions (intended)	Enforced today
Dashboard user	Developer or researcher using a browser	Create runs, read threads, review results	Own conversations and documents	None — all <code>/v1</code> resource routes are open
API client	Script, CLI or service	Plan/execute, manage records, monitor	Scoped by JWT	JWT only for <code>/auth/me</code>
Administrator (<code>admin</code> role)	Operator with the admin credential	Health, kernel, metrics, user support	Admin-only observability and kernel views	Not enforced (test marked <code>xfail</code>)
Evaluator	Researcher running benchmarks	Run harness, inspect results	<code>bench.results.read</code> , <code>llm.benchmark.read</code> capabilities	Enforced by HMAC capability gate
Agent (system principal)	Runtime-created	Execute one plan step	Only its declared capabilities + tool hints	Enforced by AgentRoster, ToolRegistry and Kernel.syscall

7.1 User workflows

1. **Run a goal** — open *Conversations* → *New run*, enter the goal, choose the lead agent label and tags, submit, then watch the thread update (UC-04).
2. **Review history** — filter the conversation list by status/agent or search text; open a thread; rename, tag or delete it (UC-05, UC-06).
3. **Operate the kernel** — call `/health/readyz`, `/v1/kernel/state`, `/v1/kernel/stats` and scrape `/metrics` (UC-13, UC-14).
4. **Evaluate** — run the harness scripts, then open *Bench Results* or call `/api/bench/results` with a capability token (UC-15, UC-16, UC-18).

7.2 Access restrictions

- Gated routes (`/llm/benchmark`, `/api/bench/results`) reject requests without a valid capability token (HTTP 403).
- `/v1/auth/me` and `/v1/auth/refresh` require a bearer token of the right `token_use`.
- Agents cannot invoke a tool without `TOOL_INVOKE` for that tool, and the roster refuses to spawn an agent lacking any declared capability.
- Production/staging start-up refuses the development secret, `DEBUG` and wildcard CORS; interactive API docs are disabled when `APP_ENV=production`.

8 Use Cases

18 use cases capture the externally visible behaviour. Figure 8.1 shows actors and use cases; each use case is then specified in tabular form.

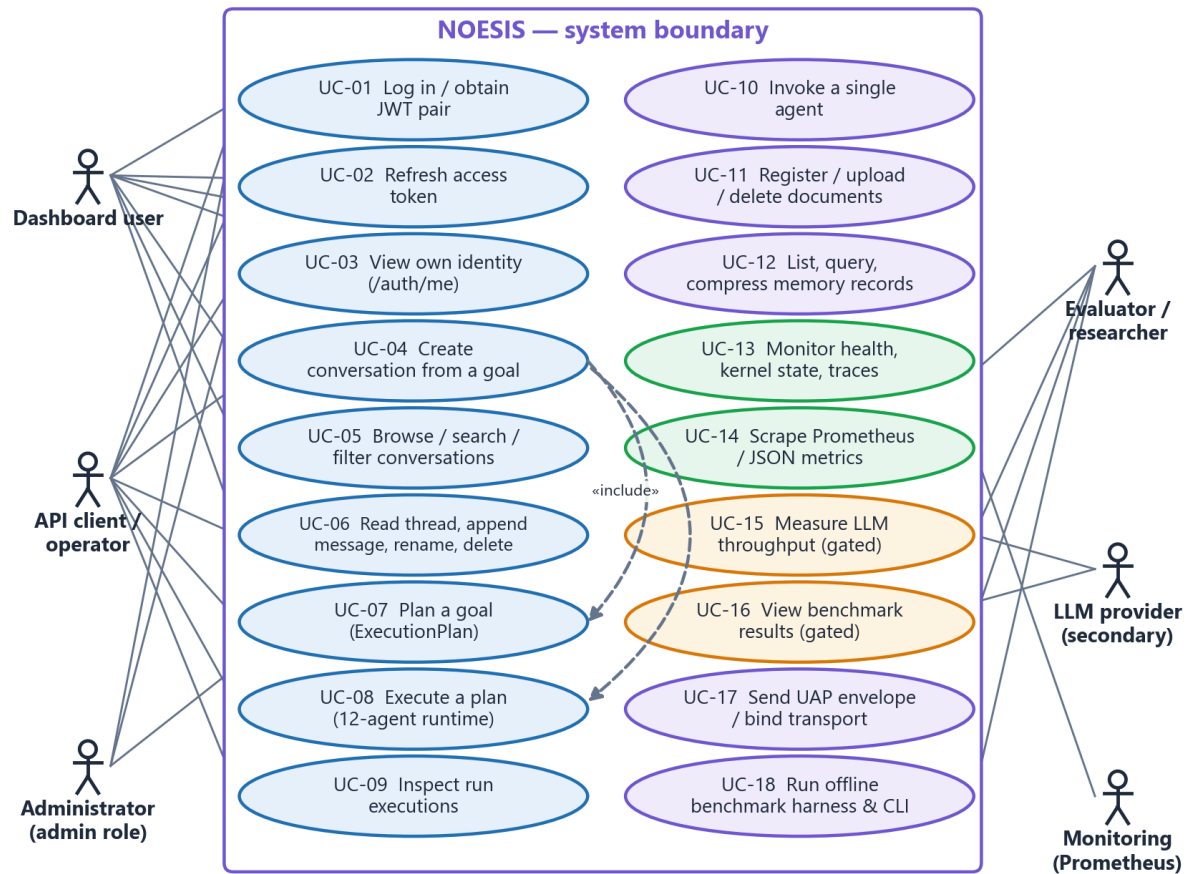


Figure 8.1: Use-case diagram

Human actors (left) and secondary/system actors (right) are connected to the use cases they participate in. UC-04 includes planning (UC-07) and execution (UC-08). Gated use cases are shown in amber, monitoring in green.

8.1 UC-01 Log in and obtain tokens

Use-case ID	UC-01
Name	Log in and obtain tokens
Primary actor	Dashboard user / API client
Secondary actors	—
Goal	Obtain an access/refresh token pair.
Preconditions	Backend running.
Trigger	User submits credentials.
Main flow	1. Client POSTs /v1/auth/login with user_id and password. 2. System validates the payload and applies the login rate limit. 3. System verifies the password hash. 4. System returns access (15 min) and refresh (14 days) tokens.
Alternate flows	A1: Client already holds a refresh token → UC-02.

Exception flows	1. E1: invalid credentials → 401. 2. E2: sixth attempt within 60 s → 429 with retry_after_s. 3. E3: malformed body → 422.
Postconditions	Client holds valid tokens.
Related requirements	FR-001

8.2 UC-02 Refresh access token

Use-case ID	UC-02
Name	Refresh access token
Primary actor	API client
Secondary actors	—
Goal	Extend a session without re-entering credentials.
Preconditions	Valid refresh token.
Trigger	Access token expiry.
Main flow	1. Client POSTs /v1/auth/refresh with Bearer refresh token. 2. System validates token_use='refresh'. 3. System returns a new access token and the same refresh token.
Alternate flows	—
Exception flows	1. E1: expired/tampered token → 401. 2. E2: > 20 refreshes/min → 429.
Postconditions	New access token issued.
Related requirements	FR-003

8.3 UC-03 View own identity

Use-case ID	UC-03
Name	View own identity
Primary actor	Authenticated client
Secondary actors	—
Goal	Confirm who the token represents.
Preconditions	Valid access token.
Trigger	Client calls /v1/auth/me.
Main flow	1. System decodes the token. 2. System returns sub, roles, jti, issued/expiry times.
Alternate flows	—
Exception flows	E1: missing or invalid token → 401.
Postconditions	—
Related requirements	FR-002

8.4 UC-04 Start an autonomous run from a goal

Use-case ID	UC-04
Name	Start an autonomous run from a goal
Primary actor	Dashboard user
Secondary actors	Local LLM (optional)
Goal	Have the agent pipeline decompose and work on a goal.
Preconditions	Frontend and backend reachable.
Trigger	User submits the New-run form.

Main flow	1. User enters a goal, lead agent and tags on /conversations/new. 2. Studio POSTs /v1/conversations. 3. System stores the conversation and the goal message, computes the plan id and starts the background run. 4. Studio navigates to the thread page. 5. System executes the plan step by step (UC-08 included) and appends one message per step plus a summary. 6. Studio refreshes and shows the completed thread.
Alternate flows	1. A1: Ollama unreachable → steps use deterministic agent output only. 2. A2: Backend unreachable → toast 'Conversation created (local mock)' and list page shows mock data.
Exception flows	1. E1: planner exception → random plan id, run continues. 2. E2: run exception → 'Run failed' message, status failed.
Postconditions	Thread with step reports; status completed or failed.
Related requirements	FR-006, FR-013, FR-014, FR-017, FR-040, FR-060

8.5 UC-05 Browse and search conversations

Use-case ID	UC-05
Name	Browse and search conversations
Primary actor	Dashboard user
Secondary actors	—
Goal	Find prior runs.
Preconditions	—
Trigger	User opens /conversations.
Main flow	1. Studio GETs /v1/conversations. 2. User filters by status/agent or types a search term. 3. User opens a thread.
Alternate flows	A1: empty result → seeded mock list displayed.
Exception flows	E1: backend error → banner and mock data.
Postconditions	—
Related requirements	FR-007, FR-060

8.6 UC-06 Work with a thread

Use-case ID	UC-06
Name	Work with a thread
Primary actor	Dashboard user / API client
Secondary actors	—
Goal	Read, extend, rename or delete a conversation.
Preconditions	Conversation exists.
Trigger	User opens a thread or issues API calls.
Main flow	1. GET /v1/conversations/{id}. 2. POST ../messages to append. 3. PATCH to rename or change status/tags. 4. DELETE to remove.
Alternate flows	—
Exception flows	1. E1: unknown id → 404. 2. E2: invalid role → 422.
Postconditions	Conversation updated.
Related requirements	FR-008 – FR-012

8.7 UC-07 Plan a goal

Use-case ID	UC-07
Name	Plan a goal
Primary actor	API client
Secondary actors	—
Goal	Obtain a deterministic ExecutionPlan.
Preconditions	—
Trigger	POST /v1/runs/plan.
Main flow	1. Client sends goal and optional seed. 2. System returns PlannerReport with steps and dependencies.
Alternate flows	A1: same goal and seed later → identical plan.
Exception flows	E1: goal < 5 chars → 422.
Postconditions	Plan available for UC-08.
Related requirements	FR-013

8.8 UC-08 Execute a plan

Use-case ID	UC-08
Name	Execute a plan
Primary actor	API client
Secondary actors	Local LLM (optional)
Goal	Run every plan step and collect reports.
Preconditions	Valid plan.
Trigger	POST /v1/runs/execute.
Main flow	1. System topologically orders steps. 2. For each step the system mints a scoped token and runs the assigned agent (LLM-enriched when available). 3. System persists TaskExecution rows. 4. System returns run_id and per-step results.
Alternate flows	A1: unregistered agent type → AWAITING_INPUT for that step.
Exception flows	1. E1: cyclic dependencies → error. 2. E2: malformed plan → 500 envelope (validation inside handler).
Postconditions	Audit rows stored.
Related requirements	FR-014, FR-015, FR-040

8.9 UC-09 Inspect run executions

Use-case ID	UC-09
Name	Inspect run executions
Primary actor	API client / Operator
Secondary actors	—
Goal	Audit what each step did.
Preconditions	A run has executed.
Trigger	GET /v1/runs/{id}/executions.
Main flow	System returns TaskExecution rows.
Alternate flows	A1: pass the conversation id to obtain rows (current behaviour).
Exception flows	E1: real run_id returns an empty list (known defect).
Postconditions	—
Related requirements	FR-016

8.10 UC-10 Invoke a single agent

Use-case ID	UC-10
Name	Invoke a single agent
Primary actor	API client
Secondary actors	—
Goal	Test one agent in isolation.
Preconditions	—
Trigger	POST /v1/agents/invoke.
Main flow	1. Client names planner, research, coding or reflection and supplies state. 2. System runs the agent with a scoped token and returns the state.
Alternate flows	—
Exception flows	E1: other agent names → 400.
Postconditions	—
Related requirements	FR-019

8.11 UC-11 Manage documents

Use-case ID	UC-11
Name	Manage documents
Primary actor	Dashboard user / API client
Secondary actors	—
Goal	Register or upload knowledge files.
Preconditions	—
Trigger	Upload or create request.
Main flow	1. Client creates metadata or uploads a file. 2. System records size, type and an estimated chunk count. 3. Client lists, opens or deletes records.
Alternate flows	—
Exception flows	E1: unknown id → 404.
Postconditions	Document metadata stored (file content is not retained).
Related requirements	FR-027 – FR-029

8.12 UC-12 Query and compact memory records

Use-case ID	UC-12
Name	Query and compact memory records
Primary actor	API client
Secondary actors	—
Goal	Retrieve or compress long-term memory rows.
Preconditions	—
Trigger	Memory API call.
Main flow	Client lists, queries (Jaccard scoring) or compresses memory rows.
Alternate flows	—
Exception flows	E1: invalid bounds → 422.
Postconditions	—
Related requirements	FR-030 – FR-032

8.13 UC-13 Monitor health and kernel

Use-case ID	UC-13
Name	Monitor health and kernel
Primary actor	Operator / Administrator
Secondary actors	—
Goal	Check service health and kernel internals.
Preconditions	—
Trigger	Operator or probe call.
Main flow	1. GET /health/readyz. 2. GET /v1/kernel/state, /stats, /traces.
Alternate flows	A1: Redis/Qdrant absent → 'warn'.
Exception flows	E1: SQL down → error envelope.
Postconditions	—
Related requirements	FR-036, FR-039, FR-048

8.14 UC-14 Scrape metrics

Use-case ID	UC-14
Name	Scrape metrics
Primary actor	Prometheus
Secondary actors	—
Goal	Collect HTTP metrics.
Preconditions	—
Trigger	Scrape interval.
Main flow	GET /metrics returns OpenMetrics text.
Alternate flows	A1: JSON snapshot via /metrics_json.
Exception flows	—
Postconditions	—
Related requirements	FR-049

8.15 UC-15 Measure LLM throughput

Use-case ID	UC-15
Name	Measure LLM throughput
Primary actor	Evaluator
Secondary actors	Local LLM
Goal	Characterise local model speed.
Preconditions	Valid capability token with llm.benchmark.read.
Trigger	Bench Runner button or API call.
Main flow	1. System checks the token. 2. System sends a prompt of N tokens to Ollama. 3. System returns latency and tokens/s.
Alternate flows	A1: Ollama unreachable or timeout → mock values with used_mock=true.
Exception flows	E1: token missing/invalid/expired/denied → 403.
Postconditions	—
Related requirements	FR-038, FR-052

8.16 UC-16 View benchmark results

Use-case ID	UC-16
Name	View benchmark results
Primary actor	Evaluator
Secondary actors	—
Goal	Inspect the latest evaluation tables.
Preconditions	Valid token with <code>bench.results.read</code> .
Trigger	Bench Hub page or API call.
Main flow	1. System locates the newest <code>tables_for_paper.json</code> . 2. System returns validated results with source metadata.
Alternate flows	1. A1: <code>refresh=true</code> re-aggregates CSVs first. 2. A2: no tables → demo payload flagged <code>meta.demo=true</code> .
Exception flows	E1: 403 from the gate.
Postconditions	—
Related requirements	FR-053, FR-062

8.17 UC-17 Exchange UAP envelopes

Use-case ID	UC-17
Name	Exchange UAP envelopes
Primary actor	Integrator
Secondary actors	—
Goal	Exercise the protocol endpoints.
Preconditions	—
Trigger	API call.
Main flow	List transports; bind in-process transport; send envelope and receive its hash.
Alternate flows	—
Exception flows	E1: invalid body → 422.
Postconditions	No message is delivered (stub).
Related requirements	FR-056

8.18 UC-18 Run offline evaluation and CLI

Use-case ID	UC-18
Name	Run offline evaluation and CLI
Primary actor	Evaluator / Developer
Secondary actors	Local LLM
Goal	Produce benchmark artefacts and operate the kernel from a shell.
Preconditions	Python environment; Ollama for adaptive mode.
Trigger	Command invocation.
Main flow	1. Run <code>scripts/bench_se50_batch.py</code> , <code>bench_humaneval.py</code> , <code>bench_mbbp.py</code> (or <code>run_w7_weekend.py</code>). 2. Harness writes CSV/JSON under <code>docs/eval</code> . 3. Stats module aggregates into <code>tables_for_paper.json</code> .
Alternate flows	A1: network failure → bundled stub corpora.
Exception flows	E1: adaptive model missing → <code>SystemExit</code> with pull instructions.

Postconditions	Artefacts available to UC-16.
Related requirements	FR-054, FR-058

9 Data Requirements

9.1 Input data

Table 9.1: Input data

Input	Source	Format	Validation
Goal / conversation title	Dashboard form, API	UTF-8 string	1–200 chars (conversation); ≥ 5 chars (plan)
Messages	API	JSON {role, content, name?, tool_call_id?}	role $\in 6$ literals; content ≥ 1 char; extra fields rejected
Plans	API (/runs/execute)	ExecutionPlan JSON	contiguous indices $0..n-1$, ≥ 1 step, description ≥ 3 chars, goal ≥ 5 chars
Credentials	Login form/API	JSON {user_id, password}	2–128 / 8–4096 chars; whitespace stripped; sanitised
Capability tokens	HTTP header	b64url(JSON).b64url(HMAC)	Signature, schema (<code>expires_at_unix_s</code> $\geq 1.7e9$), expiry, capability, deny masks
Documents	Multipart upload / JSON	Any MIME type	No size limit; content not parsed by the API
Benchmark corpora	Files / downloads	JSON, CSV	SE-50 corpus fields (id, category, difficulty, goal_string, accept_criteria ...)
Configuration	Environment / .env	Key=value	Pydantic Settings with bounds (Appendix D)

9.2 Output data

Table 9.2: Output data

Output	Consumer	Format
API responses	Dashboard, clients	JSON APIEnvelope
Agent reports	Conversation messages, run results	Pydantic $\rightarrow$ JSON (12 report types)
Audit rows	Operators	task_executions table
Metrics	Prometheus	Text exposition v0.0.4
Logs	Operators	structlog JSON (prod) / console (dev)
Workspace files	Users	Files under var/workspace written by the files tool
Evaluation artefacts	Researchers, dashboard	CSV, JSON (tables_for_paper.json)

9.3 Key domain objects

Table 9.3: Principal domain objects

Object	Fields (abridged)	Constraints
ExecutionPlan	id (UUID/UUIDv5), goal, steps[PlanStep], reasoning, created_at, metadata	≥ 1 step; contiguous indices
PlanStep	id, index, description, assigned_agent, dependencies[UUID], tool_hints[], rag_query, status, confidence, result, error, started_at, finished_at	confidence $\in [0, 1]$
ChatMessage	id, role, content, name, tool_call_id, tool_calls, timestamp, metadata	tool role $\Rightarrow$ tool_call_id; else content or tool_calls
TaskExecution	conversation_id, run_id, plan_step_id, agent_type, description, status, tokens, cost_usd, duration_ms, error, result_preview, metadata	status = TaskStatus int
MemoryEntry (tier)	memory_id, content, kind, importance, access_count, citations, tags, scope, scope_id, timestamps, embedding, metadata	score = importance $\cdot$ (0.6 + 0.3 $\cdot$ recency + 0.1 $\cdot$ access)

Object	Fields (abridged)	Constraints
CapabilityTokenPayload	owner, workspace_id, expires_at_unix_s, capabilities[], deny_masks[], issued_at_unix_s	owner/workspace non-empty
UAPMessage	message_id, intent (22), origin, goal, evidence[], confidence, artifacts[], span, ...	wire-stable; confidence rounded

9.4 Validation rules

- All request models forbid unknown fields (`extra='forbid'`), except the UAP envelope which allows extras.
- UUID path parameters are validated (malformed → 422).
- Pagination `limit` ∈ [1, 200] (lists) and [1, 500] (embedded messages); kernel traces `limit` ∈ [1, 5000].
- Memory importance, `min_importance` and confidences are bounded to [0, 1].
- Validation failures of request data return 422 with `meta.errors[{{loc, msg, type}}]`; failures inside handlers return 500 with the same structure.

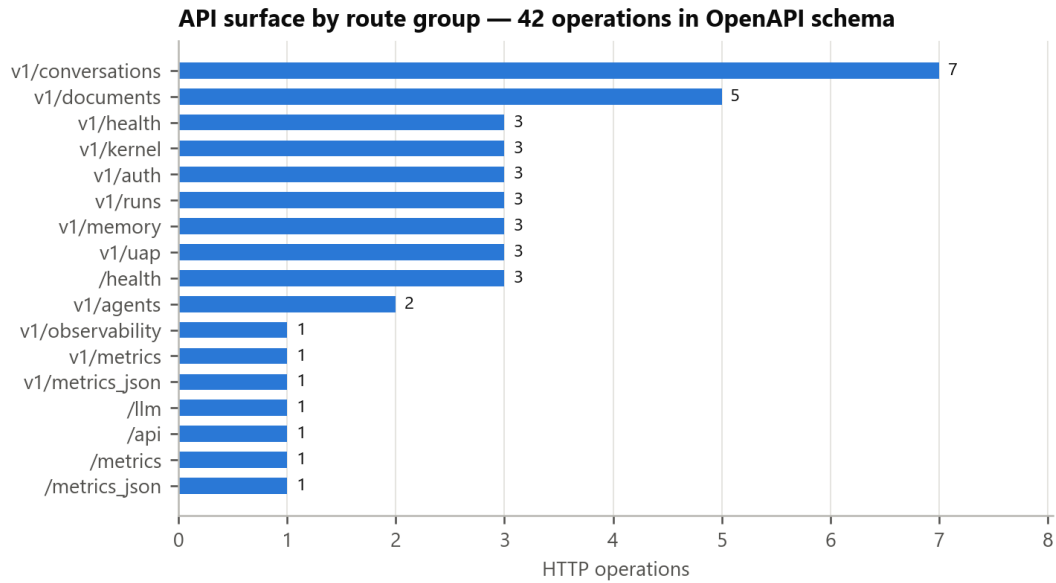
9.5 Storage requirements and data lifecycle

Table 9.4: Data lifecycle

Data	Store	Retention	Deletion
Conversations, messages	SQLite	Until deleted	DELETE conversation (messages cascade)
Task executions	SQLite	Indefinite	conversation_id set NULL on conversation delete; no API
Documents (metadata)	SQLite	Until deleted	DELETE document
Memories	SQLite	Indefinite (compression flag only)	No API
Memory tiers T1–T6	Process memory	Process lifetime	Lost on restart (persistence not wired)
Spans, metric rings	Process memory	Bounded ring buffers	Overwritten
Uploaded file bytes	—	Not stored	—
Workspace files	var/workspace	Until deleted by the files tool or user	files delete

10 API Specification

The backend publishes an OpenAPI 3.1 document at `/openapi.json` (disabled in production) with **42 operations on 36 paths**, plus a hidden landing route `GET /`. All `/v1` routes are mounted with prefix `/v1`; health and metrics routers are additionally mounted at the root, so several operations are aliases. Example requests and responses below were captured from the running application during the audit (identifiers and timestamps are real but disposable).



Extracted from `create_app().openapi()` of the working tree. The hidden GET / landing route and /docs, /redoc are not counted.

Figure 10.1: Distribution of API operations by route group (from the live OpenAPI schema)

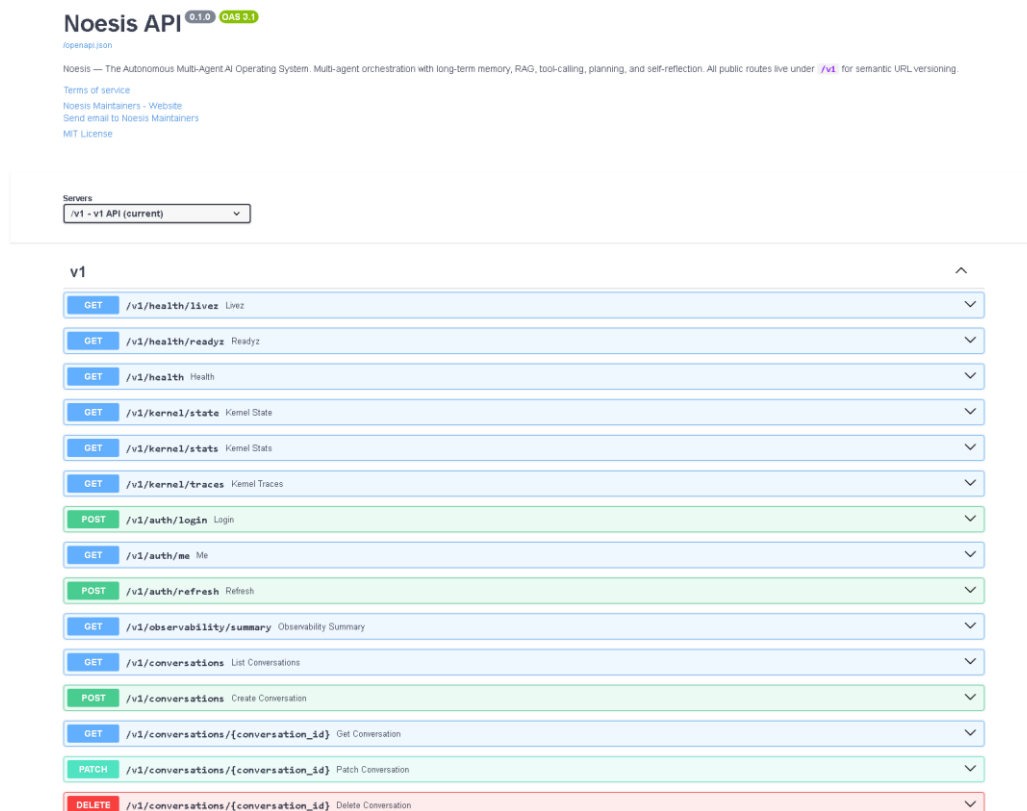


Figure 10.2: Interactive API documentation (Swagger UI) served at /docs — screenshot of the running backend

10.1 Common conventions

Table 10.1: API conventions

Element	Specification
Base URL	<code>http://<host>:8000</code> (dashboard proxy: <code>/api/backend</code>)

Element	Specification
Content type	application/json; UTF-8 (multipart/form-data for upload)
Response envelope	{ok, data, error, request_id, meta}
Correlation	X-Request-ID (request optional, response always)
Pagination	cursor (opaque, last id) + limit; meta.next_cursor
Errors	401, 403, 404, 422, 429, 500 — always enveloped
Versioning	URI prefix /v1; root aliases for probes/metrics

10.2 Health

10.2.1 GET /v1/health/livez

Endpoint	/v1/health/livez
HTTP method	GET
Purpose	Liveness probe mounted under /v1.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	As /health/livez
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

10.2.2 GET /v1/health/readyz

Endpoint	/v1/health/readyz
HTTP method	GET
Purpose	Readiness probe mounted under /v1.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	As /health/readyz
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

10.2.3 GET /v1/health

Endpoint	/v1/health
HTTP method	GET
Purpose	Readiness alias mounted under /v1.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	As /health/readyz
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 2030.4 ms):

```
GET /v1/health
-
{
  "ok": true,
  "data": {
    "status": "warn",
    "checks": {
      "sql": {
        "name": "sql",
        "status": "pass",
        "latency_ms": 3.88,
        "error": null
      },
      "redis": {
        "name": "redis",
        "status": "warn",
        "latency_ms": 0.0,
        "error": "Error 22 connecting to 127.0.0.1:6379. The remote computer refused the network connection."
      },
      "qdrant": {
        "name": "qdrant",
        "status": "warn",
        "latency_ms": 0.0,
        "error": "No module named 'qdrant_client'"
      }
    }
  },
  "error": null,
  "request_id": "6b0346c33daa4dfe8d400bd8395df0ce",
  "meta": {}
}
```

10.2.4 GET /health/livez

Endpoint	/health/livez
HTTP method	GET
Purpose	Liveness probe; always reports pass while the process runs.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	200 {status: pass}
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 2.9 ms):

```
GET /health/livez
-
{
  "ok": true,
  "data": {
    "status": "pass"
  },
  "error": null,
  "request_id": "b32a26fc2c534016b6c3275020854ca6",
  "meta": {}
}
```

10.2.5 GET /health/readyz

Endpoint	/health/readyz
HTTP method	GET

Purpose	Readiness probe: SQL (hard), Redis and Qdrant (soft) checks run concurrently.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	200 {status pass warn, checks {sql,redis,qdrant}}; error envelope when SQL fails
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 2037.6 ms):

```
GET /health/readyz
-
{
  "ok": true,
  "data": {
    "status": "warn",
    "checks": {
      "sql": {
        "name": "sql",
        "status": "pass",
        "latency_ms": 19.9,
        "error": null
      },
      "redis": {
        "name": "redis",
        "status": "warn",
        "latency_ms": 0.0,
        "error": "Error 22 connecting to 127.0.0.1:6379. The remote computer refused the network connection."
      },
      "qdrant": {
        "name": "qdrant",
        "status": "warn",
        "latency_ms": 0.0,
        "error": "No module named 'qdrant_client'"
      }
    }
  },
  "error": null,
  "request_id": "28b1fe86c7e24f8a86e29cd8f511a5b0",
  "meta": {}
}
```

10.2.6 GET /health

Endpoint	/health
HTTP method	GET
Purpose	Alias of /health/readyz.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	As readyz
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

10.3 Kernel

10.3.1 GET /v1/kernel/state

Endpoint	/v1/kernel/state
-----------------	------------------

HTTP method	GET
Purpose	Kernel lifecycle state, uptime, registered services/handlers and span count.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	200 envelope {kernel_state, uptime_s, started_at, agents_registered, services_registered[], handlers_registered[], spans_captured, spans_capacity, generated_at}
Status codes	200, 500 (kernel missing)
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 2.2 ms):

```
GET /v1/kernel/state
-
{
  "ok": true,
  "data": {
    "kernel_state": "running",
    "uptime_s": 4.292,
    "started_at": "2026-10-06T14:25:45.156959+00:00",
    "agents_registered": 0,
    "services_registered": [
      "artifacts",
      "conversations",
      "documents",
      "knowledge_objects",
      "memories",
      "messages",
      "task_executions",
      "users"
    ],
    "handlers_registered": [
      "get_clock",
      "memory_read",
      "memory_write",
      "model_chat",
      "model_route",
      "tool_invoke"
    ],
    "spans_captured": 0,
    "spans_capacity": 10000,
    "generated_at": "2026-10-06T14:25:49.448826Z"
  },
  "error": null,
  "request_id": "b46999aa67ef4bee87e93dbd6ae88d93",
  "meta": {}
}
```

10.3.2 GET /v1/kernel/stats

Endpoint	/v1/kernel/stats
HTTP method	GET
Purpose	Scheduler, memory-allocator and model-router aggregates.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	200 envelope {scheduler{...}, memory_allocator{zones, agents, active_leases, registered_agents}, model_router{profiles_registered, profiles[], outcomes_recorded, weights}}
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 1.9 ms):

```
GET /v1/kernel/stats
-
{
  "__truncated__": true,
  ... (truncated)
```

10.3.3 GET /v1/kernel/traces

Endpoint	/v1/kernel/traces
HTTP method	GET
Purpose	Most recent syscall spans (OTel-shaped).
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	limit (query, integer, required: no, ≥ 1 ; ≤ 5000 ; default 50)
Request body	None
Response body	200 envelope {limit, returned, spans[{trace_id, span_id, syscall, success, denied, latency_ms, error, ts}]}
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 1.7 ms):

```
GET /v1/kernel/traces?limit=5
-
{
  "ok": true,
  "data": {
    "limit": 5,
    "returned": 0,
    "spans": []
  },
  "error": null,
  "request_id": "a720be55be2d48689926666c16923f87",
  "meta": {}
}
```

10.4 Authentication

10.4.1 POST /v1/auth/login

Endpoint	/v1/auth/login
HTTP method	POST
Purpose	Authenticate and issue an access/refresh JWT pair.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. application/json 2. user_id: string — required: yes; min len 2; max len 128 3. password: string — required: yes; min len 8; max len 4096
Response body	200 TokenPair {access_token, refresh_token, token_type, expires_in_s}
Status codes	200, 401, 422, 429
Error responses (observed)	1. 401: "invalid credentials" 2. 401: "invalid credentials" 3. 422: "Request validation failed" 4. 429: "HTTP 429"

Request body	None
Response body	200 TokenPair
Status codes	200, 401, 429
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 1.9 ms):

```
POST /v1/auth/refresh
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJub2VzaXM6ZGV2ZWxvcG1lbnQiLCJhdwQiOiJub2VzaXM6YXBPiIiwic3V
iIjoizGVtbyIsImp0aSI6IjdmZTQzNDM0YjFjYzRjOWY4YThkOTYNTQ0NWQ3MDc3In0.mgZQ-6icx34efhsRilcgVQNe4SQdFLB_tTcuImDEEYI",
"refresh_token":
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpc3MiOiJub2VzaXM6ZGV2ZWxvcG1lbnQiLCJhdwQiOiJub2VzaXM6YXBPiIiwic3V
iIjoizGVtbyIsImp0aSI6IjdmZTQzNDM0YjFjYzRjOWY4YThkOTYNTQ0NWQ3MDc3In0.mgZQ-6icx34efhsRilcgVQNe4SQdFLB_tTcuImDEEYI",
_q3qkZ06crgewoQj1YMBobzkxhabpIUzTG40NCrI2c",
"token_type": "Bearer",
"expires_in_s": 900
}
```

10.5 Observability and metrics

10.5.1 GET /v1/observability/summary

Endpoint	/v1/observability/summary
HTTP method	GET
Purpose	Window summary of request latency, tokens, cost and tool traces (in-memory rings).
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	window_s (query, integer, required: no, default 600)
Request body	None
Response body	200 ObservabilitySummary (all counters stay 0 in the current build)
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 1.8 ms):

```
GET /v1/observability/summary?window_s=600
-
{
  "window_s": 600,
  "total_requests": 0,
  "total_token_cost_usd": 0.0,
  "total_prompt_tokens": 0,
  "total_completion_tokens": 0,
  "latency_p50_ms": 0,
  "latency_p95_ms": 0,
  "tools": [],
  "errors": 0
}
```

10.5.2 GET /v1/metrics

Endpoint	/v1/metrics
HTTP method	GET

Purpose	Prometheus exposition (same as /metrics).
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	text/plain; version=0.0.4
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

10.5.3 GET /v1/metrics_json

Endpoint	/v1/metrics_json
HTTP method	GET
Purpose	JSON snapshot of all counters and summaries.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	200 envelope {counters{...}, summaries{...}}
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 1.9 ms):

```
GET /v1/metrics_json
-
{
  "__truncated__": true,
  ... (truncated)
```

10.5.4 GET /metrics

Endpoint	/metrics
HTTP method	GET
Purpose	Prometheus scrape endpoint.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	text/plain; version=0.0.4
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

10.5.5 GET /metrics_json

Endpoint	/metrics_json
HTTP method	GET
Purpose	JSON metrics snapshot (root mount).
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None

Response body	200 envelope
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

10.6 Conversations

10.6.1 GET /v1/conversations

Endpoint	/v1/conversations
HTTP method	GET
Purpose	Paginated, filterable conversation list.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	1. <code>status</code> (query, string null, required: no) 2. <code>agent</code> (query, string null, required: no) 3. <code>search</code> (query, string null, required: no) 4. <code>cursor</code> (query, string null, required: no) 5. <code>limit</code> (query, integer, required: no, ≥ 1; ≤ 200; default 25)
Request body	None
Response body	200 envelope [ConversationEnriched] + meta {limit, next_cursor, total}
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 7.9 ms):

```
GET /v1/conversations?limit=5
-
{
  "ok": true,
  "data": [
    {
      "id": "7e870c3d-bd34-4e74-b9f2-748e40c3fc91",
      "title": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
      "status": "completed",
      "agent": "orchestrator",
      "preview": "[CODING: CodingPatchReport] coding: (no diff - patch
empty)\npatched_content_sha256=0694726448f07cd3a7ff347b3a829cfc9cbd45451ac04b6ba2a918c2437187b4",
      "tags": [
        "audit"
      ],
      "updated_at": "2026-10-06T14:25:58.131095",
      "created_at": "2026-10-06T14:25:56.318403",
      "message_count": 7,
      "token_count": 0,
      "cost_usd": 0.0,
      "user_id": null
    }
  ],
  "error": null,
  "request_id": "5898970480db41c5bef3af0395af373c",
  "meta": {
    "limit": 5,
    "next_cursor": null,
    "total": 1
  }
}
```

10.6.2 POST /v1/conversations

Endpoint	/v1/conversations
HTTP method	POST
Purpose	Create a conversation from a goal and start the background run.

Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. application/json 2. title: string — required: yes; min len 1; max len 200 3. agent: string — required: no; default "orchestrator" 4. tags: array<string> — required: no 5. user_id: string null — required: no
Response body	200 envelope ConversationEnriched + meta {plan_id, run_id}
Status codes	200, 422, 500
Error responses (observed)	422: "Request validation failed"

Example request / response (measured: HTTP 200 in 1731.4 ms):

POST /v1/conversations

```
{
  "title": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
  "agent": "orchestrator",
  "tags": [
    "audit"
  ]
}
-
{
  "ok": true,
  "data": {
    "id": "7e870c3d-bd34-4e74-b9f2-748e40c3fc91",
    "title": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
    "status": "active",
    "agent": "orchestrator",
    "preview": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
    "tags": [
      "audit"
    ],
    "updated_at": "2026-10-06T14:25:58.037657Z",
    "created_at": "2026-10-06T14:25:56.318403Z",
    "message_count": 1,
    "token_count": 0,
    "cost_usd": 0.0,
    "user_id": null
  },
  "error": null,
  "request_id": "51a131deac9e457c9977bb5aa1b3619b",
  "meta": {
    "plan_id": "f207bd06-d02e-5459-b9be-0a35a633adfb",
    "run_id": "bb087c9c-cc2e-49ad-9ee7-f345e3ad2f8f"
  }
}
```

10.6.3 GET /v1/conversations/{conversation_id}

Endpoint	/v1/conversations/{conversation_id}
HTTP method	GET
Purpose	Fetch one conversation with optional messages.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	1. conversation_id (path, string(uuid), required: yes) 2. include_messages (query, boolean, required: no, default true) 3. limit_messages (query, integer, required: no, ≥ 1; ≤ 500; default 50)
Request body	None
Response body	200 envelope ConversationEnriched + messages[]
Status codes	200, 404, 422
Error responses (observed)	404: "Conversation 00000000-0000-0000-0000-000000000000 not found"

Example request / response (measured: HTTP 200 in 6.4 ms):

```
GET /v1/conversations/7e870c3d-bd34-4e74-b9f2-748e40c3fc91?limit_messages=3
-
{
  "ok": true,
  "data": {
    "id": "7e870c3d-bd34-4e74-b9f2-748e40c3fc91",
    "title": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
    "status": "completed",
    "agent": "orchestrator",
    "preview": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
    "tags": [
      "audit"
    ],
    "updated_at": "2026-10-06T14:25:58.131095",
    "created_at": "2026-10-06T14:25:56.318403",
    "message_count": 3,
    "token_count": 0,
    "cost_usd": 0.0,
    "user_id": null,
    "messages": [
      {
        "id": "0b92183d-46c8-4290-bd09-5ab6c4b57e6c",
        "conversation_id": "7e870c3d-bd34-4e74-b9f2-748e40c3fc91",
        "role": "assistant",
        "content": "[REFLECTION:ReflectionReport] reflection: Reflected on answer of 482 chars, 0 tool call(s), 0 mistake(s) detected, confidence=0.90.",
        "name": null,
        "tool_call_id": null,
        "tool_calls": [],
        "prompt_tokens": 0,
        "completion_tokens": 0,
        "latency_ms": null,
        "created_at": "2026-10-06T14:25:58.118164",
        "metadata": {
          "step_id": "54319aca-f750-5e0b-8d85-8903b9b20ba7",
          ... (truncated)
        }
      }
    ]
  }
}
```

10.6.4 PATCH /v1/conversations/{conversation_id}

Endpoint	/v1/conversations/{conversation_id}
HTTP method	PATCH
Purpose	Update title, status or tags.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	conversation_id (path, string(uuid), required: yes)
Request body	1. application/json 2. title: string null — required: no; min len 1; max len 200 3. status: enum{active, completed, paused, failed} null — required: no 4. tags: array<string> null — required: no
Response body	200 envelope ConversationEnriched
Status codes	200, 404, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 13.7 ms):

```
PATCH /v1/conversations/7e870c3d-bd34-4e74-b9f2-748e40c3fc91
-
{
  "title": "Renamed audit thread",
  "tags": [
    "audit",
    "doc"
  ]
}
-
{
  "ok": true,
```

```

"data": {
  "id": "7e870c3d-bd34-4e74-b9f2-748e40c3fc91",
  "title": "Renamed audit thread",
  "status": "completed",
  "agent": "orchestrator",
  "preview": "[CODING:CodingPatchReport] coding: (no diff - patch
empty)\npatched_content_sha256=0694726448f07cd3a7ff347b3a829cfc9cbd45451ac04b6ba2a918c2437187b4",
  "tags": [
    "audit",
    "doc"
  ],
  "updated_at": "2026-10-06T14:25:58.574480Z",
  "created_at": "2026-10-06T14:25:56.318403",
  "message_count": 7,
  "token_count": 0,
  "cost_usd": 0.0,
  "user_id": null
},
"error": null,
"request_id": "fbdd41d4d7df46cfa0bbe78dbc364980",
"meta": {}
}

```

10.6.5 DELETE /v1/conversations/{conversation_id}

Endpoint	/v1/conversations/{conversation_id}
HTTP method	DELETE
Purpose	Delete a conversation (messages cascade).
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	conversation_id (path, string(uuid), required: yes)
Request body	None
Response body	200 envelope {deleted: true}
Status codes	200, 404, 422
Error responses (observed)	Standard envelope errors (see conventions)

10.6.6 GET /v1/conversations/{conversation_id}/messages

Endpoint	/v1/conversations/{conversation_id}/messages
HTTP method	GET
Purpose	Page through messages of a conversation.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	1. conversation_id (path, string(uuid), required: yes) 2. cursor (query, string null, required: no) 3. limit (query, integer, required: no, ≥ 1 ; ≤ 200 ; default 25)
Request body	None
Response body	200 envelope [Message] + meta
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 5.0 ms):

```

GET /v1/conversations/7e870c3d-bd34-4e74-b9f2-748e40c3fc91/messages?limit=2
-
{
  "ok": true,
  "data": [
    {
      "id": "0b92183d-46c8-4290-bd09-5ab6c4b57e6c",
      "conversation_id": "7e870c3d-bd34-4e74-b9f2-748e40c3fc91",
      "role": "assistant",

```

```

"content": "[REFLECTION:ReflectionReport] reflection: Reflected on answer of 482 chars, 0 tool
call(s), 0 mistake(s) detected, confidence=0.90.",
"name": null,
"tool_call_id": null,
"tool_calls": [],
"prompt_tokens": 0,
"completion_tokens": 0,
"latency_ms": null,
"created_at": "2026-10-06T14:25:58.118164",
"metadata": {
  "step_id": "54319aca-f750-5e0b-8d85-8903b9b20ba7",
  "run_id": "21dbcb3e-c379-4355-a144-0d9bc1609edc",
  "tokens_in": 0,
  "tokens_out": 0,
  "cost_usd": 0.0,
  "status": 3,
  "report_type": "ReflectionReport"
}
},
{
  "id": "10ef1f6d-78b4-4a35-9545-d8e997be77f1",
  "conversation_id": "7e870c3d-bd34-4e74-b9f2-748e40c3fc91",
  "role": "assistant",
  "content": "[CODING:CodingPatchReport] coding: (no diff - patch
empty)\npatched_content_sha256=0694726448f07cd3a7ff347b3a829cfc9cbd45451ac04b6ba2a918c2437187b4",
  "name": null,
  "tool_call_id": null,
  ... (truncated)

```

10.6.7 POST /v1/conversations/{conversation_id}/messages

Endpoint	/v1/conversations/{conversation_id}/messages
HTTP method	POST
Purpose	Append a message and update counters/status.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	conversation_id (path, string(uuid), required: yes)
Request body	1. application/json 2. role: enum{system, user, assistant, tool, observation, reflection} — required: yes 3. content: string — required: yes; min len 1 4. name: string null — required: no 5. tool_call_id: string null — required: no
Response body	200 envelope Message
Status codes	200, 404, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 14.4 ms):

POST /v1/conversations/7e870c3d-bd34-4e74-b9f2-748e40c3fc91/messages

```

{
  "role": "user",
  "content": "Please also document edge cases."
}
-
{
  "ok": true,
  "data": {
    "id": "547f36a7-ecb2-4424-9e38-fb1d1e6832ec",
    "conversation_id": "7e870c3d-bd34-4e74-b9f2-748e40c3fc91",
    "role": "user",
    "content": "Please also document edge cases.",
    "name": null,
    "tool_call_id": null,
    "tool_calls": [],
    "prompt_tokens": 0,
    "completion_tokens": 0,
    "latency_ms": null,
    "created_at": "2026-10-06T14:25:58.588185Z",
    "metadata": {}
  }
}

```

```

},
"error": null,
"request_id": "b00d4640531e4ea5984dfda96de39d45",
"meta": {}
}

```

10.7 Agents

10.7.1 GET /v1/agents

Endpoint	/v1/agents
HTTP method	GET
Purpose	List the built-in agent manifests.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	200 envelope [AgentManifest{name, agent_type, description, required_capabilities[], report_type}]
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 1.9 ms):

```

GET /v1/agents
-
{
  "ok": true,
  "data": [
    {
      "name": "planner",
      "agent_type": "planner",
      "description": "Decompose a goal into a dependency-ordered ExecutionPlan (seeded deterministic).",
      "required_capabilities": [
        "rag_search:*"
      ],
      "report_type": "PlannerReport"
    },
    {
      "name": "research",
      "agent_type": "research",
      "description": "Seed-URL-driven cross-source research report.",
      "required_capabilities": [
        "tool_invoke:web_fetch"
      ],
      "report_type": "ResearchReport"
    },
    {
      "name": "coding",
      "agent_type": "coding",
      "description": "Instruction→patch-list alpha CodingAgent with dry-run diff.",
      "required_capabilities": [
        "tool_invoke:files",
        "tool_invoke:shell"
      ],
      "report_type": "CodingPatchReport"
    },
    {
      "name": "reflection",
      "agent_type": "reflection",
      "description": "Mistake-detector + confidence scorer + replan recommender.",
      "required_capabilities": [
        "memory_read:*"
      ],
      "report_type": "ReflectionReport"
    },
    {
      "name": "memory",
      "agent_type": "memory",

```

... (truncated)

10.7.2 POST /v1/agents/invoke

Endpoint	/v1/agents/invoke
HTTP method	POST
Purpose	Run planner, research, coding or reflection on a state dict.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. application/json 2. agent: string — required: yes; min len 1 3. state: object — required: no
Response body	200 envelope output state {..., status, status_history[], report, report_type}
Status codes	200, 400, 422
Error responses (observed)	400: "Unknown agent 'judge'"

Example request / response (measured: HTTP 200 in 3.7 ms):

```
POST /v1/agents/invoke

{
  "agent": "planner",
  "state": {
    "goal": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
    "seed": 42
  }
}
-
{
  "__truncated__": true,
  ... (truncated)
}
```

10.8 Runs

10.8.1 POST /v1/runs/plan

Endpoint	/v1/runs/plan
HTTP method	POST
Purpose	Deterministically decompose a goal into an ExecutionPlan.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. application/json 2. goal: string — required: yes; min len 5 3. seed: integer null — required: no 4. conversation_id: string(uuid) null — required: no
Response body	200 envelope PlannerReport
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 1803.5 ms):

```
POST /v1/runs/plan

{
  "goal": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
  "seed": 42
}
```

```

}
-
{
  "__truncated__": true,
  ... (truncated)
}

```

10.8.2 POST /v1/runs/execute

Endpoint	/v1/runs/execute
HTTP method	POST
Purpose	Execute a plan step by step and return per-step results.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. application/json 2. plan: object — required: yes 3. conversation_id: string(uuid) null — required: no 4. user_id: string null — required: no
Response body	200 envelope {run_id, steps[...]}
Status codes	200, 422, 500
Error responses (observed)	500: "Internal data validation error"

Example request / response (measured: HTTP 200 in 1756.2 ms):

```

POST /v1/runs/execute

{
  "plan": {
    "id": "087b732a-c670-58bf-8a63-4745d0ec9dbf",
    "goal": "Write a Python function parse_csv.py that parses CSV files and add unit tests",
    "steps": [
      {
        "id": "948796c3-0b7b-5dda-a5cb-d6b48aea1b3b",
        "index": 0,
        "description": "Draft implementation for parse_csv",
        "assigned_agent": "coding",
        "dependencies": [],
        "tool_hints": [
          "files",
          "shell"
        ],
        "rag_query": null,
        "status": 3,
        "confidence": 0.95,
        "result": null,
        ... (truncated)
      }
    ]
  },
  ... (truncated)
}
-
{
  "__truncated__": true,
  ... (truncated)
}

```

10.8.3 GET /v1/runs/{run_id}/executions

Endpoint	/v1/runs/{run_id}/executions
HTTP method	GET
Purpose	List TaskExecution rows (filter uses conversation_id — see FR-016).
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	1. run_id (path, string(uuid), required: yes) 2. cursor (query, string null, required: no) 3. limit (query, integer, required: no, ≥ 1; ≤ 200; default 25)
Request body	None
Response body	200 envelope [TaskExecution] + meta

Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 7.9 ms):

```
GET /v1/runs/5ec84288-96aa-48f4-9263-ae836e52acf4/executions
-
{
  "ok": true,
  "data": [],
  "error": null,
  "request_id": "ff1b001ff8e644c8b8def5dc012684cb",
  "meta": {
    "limit": 25,
    "next_cursor": null,
    "total": 0
  }
}
```

10.9 Documents

10.9.1 GET /v1/documents

Endpoint	/v1/documents
HTTP method	GET
Purpose	List document records with status and search filters.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	1. <code>status</code> (query, string null, required: no) 2. <code>search</code> (query, string null, required: no) 3. <code>cursor</code> (query, string null, required: no) 4. <code>limit</code> (query, integer, required: no, ≥ 1; ≤ 200; default 25)
Request body	None
Response body	200 envelope [Document] + meta
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 5.8 ms):

```
GET /v1/documents
-
{
  "ok": true,
  "data": [
    {
      "id": "13a90f89-62ff-479b-9c7a-6d8821d9430b",
      "user_id": null,
      "title": "Design notes",
      "source_uri": "file:///notes.md",
      "mime_type": "text/markdown",
      "byte_size": 0,
      "chunk_count": 0,
      "ingestion_status": "pending",
      "metadata": {},
      "created_at": "2026-10-06T14:25:58.616882"
    }
  ],
  "error": null,
  "request_id": "37ae7fdef5b74c66818a5f1220e0ad16",
  "meta": {
    "limit": 25,
    "next_cursor": null,
    "total": 1
  }
}
```

10.9.2 POST /v1/documents

Endpoint	/v1/documents
HTTP method	POST
Purpose	Create document metadata (status pending).
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. application/json 2. title: string — required: yes; min len 1 3. source_uri: string — required: yes; min len 1 4. mime_type: string — required: no; default "application/octet-stream" 5. user_id: string null — required: no 6. metadata: object — required: no
Response body	200 envelope Document
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 7.3 ms):

```
POST /v1/documents

{
  "title": "Design notes",
  "source_uri": "file:///notes.md",
  "mime_type": "text/markdown"
}
-
{
  "ok": true,
  "data": {
    "id": "13a90f89-62ff-479b-9c7a-6d8821d9430b",
    "user_id": null,
    "title": "Design notes",
    "source_uri": "file:///notes.md",
    "mime_type": "text/markdown",
    "byte_size": 0,
    "chunk_count": 0,
    "ingestion_status": "pending",
    "metadata": {},
    "created_at": "2026-10-06T14:25:58.616882Z"
  },
  "error": null,
  "request_id": "e268ce6aa4cb47f1b5fa640d9de24e49",
  "meta": {}
}
```

10.9.3 GET /v1/documents/{document_id}

Endpoint	/v1/documents/{document_id}
HTTP method	GET
Purpose	Fetch one document record.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	document_id (path, string(uuid), required: yes)
Request body	None
Response body	200 envelope Document
Status codes	200, 404, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 3.6 ms):

```
GET /v1/documents/13a90f89-62ff-479b-9c7a-6d8821d9430b
-
{
  "ok": true,
  "data": {
    "id": "13a90f89-62ff-479b-9c7a-6d8821d9430b",
    "user_id": null,
    "title": "Design notes",
    "source_uri": "file:///notes.md",
    "mime_type": "text/markdown",
    "byte_size": 0,
    "chunk_count": 0,
    "ingestion_status": "pending",
    "metadata": {},
    "created_at": "2026-10-06T14:25:58.616882"
  },
  "error": null,
  "request_id": "b01f9e2be5644128859791cc6a21f72d",
  "meta": {}
}
```

10.9.4 DELETE /v1/documents/{document_id}

Endpoint	/v1/documents/{document_id}
HTTP method	DELETE
Purpose	Delete a document record.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	document_id (path, string(uuid), required: yes)
Request body	None
Response body	200 envelope {deleted: true}
Status codes	200, 404, 422
Error responses (observed)	404: "Document 13a90f89-62ff-479b-9c7a-6d8821d9430b not found"

Example request / response (measured: HTTP 200 in 9.4 ms):

```
DELETE /v1/documents/13a90f89-62ff-479b-9c7a-6d8821d9430b
-
{
  "ok": true,
  "data": {
    "deleted": true
  },
  "error": null,
  "request_id": "6849b4a95b2546649233696059ec7217",
  "meta": {}
}
```

10.9.5 POST /v1/documents/upload

Endpoint	/v1/documents/upload
HTTP method	POST
Purpose	Multipart upload; records size/type and a simulated chunk count.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. multipart/form-data 2. file: string — required: yes 3. user_id: string null — required: no 4. title: string null — required: no 5. source_uri: string null — required: no

Response body	200 envelope Document (ingestion_status ready)
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 13.9 ms):

```
POST /v1/documents/upload
-
{
  "ok": true,
  "data": {
    "id": "8b31bb09-0e91-4e77-8ee1-c29b6791913d",
    "user_id": null,
    "title": "Audit upload",
    "source_uri": "upload://81b3404fba504815bb01ed2e3eea8a67/audit.txt",
    "mime_type": "text/plain",
    "byte_size": 6300,
    "chunk_count": 3,
    "ingestion_status": "ready",
    "metadata": {
      "filename": "audit.txt",
      "uploaded_at": "2026-10-06T14:25:58.634165+00:00"
    },
    "created_at": "2026-10-06T14:25:58.634172Z"
  },
  "error": null,
  "request_id": "81b3404fba504815bb01ed2e3eea8a67",
  "meta": {}
}
```

10.10 Memory

10.10.1 GET /v1/memory

Endpoint	/v1/memory
HTTP method	GET
Purpose	List long-term memory rows.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	1. <code>memory_type</code> (query, string null, required: no) 2. <code>min_importance</code> (query, number, required: no, ≥ 0.0; ≤ 1.0; default 0.0) 3. <code>cursor</code> (query, string null, required: no) 4. <code>limit</code> (query, integer, required: no, ≥ 1; ≤ 200; default 25)
Request body	None
Response body	200 envelope [Memory] + meta
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 5.8 ms):

```
GET /v1/memory
-
{
  "ok": true,
  "data": [],
  "error": null,
  "request_id": "b11e5a3a5b944117871896c34d8e33d9",
  "meta": {
    "limit": 25,
    "next_cursor": null,
    "total": 0
  }
}
```

10.10.2 POST /v1/memory/query

Endpoint	/v1/memory/query
HTTP method	POST
Purpose	Rank memory rows by Jaccard token overlap + 0.1·importance.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. application/json 2. content: string — required: yes; min len 1 3. tiers: array<string> — required: no 4. top_k: integer — required: no; ≥ 1.0 ; ≤ 200.0 ; default 10 5. min_importance: number — required: no; ≥ 0.0 ; ≤ 1.0 ; default 0.0
Response body	200 envelope [Memory + score]
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 4.9 ms):

```
POST /v1/memory/query

{
  "content": "csv parsing tests",
  "top_k": 5
}
-
{
  "ok": true,
  "data": [],
  "error": null,
  "request_id": "57019ed3563f45929aa96bd33b62a43c",
  "meta": {}
}
```

10.10.3 POST /v1/memory/compress

Endpoint	/v1/memory/compress
HTTP method	POST
Purpose	Flag rows older than N days as compressed.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. application/json 2. older_than_days: integer — required: no; ≥ 1.0 ; ≤ 3650.0 ; default 30 3. tiers: array<string> — required: no
Response body	200 envelope {compressed_count, ids[]}
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 4.2 ms):

```
POST /v1/memory/compress

{
  "older_than_days": 30
}
-
{
  "ok": true,
  "data": {
    "compressed_count": 0,

```

```

    "ids": []
  },
  "error": null,
  "request_id": "ec26b4c34a90434f9080b0cf744a1eb0",
  "meta": {}
}

```

10.11 UAP

10.11.1 GET /v1/uap/transport

Endpoint	/v1/uap/transport
HTTP method	GET
Purpose	Static list of UAP transports.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	None
Response body	200 envelope [{id, scheme, endpoint, state}]
Status codes	200
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 1.5 ms):

```

GET /v1/uap/transport
-
{
  "ok": true,
  "data": [
    {
      "id": "inproc",
      "scheme": "memory",
      "endpoint": "astra.uap.inproc",
      "state": "available"
    },
    {
      "id": "http",
      "scheme": "https",
      "endpoint": "/v1/uap/envelope/send",
      "state": "available"
    }
  ],
  "error": null,
  "request_id": "4d5a8abeb48e4916b8ebf1345f0b7177",
  "meta": {}
}

```

10.11.2 POST /v1/uap/transport/inproc

Endpoint	/v1/uap/transport/inproc
HTTP method	POST
Purpose	Return an in-process binding descriptor and random token string.
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. application/json 2. actor_id: string null — required: no 3. capabilities: array<string> — required: no
Response body	200 envelope {transport_id, actor_id, token, capabilities[], send_url, created_at}
Status codes	200, 422

Error responses (observed)	Standard envelope errors (see conventions)
----------------------------	--

Example request / response (measured: HTTP 200 in 1.8 ms):

```
POST /v1/uap/ transports/inproc

{
  "actor_id": "audit-actor",
  "capabilities": [
    "send_message"
  ]
}
-
{
  "ok": true,
  "data": {
    "transport_id": "inproc",
    "actor_id": "audit-actor",
    "token": "uap.8268e5ddb77746ff93df5523d689aa68.1791296758",
    "capabilities": [
      "send_message"
    ],
    "send_url": "/v1/uap/envelope/send",
    "created_at": "2026-10-06T14:25:58.676987+00:00"
  },
  "error": null,
  "request_id": "547991d916e54173a85429b52470ba48",
  "meta": {}
}
```

10.11.3 POST /v1/uap/envelope/send

Endpoint	/v1/uap/envelope/send
HTTP method	POST
Purpose	Accept an envelope and return its payload hash (no delivery).
Authentication	None (open route)
Request headers	Accept: application/json; X-Request-ID (optional)
Parameters	None
Request body	1. application/json 2. id: string — required: no 3. from: string null — required: no 4. to: string null — required: no 5. kind: string — required: no; default "message" 6. payload: object — required: no
Response body	200 envelope {accepted, envelope_id, routed_to, payload_sha256, request_id, received_at}
Status codes	200, 422
Error responses (observed)	Standard envelope errors (see conventions)

Example request / response (measured: HTTP 200 in 2.1 ms):

```
POST /v1/uap/envelope/send

{
  "from": "a",
  "to": "b",
  "kind": "message",
  "payload": {
    "x": 1
  }
}
-
{
  "ok": true,
  "data": {
    "accepted": true,
    "envelope_id": "182ce7fd54f144f08e96d55db0c6504b",
    "routed_to": "b",
  },
  "error": null,
  "request_id": "547991d916e54173a85429b52470ba48",
  "meta": {}
}
```

```

"payload_sha256": "9173335d6fdd7444c3006dbfd621acb01b9fcc420660321ccdd2009e912c7f10",
"request_id": "e95a1d28989f47b5840bb8e1a5a8f94c",
"received_at": "2026-10-06T14:25:58.679144+00:00"
},
"error": null,
"request_id": "e95a1d28989f47b5840bb8e1a5a8f94c",
"meta": {}
}

```

10.12 Benchmarking (capability-gated)

10.12.1 GET /llm/benchmark

Endpoint	/llm/benchmark
HTTP method	GET
Purpose	Measure LLM latency and tokens/s against Ollama (mock fallback).
Authentication	X-Noesis-Capability-Token with capability llm.benchmark.read
Request headers	Accept: application/json; X-Noesis-Capability-Token: <b64payload>.<b64mac>; X-Request-ID (optional)
Parameters	- prompt_tokens (query, integer, required: no, ≥ 8; ≤ 32768; default 256) - model (query, string null, required: no) - X-Noesis-Capability-Token (header, string null, required: no)
Request body	None
Response body	200 envelope BenchmarkResult
Status codes	200, 403, 422
Error responses (observed)	403: "C1_DENY_MISSING_CAP: missing X-Noesis-Capability-Token header (MAC-gated endpoint requires a signed capability token)" 403: "C1_DENY_MAC: malformed capability token (expected b64payload.b64mac)" 403: "C1_DENY_MAC: capability token HMAC signature verification failed" 403: "C1_DENY_EXPIRED: capability token expired at unix_ts=1791296748"

Example request / response (measured: HTTP 200 in 2224.5 ms):

```

GET /llm/benchmark?prompt_tokens=64
X-Noesis-Capability-Token: eyJjYXBhYm1saXRpZXMiOiBb...
-
{
  "ok": true,
  "data": {
    "provider": "ollama",
    "model": "gpt-4o-mini",
    "reachable": false,
    "used_mock": true,
    "prompt_tokens": 64,
    "completion_tokens": 32,
    "total_tokens": 96,
    "latency_ms": 19.9,
    "first_token_latency_ms": 6.96,
    "prompt_tokens_per_sec": 3216.08,
    "completion_tokens_per_sec": 1608.04,
    "fallback_provider": "ollama",
    "error": null
  },
  "error": null,
  "request_id": "cba7bff44e05479e9341b152883d197d",
  "meta": {}
}

```

10.12.2 GET /api/bench/results

Endpoint	/api/bench/results
HTTP method	GET
Purpose	Serve the newest benchmark tables (demo payload if none).

Authentication	X-Noesis-Capability-Token with capability <code>bench.results.read</code>
Request headers	Accept: application/json; X-Noesis-Capability-Token: <b64payload>.<b64mac>; X-Request-ID (optional)
Parameters	1. <code>refresh</code> (query, boolean, required: no, default false) 2. <code>X-Noesis-Capability-Token</code> (header, string null, required: no)
Request body	None
Response body	200 envelope BenchmarkResults + meta {demo, source, generated_at}
Status codes	200, 403
Error responses (observed)	403: "C1_DENY_MISSING_CAP: missing X-Noesis-Capability-Token header (MAC-gated endpoint requires a signed capability token)"

Example request / response (measured: HTTP 200 in 8.9 ms):

```
GET /api/bench/results
X-Noesis-Capability-Token: eyJjYXBhYmlsaXRpZXMiOiBb...
-
{
  "__truncated__": true,
  ... (truncated)
```

11 Database Design

The relational schema is defined with SQLAlchemy 2 declarative models in `backend/noesis/database/models.py`. In development `Base.metadata.create_all()` creates it at start-up; Alembic revision `0001_milestone0_schema` creates the six original tables and `0002_milestone5_mvp` is an intentional no-op stamp. Start-up code additionally adds `metadata_json` columns and the `memory_tiers_promotions` table with SQLite-specific `PRAGMA/sqlite_master` statements.

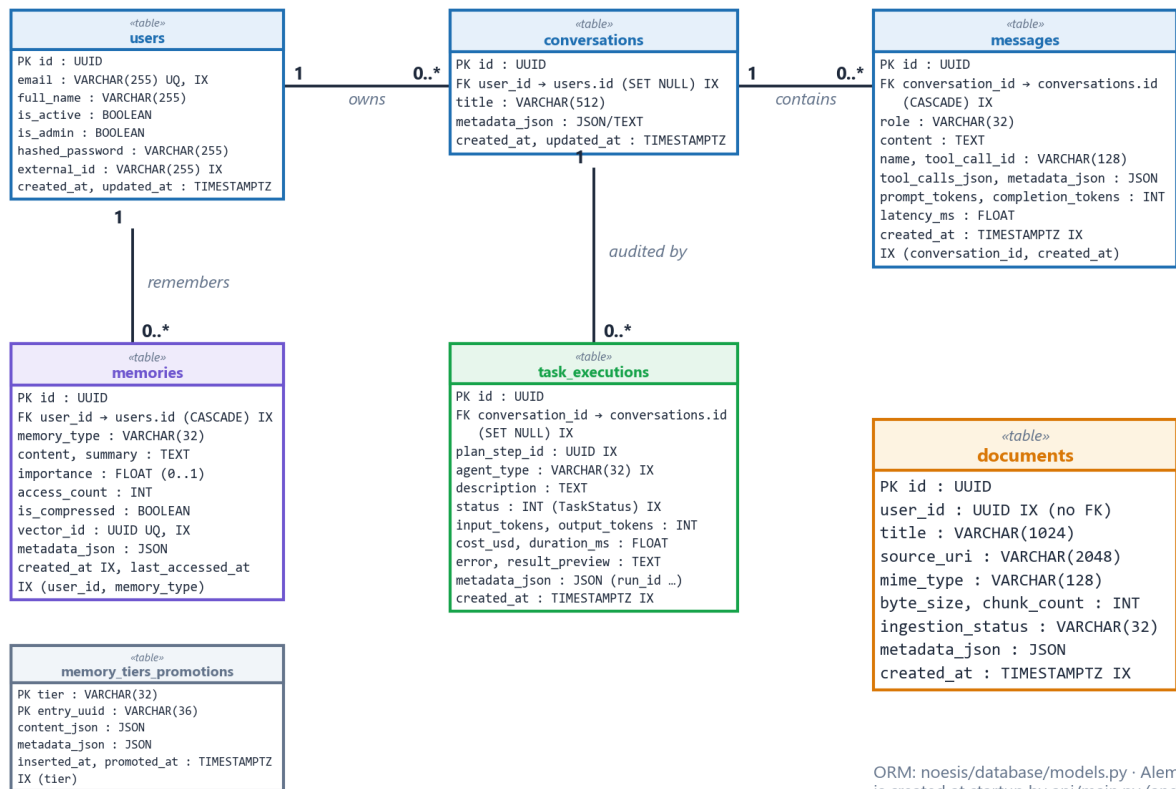


Figure 11.1: Entity–relationship diagram of the relational schema (as defined in the ORM)

Users own conversations and memories; conversations contain messages and are referenced by task executions. Documents carry a user_id without a foreign key. memory_tiers_promotions is standalone with a composite primary key (tier, entry_uuid).

Table 11.1: Table documents

Column	Type	Null	Keys / constraints	Default
id	UUID	NOT NULL	PK	now()
user_id	UUID	NULL	INDEX	
title	VARCHAR(1024)	NOT NULL		
source_uri	VARCHAR(2048)	NOT NULL		
mime_type	VARCHAR(128)	NOT NULL		'application/octet-stream'
byte_size	INTEGER	NOT NULL		0
chunk_count	INTEGER	NOT NULL		0
ingestion_status	VARCHAR(32)	NOT NULL		'pending'
metadata_json	JSONB	NULL		
created_at	DATETIME	NOT NULL	INDEX	now()

Indexes and constraints: ix_documents_created_at (created_at); ix_documents_user_id (user_id).

Table 11.2: Table memory_tiers_promotions

Column	Type	Null	Keys / constraints	Default
tier	VARCHAR(32)	NOT NULL	PK	
entry_uuid	VARCHAR(36)	NOT NULL	PK	
content_json	JSONB	NOT NULL		
metadata_json	JSONB	NULL		
inserted_at	DATETIME	NOT NULL		now()
promoted_at	DATETIME	NULL		

Indexes and constraints: ix_memory_tiers_tier (tier).

Table 11.3: Table users

Column	Type	Null	Keys / constraints	Default
id	UUID	NOT NULL	PK	now()
email	VARCHAR(255)	NOT NULL	UNIQUE, INDEX	
full_name	VARCHAR(255)	NULL		
is_active	BOOLEAN	NOT NULL		True
is_admin	BOOLEAN	NOT NULL		False
hashed_password	VARCHAR(255)	NULL		
external_id	VARCHAR(255)	NULL	INDEX	
created_at	DATETIME	NOT NULL		now()
updated_at	DATETIME	NOT NULL		now()

Indexes and constraints: ix_users_external_id (external_id); ix_users_email (email) UNIQUE.

Table 11.4: Table conversations

Column	Type	Null	Keys / constraints	Default
id	UUID	NOT NULL	PK	now()
user_id	UUID	NULL	FK → users.id ON DELETE SET NULL, INDEX	
title	VARCHAR(512)	NOT NULL		'Untitled conversation'
metadata_json	JSONB	NULL		
created_at	DATETIME	NOT NULL		now()
updated_at	DATETIME	NOT NULL		now()

Indexes and constraints: ix_conversations_user_id (user_id).

Table 11.5: Table memories

Column	Type	Null	Keys / constraints	Default
id	UUID	NOT NULL	PK	now()
user_id	UUID	NULL	FK → users.id ON DELETE CASCADE, INDEX	
memory_type	VARCHAR(32)	NOT NULL		

Column	Type	Null	Keys / constraints	Default
content	TEXT	NOT NULL		
summary	TEXT	NULL		
importance	FLOAT	NOT NULL		0.5
access_count	INTEGER	NOT NULL		0
is_compressed	BOOLEAN	NOT NULL		False
vector_id	UUID	NOT NULL	INDEX	
metadata_json	JSONB	NULL		
created_at	DATETIME	NOT NULL	INDEX	now()
last_accessed_at	DATETIME	NOT NULL		now()

Indexes and constraints: `ix_memories_user_id` (`user_id`); `ix_memories_user_type` (`user_id`, `memory_type`); `ix_memories_created_at` (`created_at`); `ix_memories_vector_id` (`vector_id`); `uq_memories_vector_id` UNIQUE (`vector_id`).

Table 11.6: Table messages

Column	Type	Null	Keys / constraints	Default
id	UUID	NOT NULL	PK	now()
conversation_id	UUID	NOT NULL	FK → conversations.id ON DELETE CASCADE, INDEX	
role	VARCHAR(32)	NOT NULL		
content	TEXT	NULL		
name	VARCHAR(128)	NULL		
tool_call_id	VARCHAR(128)	NULL		
tool_calls_json	JSONB	NULL		
prompt_tokens	INTEGER	NOT NULL		0
completion_tokens	INTEGER	NOT NULL		0
latency_ms	FLOAT	NULL		
metadata_json	JSONB	NULL		
created_at	DATETIME	NOT NULL	INDEX	now()

Indexes and constraints: `ix_messages_created_at` (`created_at`); `ix_messages_convo_created` (`conversation_id`, `created_at`); `ix_messages_conversation_id` (`conversation_id`).

Table 11.7: Table task_executions

Column	Type	Null	Keys / constraints	Default
id	UUID	NOT NULL	PK	now()
conversation_id	UUID	NULL	FK → conversations.id ON DELETE SET NULL, INDEX	
plan_step_id	UUID	NULL	INDEX	
agent_type	VARCHAR(32)	NOT NULL		
description	TEXT	NOT NULL		
status	INTEGER	NOT NULL		0

Column	Type	Null	Keys / constraints	Default
input_tokens	INTEGER	NOT NULL		0
output_tokens	INTEGER	NOT NULL		0
cost_usd	FLOAT	NOT NULL		0.0
duration_ms	FLOAT	NOT NULL		0.0
error	TEXT	NULL		
result_preview	TEXT	NULL		
metadata_json	JSONB	NULL		
created_at	DATETIME	NOT NULL	INDEX	now()

Indexes and constraints: `ix_task_executions_created_at` (`created_at`);
`ix_task_executions_conversation_id` (`conversation_id`); `ix_tasks_status` (`status`);
`ix_task_executions_plan_step_id` (`plan_step_id`); `ix_tasks_agent` (`agent_type`).

11.1 Relationships and referential actions

Table 11.8: Relationships

Relationship	Cardinality	On delete
users → conversations	1 : 0..*	SET NULL (FK); ORM cascade delete-orphan
users → memories	1 : 0..*	CASCADE
conversations → messages	1 : 0..*	CASCADE
conversations → task_executions	1 : 0..*	SET NULL

11.2 Transactions and pagination

- Repositories open a `session_scope()` per operation: commit on success, rollback on exception, always close.
- List queries use keyset pagination ordered by primary key (`id > cursor`) and return `next_cursor` when more rows exist.
- JSON columns use PostgreSQL JSONB with a String variant on SQLite; DTO ↔ ORM mappers serialise metadata and tool calls.
- The `run_id` of a task execution is stored only inside `metadata_json`; there is no column or index for it (root cause of FR-016).

12 AI/ML Specification

Scope: NOESIS does **not** train, fine-tune or host machine-learning models, and contains no dataset preparation, feature engineering or training loop. Its AI behaviour comes from (i) deterministic, rule-based agents and (ii) optional calls to pre-trained LLMs served by external runtimes. Confusion matrices, accuracy/loss curves or ROC curves therefore do not apply and are not presented.

12.1 Models used

Table 12.1: Language models referenced by the code

Model	Role	Where configured	Verified
qwen2.5-coder:7b-instruct-q4_K_M (Ollama)	Default agent-enrichment model (AGENTS_OLLAMA_MODEL) and benchmark model	config.py; factory.get_default_local_ollama	Yes — live calls measured
llama3.1:8b (Ollama)	Default OLLAMA_MODEL; router profile card	config.py; kernel default models	No
gpt-4o-mini / gpt-4o	DEFAULT_MODEL / router profile cards	config.py; kernel	No (no API key)
claude-3-5-sonnet-latest, claude-3-opus-latest	Router profile cards	kernel	No
gemini-1.5-pro	Router profile card	kernel	No
meta-llama/llama-3.1-405b-instruct (OpenRouter)	Router profile card	kernel	No
text-embedding-3-small (1536-d)	Default embedding model	config.py	No (embeddings not used by any route)

12.2 Inference pipeline

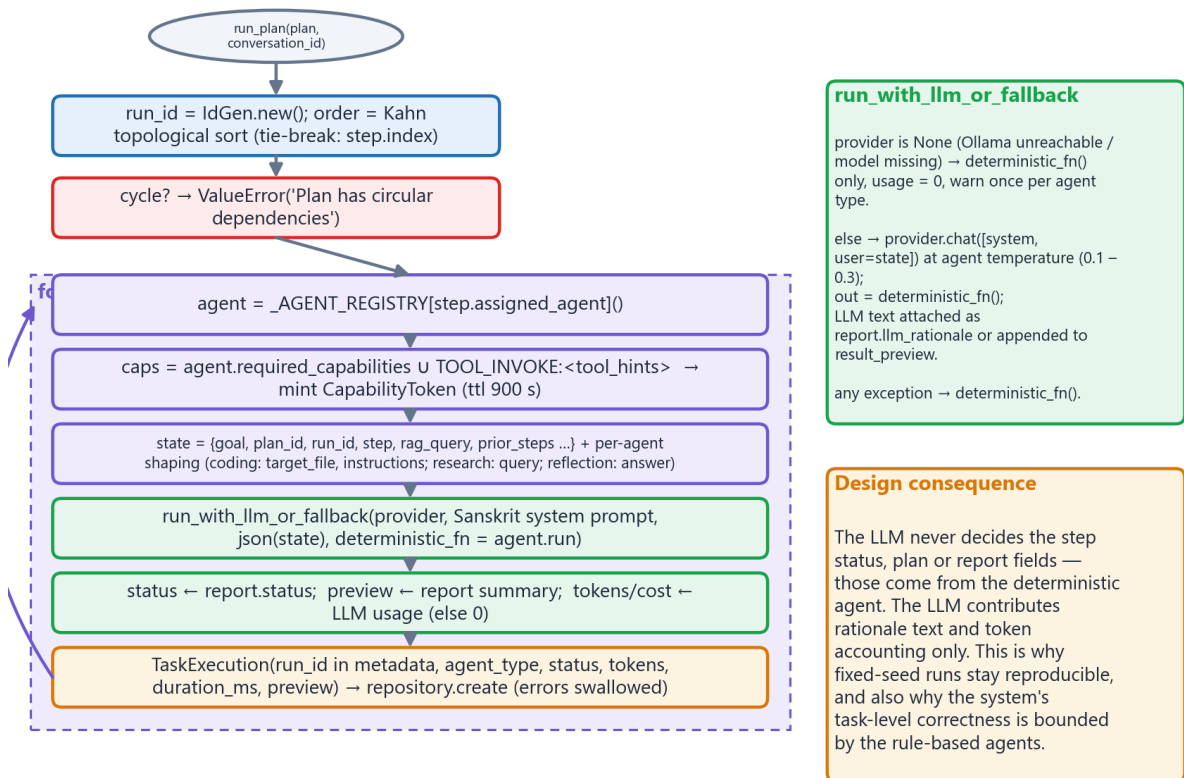


Figure 12.1: Agent runtime: per-step execution with LLM enrichment and deterministic fallback

For each step the runtime first computes the deterministic agent output. When a provider is available it also sends the agent's system prompt and the serialised step state ($\leq 6\,000$ characters) to the model at the agent's temperature (Planner/Coding/Judge 0.1, Critic 0.3,

others 0.2) and attaches the reply. The model never changes the step status or the structured report.

Table 12.2: Per-agent generation settings (services/kernel_services.py)

Agent	Temperature	Sanskrit codename (system prompt)
planner	0.1	Manan
research	0.2	Anveshak
coding	0.1	Vidya
reflection	0.2	Darshak
memory	0.2	Paalak
rag	0.2	Vivechak-II
tool	0.2	Karmakarta
judge	0.1	Parikshak
critic	0.3	Vivechak
executor	0.2	Kriyakārī
supervisor	0.2	Nirikshak
orchestrator	0.2	Samanyaka

12.3 Evaluation datasets and harness

Table 12.3: Evaluation datasets and harness semantics

Benchmark	Content	What the harness measures	Status of data
Noesis-SE50	50 project-authored software-engineering goals in 5 categories × 3 difficulties (corpus.csv/json)	Executor SIGNOFF of the seeded 12-phase pipeline (pass_fail_heuristic)	Full corpus in repo
HumanEval [23]	164 Python problems (downloaded at run time)	Fraction of generated samples that pass <code>compile()</code> — no unit tests executed	Oct-06 run used a 5-task bundled stub
MBPP [24]	Basic Python problems (sanitised first-500 stub bundled)	Fraction of samples that pass <code>compile()</code>	Oct-06 run used 5 tasks
Self-host 10 PRs	10 change requests on the NOESIS repo (selfhost_10prs.json)	Report CSVs (smoke)	Smoke artefacts only

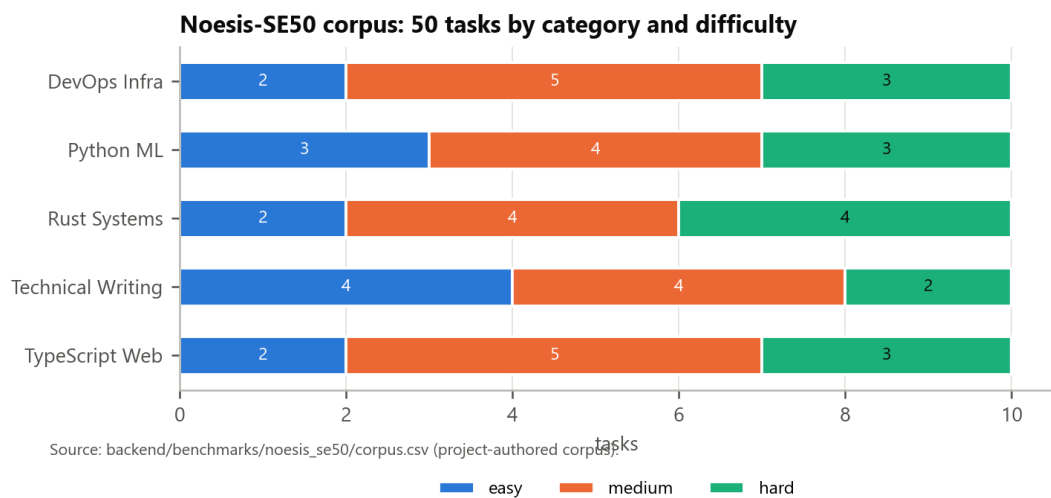


Figure 12.2: Composition of the Noesis-SE50 corpus

12.4 Evaluation metrics and statistics

Table 12.4: Metrics and statistical tests

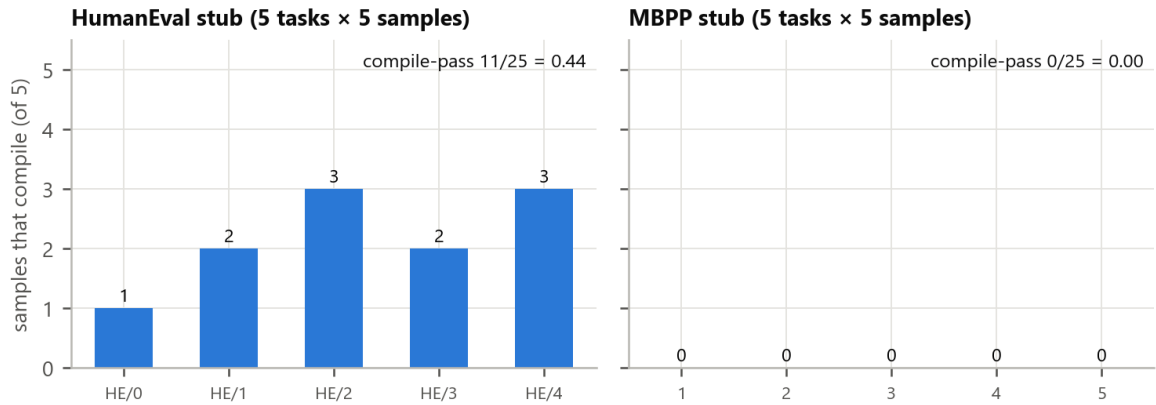
Metric / test	Definition	Implementation
pass@k	Unbiased estimator $1 - C(n-c, k)/C(n, k)$ [23]	benchmarks/stats.py: pass_at_k
Bootstrap CI	Percentile bootstrap, 10 000 resamples, seed 42 [32] (not BCa, despite README wording)	_bootstrap_pass1
Wilson interval	Score interval for a proportion [34]	_wilson_ci
McNemar	χ^2 with continuity correction; mid-p halving when a cell is 0; warns when discordant pairs < 25 [33]	mcnemar_pvalue
Wilcoxon signed-rank	Normal approximation ($N \geq 20$)	wilcoxon_signed_rank_pvalue_approx
Determinism score	Identical SHA-256 pairs within (goal, seed) groups / total pairs	scripts/determinism_manifest.py

12.5 Measured and reported results

The following results are either measured during this audit or reported from committed artefacts. They characterise the pipeline, not the general code-generation ability of the system.

Table 12.5: AI-related results

Result	Value	Type
Planner determinism, SE-50 (50 goals $\times$ 3 runs)	50/50 goals with one SHA-256	Measured
Planner seed sensitivity (5 seeds)	5 distinct plan hashes	Measured
SE-50 pipeline sign-off, Oct-06 run (5 tasks $\times$ 3 runs)	15/15 SIGNOFF (pass_at_1 = 1.0)	Reported — acceptance criteria are synthesised as met by the harness, so this is a pipeline-integrity check
SE-50 determinism manifest (50 goals $\times$ 3 runs)	50/50 groups identical; 450/450 ordered pairs incl. self-pairs; 141/150 runs status FAILED	Reported (docs/eval/se50_determinism_3runs.csv)
HumanEval stub compile-pass (5 tasks $\times$ 5 samples, T 0.7)	11/25 = 0.44 (unbiased compile-pass@1)	Reported, recomputed
MBPP stub compile-pass (5 $\times$ 5)	0/25 = 0.00	Reported
Local LLM throughput (qwen2.5-coder 7B)	≈ 23 – 24 completion tokens/s	Measured
End-to-end 5-step run with LLM enrichment	60.4 s, 2 594 tokens in background run	Measured

Syntax-compile pass per task — W7 run of 2026-10-06 (qwen2.5-coder 7B, T = 0.7)

Source: docs/eval/w7_weekend_real_Oct06_real. The harness checks `compile()` only — it does not execute unit tests, so these are NOT functional pass@k scores.

Figure 12.3: Syntax-compile pass per task in the W7 run of 6 October 2026

HumanEval stub samples compiled in 11 of 25 cases; MBPP stub samples in none. Because only `compile()` is checked, these numbers say nothing about functional correctness. The README's comparative pass@1 figures for GPT-4o, Claude 3.5 Sonnet and Gemini 1.5 Pro have no supporting result files in the repository and are not reproduced here (Appendix F).

12.6 Model limitations

- Agent decisions are keyword- and length-based heuristics; the LLM's text is advisory and never parsed into actions.
- The RAG agent returns synthetic chunks; no embeddings or vector search are used at run time.
- Local 7B inference is slow (tens of seconds per step); prompts above $\approx 1\,000$ tokens hit the 120 s timeout.
- Code benchmarks do not execute tests; SE-50 sign-off is pre-determined by synthesised criteria.
- Prompt-injection resistance is not evaluated; LLM output is stored verbatim in messages.

13 Algorithms

This chapter specifies the algorithms implemented in the code. Complexities use n = number of plan steps, d = dependency edges, t = query tokens, F = fragments, E = memory entries.

13.1 A1 Deterministic goal decomposition (PlannerAgent)

Algorithm	A1 — Deterministic goal decomposition (PlannerAgent)
Purpose	Turn a goal into a reproducible dependency DAG.
Inputs	goal (str ≥ 5), seed (int, default 1337)
Outputs	PlannerReport with ExecutionPlan
Complexity	$O(n^2)$ for dependency id construction ($n \leq 13$) plus $O(\text{goal})$ tokenisation
Implementation	agents/core.py PlannerAgent.run, _classify, _rng (mulberry32)

```

tokens ← lower(regex \w{3,} over goal)
rng ← mulberry32(seed)
seed_str ← f"{seed}:{len(goal)}:{hex(sha256(goal)[:8])}"
n ← min(12, max(3, 2 + |tokens| // 6))
for i in 0..n-1:
  agent ← first keyword-group match in goal, else round_robin[(i + [97·rng()]) mod 3]
  topic ← tokens[(7i + 3) mod |tokens|]
  deps ← last max(1, min(i, 1 + [2·rng()])) earlier step ids (none for i = 0)
  id ← uuid5(NS, 'step:' + seed_str + ':' + i)
  append REFLECTION step depending on all previous steps
plan.id ← uuid5(NS, 'plan:' + seed_str + ':' + goal); plan.created_at ← 2026-01-01T00:00Z
memory_agent.on_plan(plan); for each step on_step_completed(step); run_all_promotions(3)

```

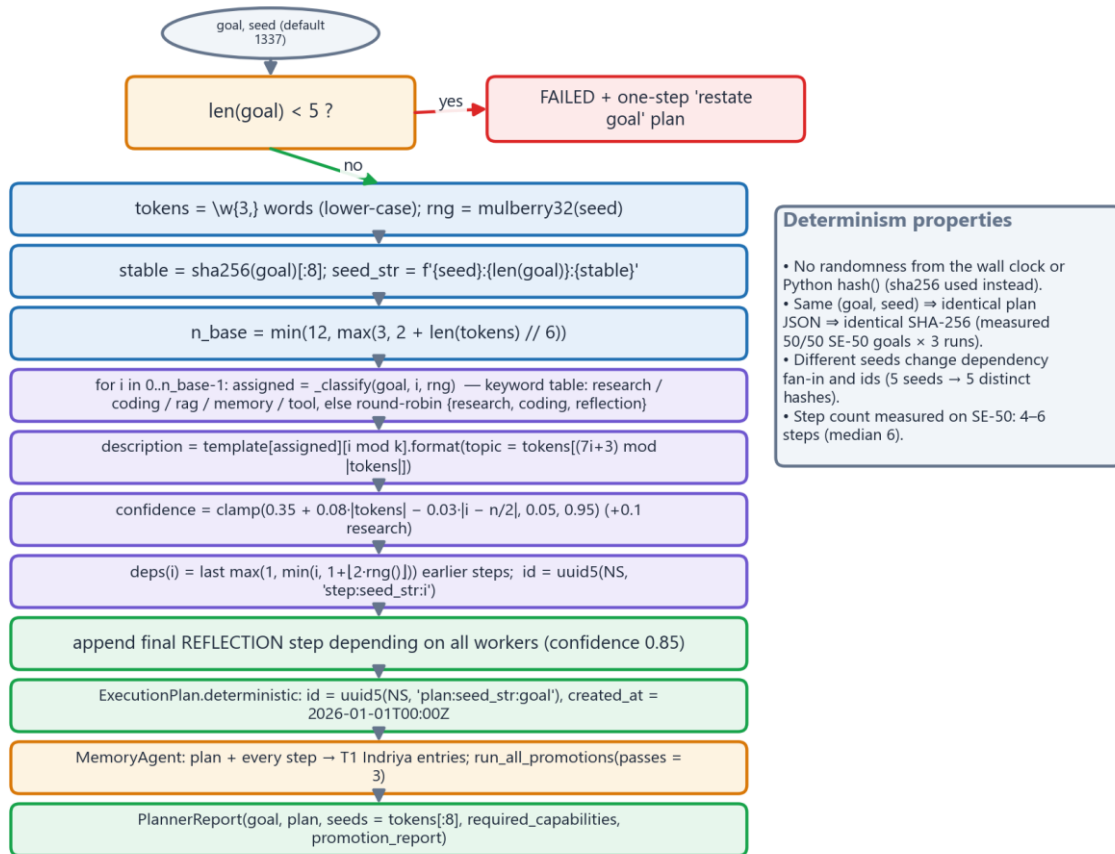


Figure 13.1: Deterministic goal decomposition (PlannerAgent)

13.2 A2 Topological execution with scoped tokens (AgentSvc)

Algorithm	A2 — Topological execution with scoped tokens (AgentSvc)
Purpose	Execute steps respecting dependencies.
Inputs	ExecutionPlan

Outputs	run_id, StepRunResult list
Complexity	$O(n^2 + d)$ (Kahn with re-sorting of a small ready list)
Implementation	services/kernel_services.py AgentSvc._topological_order, run_step, run_plan

```

indeg[s] ← |deps(s) n steps|
ready ← steps with indeg 0 sorted by index
while ready: s ← pop_front(ready); order.append(s)
    for o in steps (by index) with s ∈ deps(o): indeg[o]--; if 0: insert o; sort ready by index
if |order| < |steps|: raise ValueError('circular dependencies')
for s in order:
    caps ← agent(s).required_capabilities ∪ {TOOL_INVOKE:h | h ∈ s.tool_hints}
    token ← mint(caps, ttl 900 s)
    out, usage ← run_with_llm_or_fallback(provider, prompt(agent), json(state), agent.run)
    persist TaskExecution(out, usage)

```

13.3 A3 Six-tier promotion with hash chain (PromotionController)

Algorithm	A3 — Six-tier promotion with hash chain (PromotionController)
Purpose	Move memory entries up tiers on evidence and log every move.
Inputs	Tier contents, plan refs, corpus, clock, review marks
Outputs	PromotionReport
Complexity	$O(E \log E)$ per pass (sorted snapshots); fingerprint $O(\text{content})$
Implementation	memory/promotion.py

```

for edge in [T1→T2, T2→T3, T3→T4, T4→T5, T5→T6]:          # bottom-up, single pass
    for e in sorted(tier(edge.from), key = uuid hex):
        if trigger(edge, e): migrate(e)
migrate(e): state ← sha256(canonical_json(e, from))
    ev ← sha256(from→to | id | trigger | state | tail)
    tail ← sha256(tail | ev); e.metadata += {prev_sha, event_sha, promoted_from}
triggers: T1→T2 access_count ≥ 3 ∧ last access within 60 s; T2→T3 overlap(fp(e), corpus) ≥ 0.8;
          T3→T4 ≥ 2 distinct plan refs; T4→T5 age ≥ 7 d ∧ cross-goal ref; T5→T6 human review
(default)

```

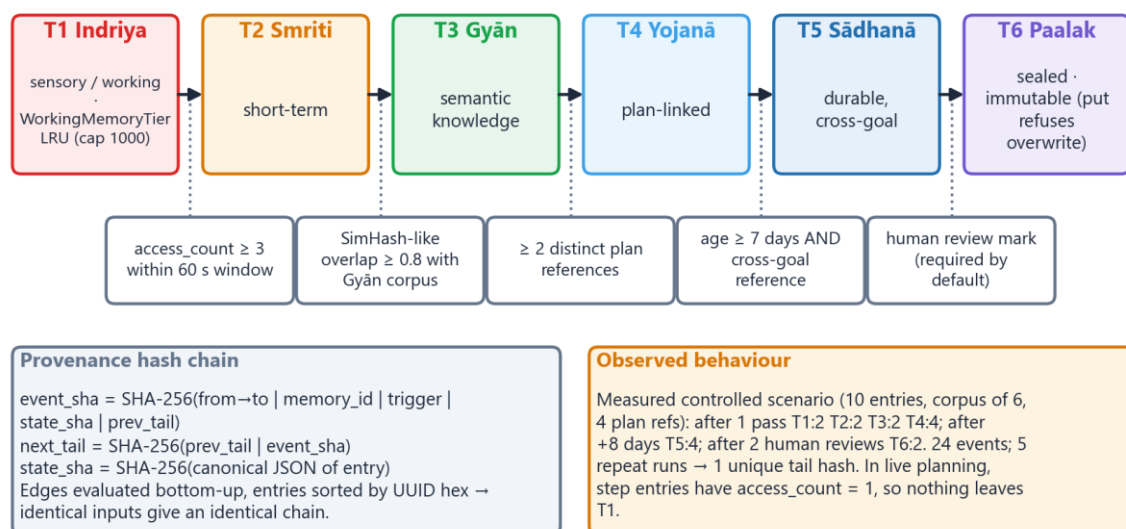


Figure 13.2: Six-tier promotion with hash chain (PromotionController)

13.4 A4 Tri-state acceptance decision (ExecutorAgent)

Algorithm	A4 — Tri-state acceptance decision (ExecutorAgent)
Purpose	Decide sign-off, replan or reject.
Inputs	plan (optional), accept_criteria (optional)
Outputs	ExecutorTriStateDecision
Complexity	$O(n + \text{criteria})$
Implementation	agents/core.py ExecutorAgent.evaluate_decision

```

score ← 1 -  $\sum_{\text{unmet}} w(\text{sev}) / \sum_{\text{all}} w(\text{sev})$ ; w = {low .05, medium .10, high .20, critical .35}
penalty ← min(.30, .08·failed) + min(.10, .03·skipped) + min(.20, .05·pending) + min(.12,
.04·low_conf)
conf ← clamp(score - penalty, 0, 1)
return SIGNOFF if conf ≥ .90 else REPLAN if conf ≥ .50 else REJECT

```

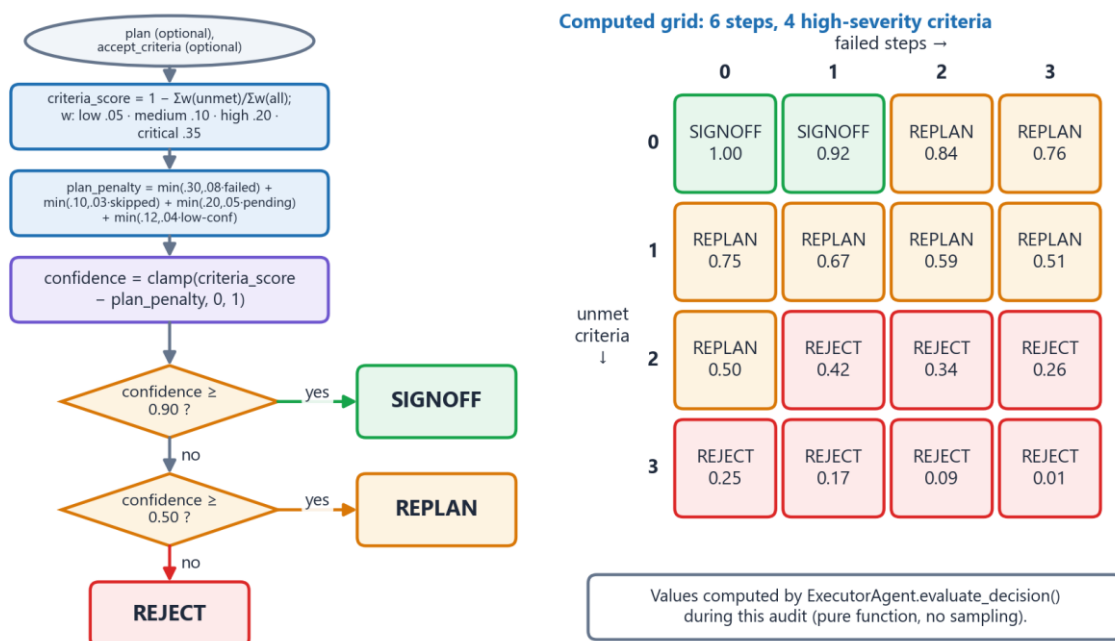


Figure 13.3: Tri-state acceptance decision (ExecutorAgent)

13.5 A5 Heuristic review agents (Reflection, Judge, Critic)

Algorithm	A5 — Heuristic review agents (Reflection, Judge, Critic)
Purpose	Score answers without an LLM.
Inputs	answer / candidates / draft
Outputs	Reports with scores
Complexity	$O(\text{text})$
Implementation	agents/core.py

```

Reflection: mistakes += missing_citations (≥2 sentences, no citations) | tool_failures (exit ∉ {0, -1}) | contradiction ('is X' and 'is not X')
           confidence = max(0, 0.9 -  $\sum \{ \text{high .20, medium .08, low .02} \})$ 
Judge: score = min(.99, .4 + .5·unique_tokens/max(6, tokens) + .1·min(5, count('[') + count('src')))
Critic: five axes (structure, clarity, citation_support, completeness, grammar) from text statistics; overall = mean

```

13.6 A6 Capability-token verification (HTTP gate)

Algorithm	A6 — Capability-token verification (HTTP gate)
Purpose	Authorise gated routes.
Inputs	Header token, required capability
Outputs	Payload or 403
Complexity	$O(\text{token} + \text{caps} + \text{masks})$
Implementation	api/middleware/capability_gate.py

```

payload_b64, mac_b64 ← split(token, '.')
if not compare_digest(b64d(mac_b64), HMAC_SHA256(secret, b64d(payload_b64))): deny C1_DENY_MAC
p ← CapabilityTokenPayload(json(b64d(payload_b64)))
if p.expires_at_unix_s < now: deny C1_DENY_EXPIRED
if required ∉ p.capabilities: deny C1_DENY_MISSING_CAP
if ∃ m ∈ p.deny_masks: fnmatch(required, m): deny C1_DENY_MASK

```

13.7 A7 Weighted model routing (ModelRouter)

Algorithm	A7 — Weighted model routing (ModelRouter)
Purpose	Choose a model card for a call.
Inputs	RoutingConstraint, token estimates
Outputs	Primary + 2 fallbacks
Complexity	$O(P \log P)$, $P = 7$ profiles
Implementation	kernel/model_router.py

```

C ← {p | matches(p, constraint)}; if C = ∅: return default (gpt-4o-mini)
cost ← in/1000·c_in + out/1000·c_out
score(p) ← .30q + .15r + .15tool + .05code + .05vision + .05mm + .15(1 - min(1, cost)) + .10(1 - clip((p95 - 200)/29 800))
return argsort(score) desc → primary, next two

```



Profiles are static cards in Kernel.register_default_models (quality/latency/cost values are configuration, not measurements). The router is reachable through the MODEL_ROUTE syscall and /v1/kernel/stats; the HTTP agent runtime does not use it to pick a model.

Figure 13.4: Weighted model routing (ModelRouter)

13.8 A8 Priority/deadline scheduling (Scheduler)

Algorithm	A8 — Priority/deadline scheduling (Scheduler)
Purpose	Run kernel tasks fairly.
Inputs	Coroutine functions with priority and deadline
Outputs	Task states, interrupts

Complexity	$O(\log Q)$ per push/pop
Implementation	kernel/scheduler.py

```

heap key ← (-priority, -deadline_μs, seq)
loop: pop; skip cancelled/done; if deadline missed: emit DEADLINE_MISSED
    await wait_for(task, 0.5 s): DONE | timeout → TIMESLICE + re-push | error → FAILED +
AGENT_CRASHED

```

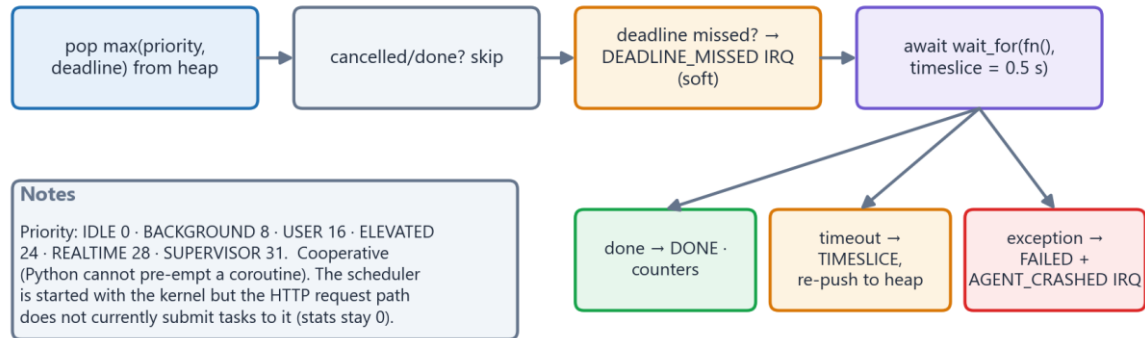


Figure 13.5: Priority/deadline scheduling (Scheduler)

13.9 A9 BM25 + Reciprocal Rank Fusion retrieval

Algorithm	A9 — BM25 + Reciprocal Rank Fusion retrieval
Purpose	Rank text fragments for a query.
Inputs	query, top_k
Outputs	SearchResult list
Complexity	$O(\sum_t \text{posting}(t)) + O(R \log R)$
Implementation	rag/pipeline.py

```

idf(t) ← ln((F - n_t + 0.5)/(n_t + 0.5) + 1) [20]
bm25(f) ←  $\sum_t \text{idf}(t) \cdot \text{tf}(t, f) (k_1 + 1) / (\text{tf} + k_1 (1 - b + b \cdot |f| / \text{avg}|f|))$ ,  $k_1 = 1.5$ ,  $b = 0.75$ 
rrf(f) ←  $\sum_{\text{rankers}} 1 / (60 + \text{rank}_r(f))$  [21]; normalise by max

```

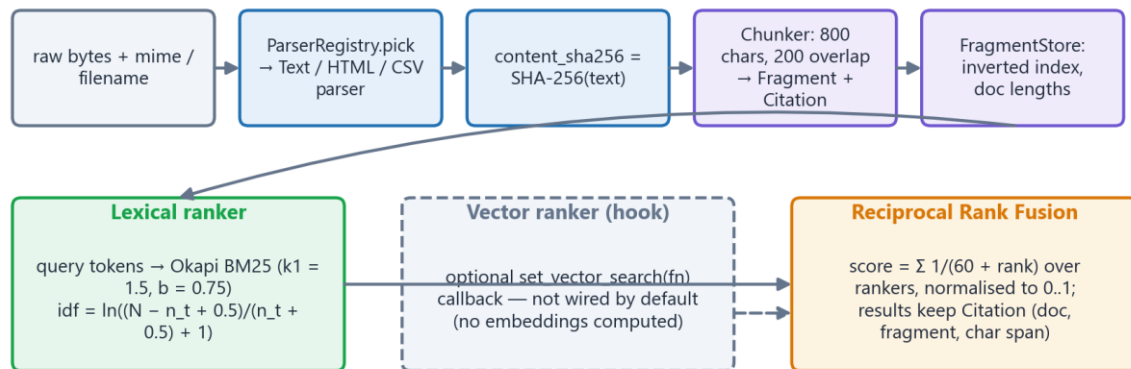


Figure 13.6: BM25 + Reciprocal Rank Fusion retrieval

13.10 A10 Sliding-window rate limiting

Algorithm	A10 — Sliding-window rate limiting
Purpose	Throttle login/refresh.
Inputs	key, limit, window
Outputs	allow or RateLimitExceeded(retry_after)
Complexity	$O(k)$ prune of expired timestamps
Implementation	security/___init___py InMemoryRateLimiter

```

drop timestamps older than now - window
if count ≥ limit: raise RateLimitExceeded(retry_after = window - (now - oldest) + 1)
append now

```

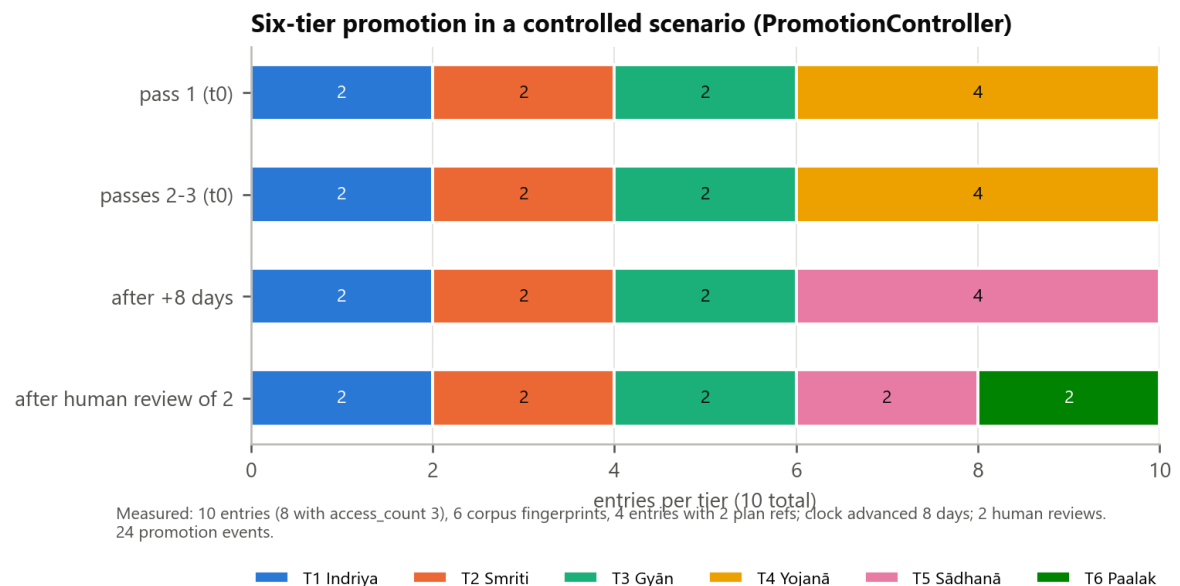


Figure 13.7: Measured tier occupancy in a controlled promotion scenario

Entries move from T1 to T4 in the first pass (access, corpus overlap and plan references are present), to T5 after the clock is advanced by eight days, and two reach T6 after a human-review mark. Five repetitions produced the same provenance tail hash.

14 UI/UX Requirements

Screens are documented from screenshots of the actual application, captured with headless Microsoft Edge (1600 × 1000 px) against an isolated copy of the backend seeded with four conversations and two documents (Appendix G). Each screen's data source is stated explicitly because several pages render placeholder data.

14.1 General UI requirements

Table 14.1: General UI requirements

ID	Requirement	Status
UI-01	A persistent sidebar shall provide ten destinations (Dashboard, Conversations, Agents, Bench Results, Bench Runner, Execution Timeline, Memory Explorer, Documents, Observability, Settings).	Implemented
UI-02	HTTP 401, 403, 429 and 5xx responses shall produce user-visible toasts with actionable text.	Implemented
UI-03	When live data is unavailable a page shall say so ('Backend unreachable — falling back to local mock data' / 'Demo mode').	Partially implemented (pages that receive empty arrays show mock data without the banner)
UI-04	The UI shall support a light/dark theme toggle.	Implemented
UI-05	Runtime data shall be validated with Zod schemas before rendering.	Partially implemented (bench, observability, traces, memory; not conversations)
UI-06	The UI should meet WCAG 2.1 AA.	Not verified (axe script exists, not run)

14.2 Dashboard (/)

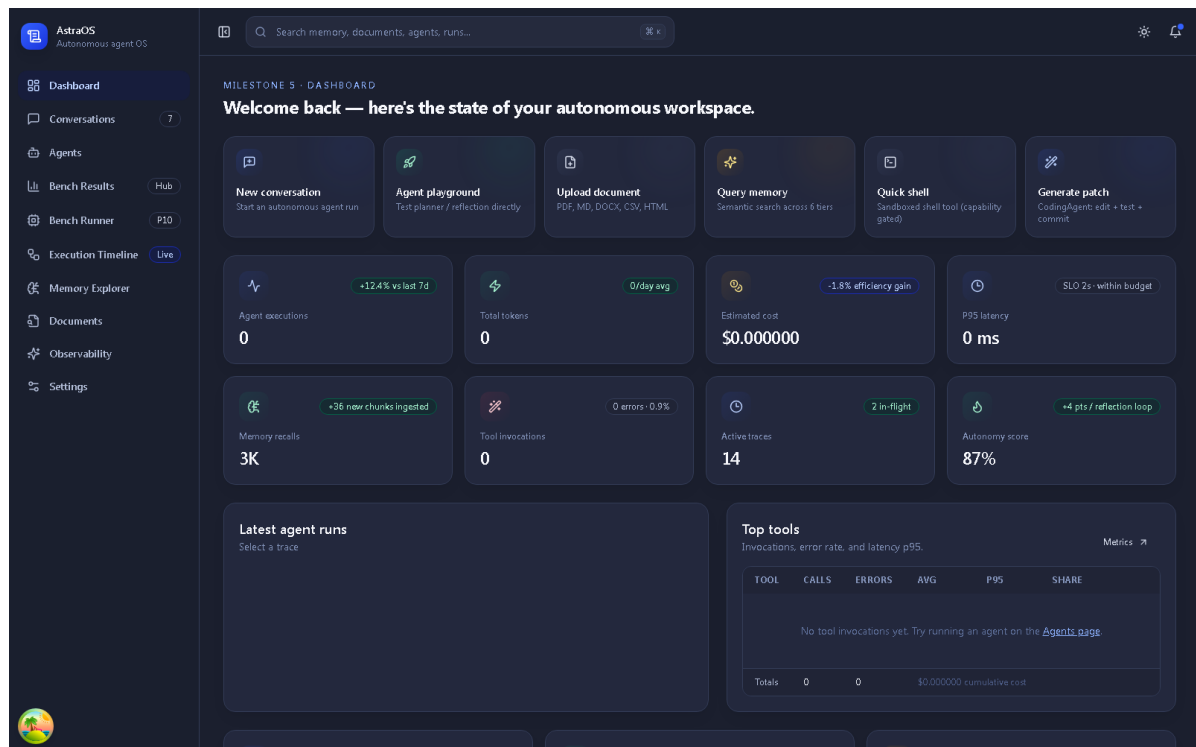


Figure 14.1: Dashboard (/) — screenshot of the running application

Purpose	Landing overview (the sidebar still carries the legacy product name 'AstraOS'): quick actions, KPI tiles (agent executions, tokens, cost, latency, memory, tool calls), latest agent runs, top tools.
Components	Quick-action cards; KPI grid; mini timeline; top-tools table; metrics charts
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	Calls /v1/observability/summary (all zeros in this build) and /v1/kernel/traces (envelope not unwrapped) → seeded placeholder values are displayed.
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Placeholder

14.3 Conversations (/conversations)

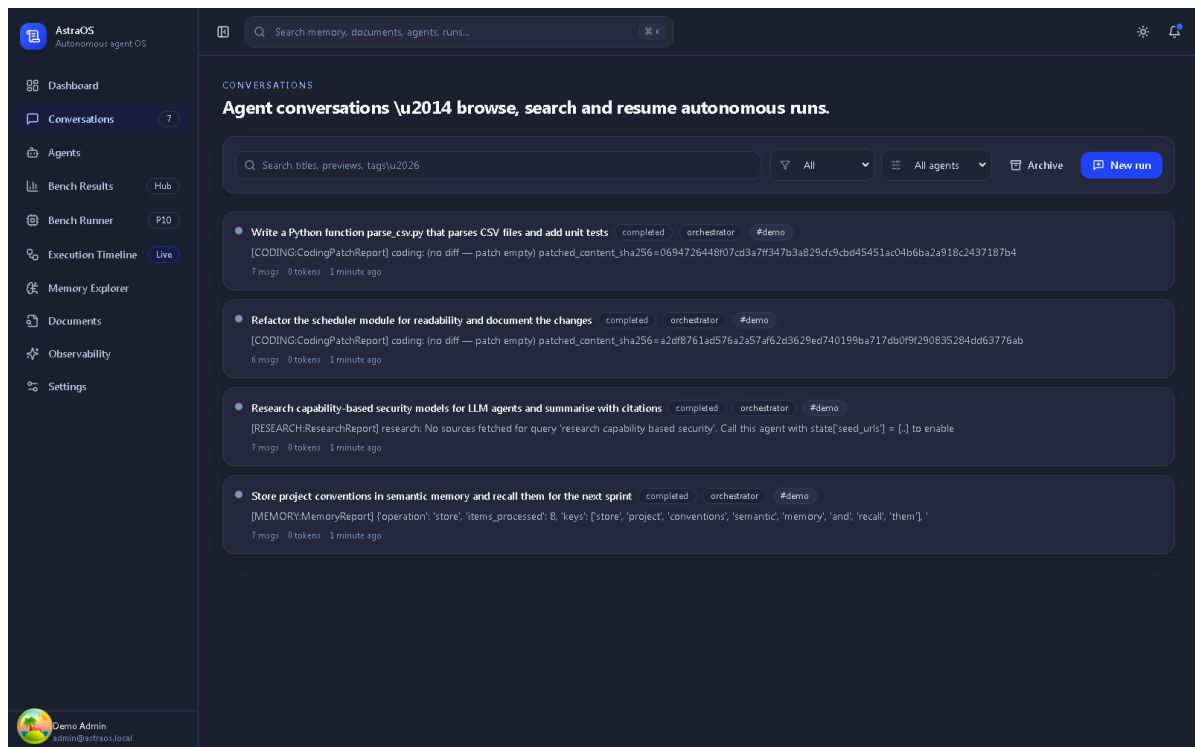


Figure 14.2: Conversations (/conversations) — screenshot of the running application

Purpose	Browse, search, filter and archive agent runs.
Components	Search box, status and agent filters, Archive and New-run buttons, conversation cards with status pill, codename badge, tags, message/token counts
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	Live: GET /v1/conversations (envelope unwrapped). Falls back to 11 mock conversations when the list is empty or the backend is unreachable.
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Live

14.4 New run (/conversations/new)

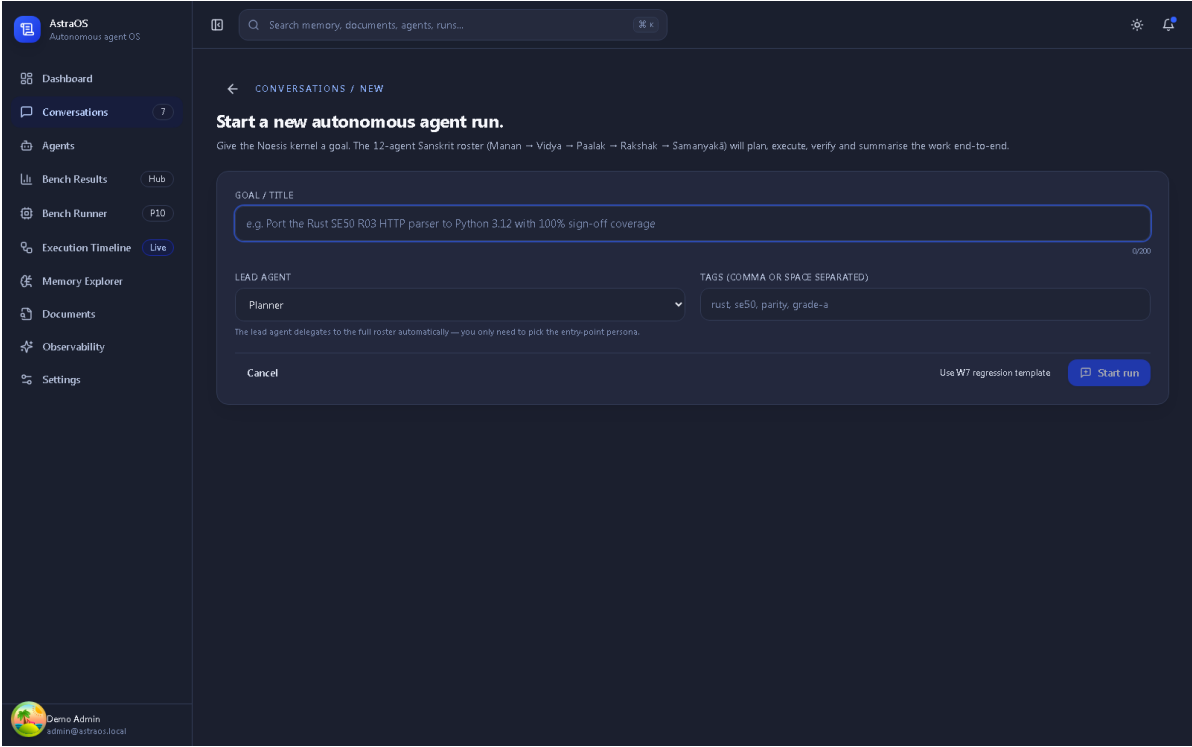


Figure 14.3: New run (/conversations/new) — screenshot of the running application

Purpose	Start a new autonomous run from a goal.
Components	Goal/title input, lead-agent select, tags input, Cancel and Start-run buttons
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	POST /v1/conversations; on success navigates to the thread; on error shows 'Conversation created (local mock)'.
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Live

14.5 Thread (/conversations/[id])

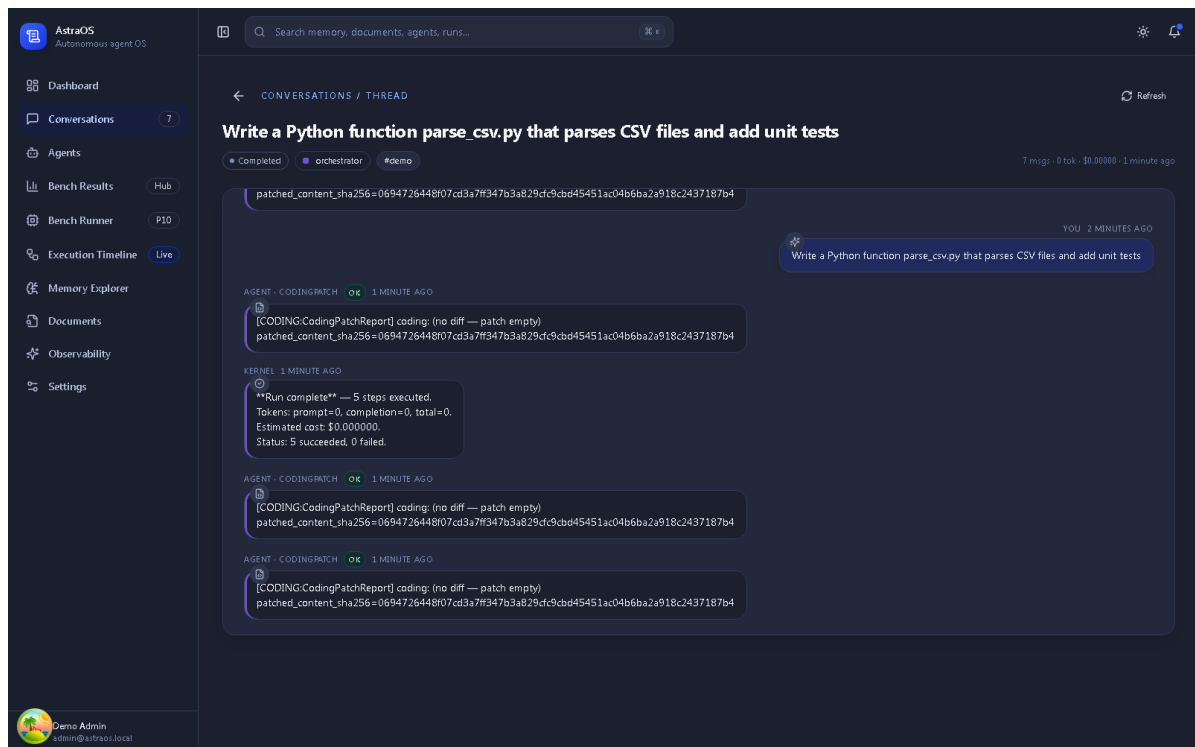


Figure 14.4: Thread (/conversations/[id]) — screenshot of the running application

Purpose	Show the goal and each agent's step report as chat bubbles.
Components	Header with status and agent, refresh button, message list with role-specific bubbles and codename badges
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	Live: GET /v1/conversations/{id}; disabled in demo mode.
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Live

14.6 Agents (/agents)

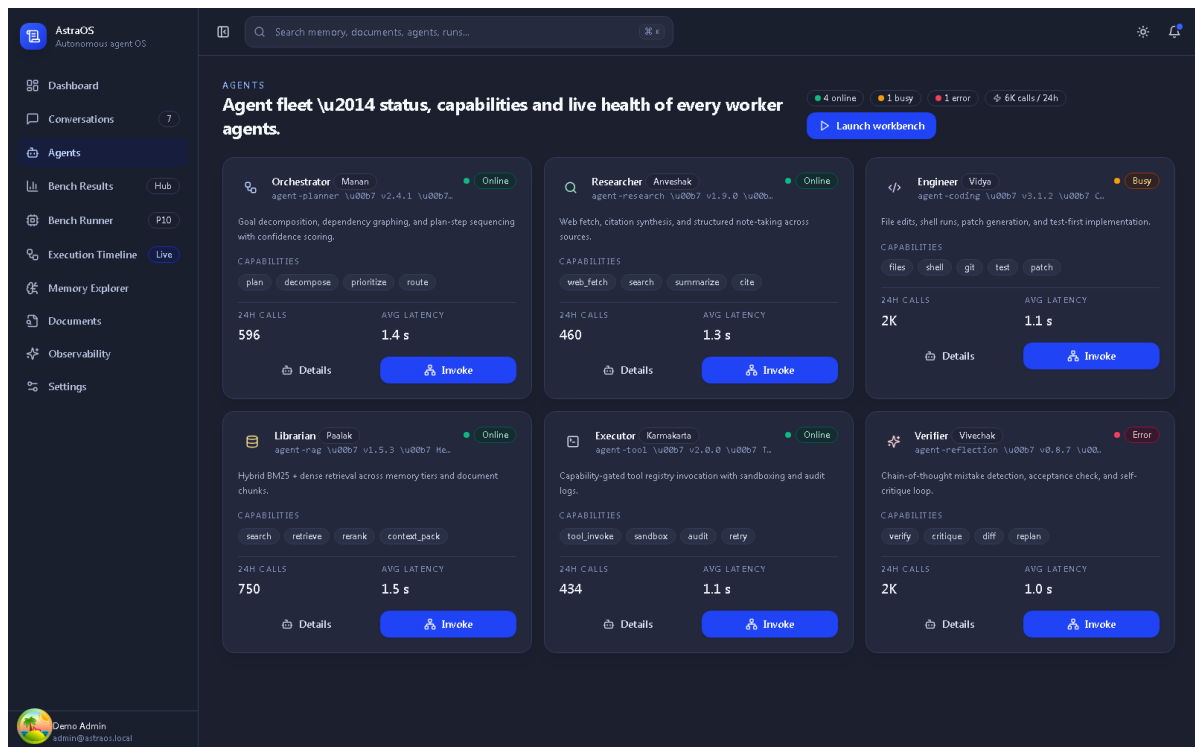


Figure 14.5: Agents (/agents) — screenshot of the running application

Purpose	Agent fleet status and a workbench to launch an agent.
Components	Agent cards (status, capabilities, 24 h calls, latency), Launch workbench, Invoke buttons
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	listAgents expects an array but receives an envelope → 9 seeded mock agents are shown; workbench posts {agent_id, prompt} to /v1/runs/execute, which the API rejects (schema mismatch).
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Placeholder

14.7 Bench Runner (/agents/benchmark)

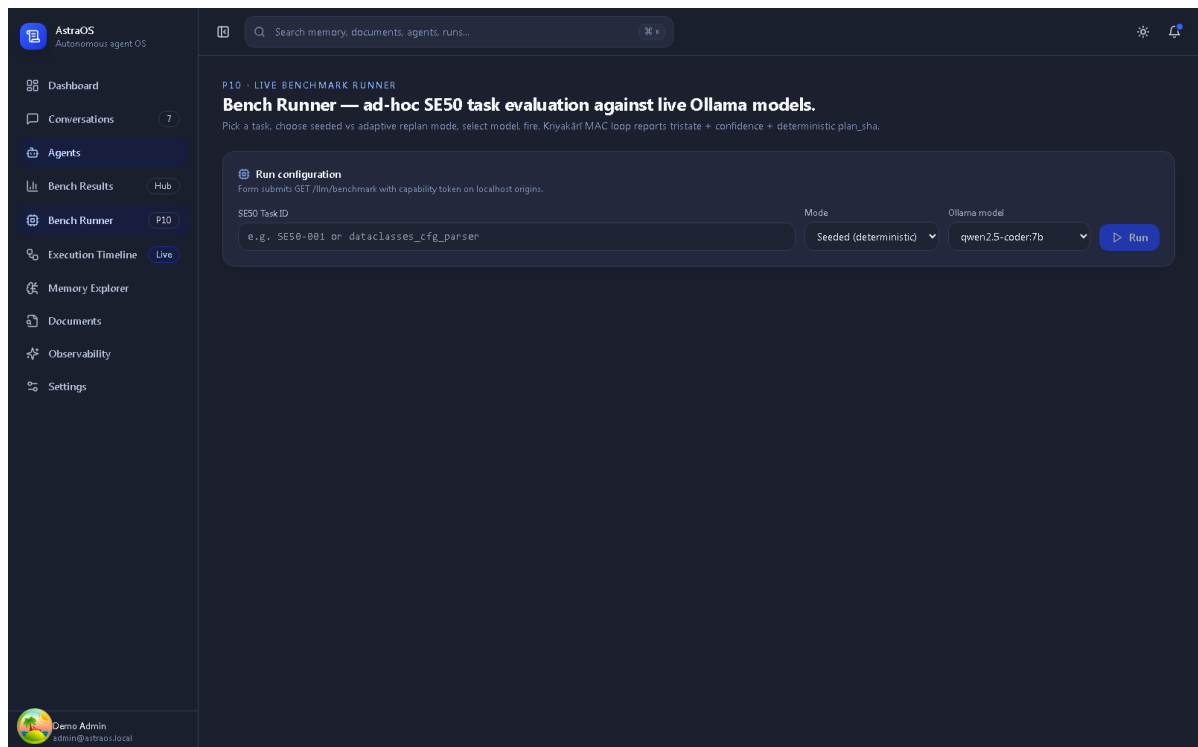


Figure 14.6: Bench Runner (/agents/benchmark) — screenshot of the running application

Purpose	Run an ad-hoc LLM benchmark.
Components	Task ID, mode and model selects, Run button; result card with tri-state badge
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	GET http://localhost:8000/llm/benchmark with the development capability token; results labelled 'Mock / offline' when applicable.
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Live (localhost)

14.8 Bench Results Hub (/benchmarks)

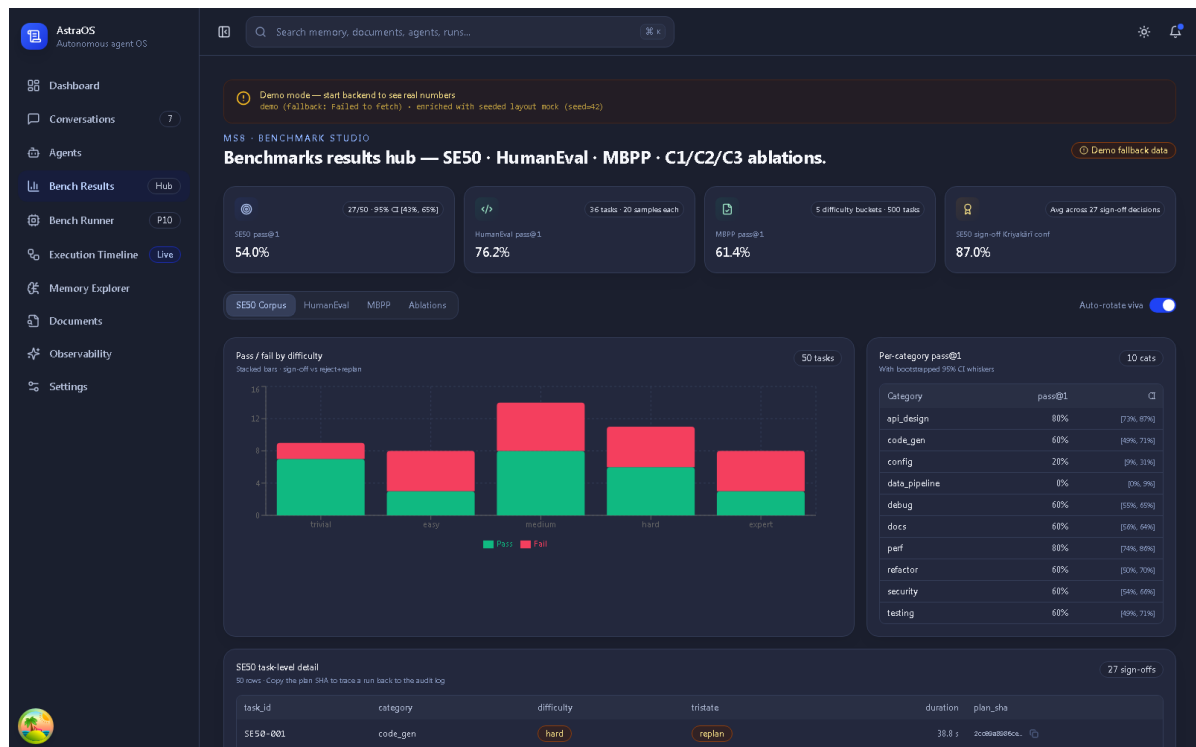


Figure 14.7: Bench Results Hub (/benchmarks) — screenshot of the running application

Purpose	SE-50, HumanEval, MBPP and ablation results with charts.
Components	KPI tiles, tab bar (SE50 / HumanEval / MBPP / Ablations), stacked pass/fail chart, per-category table, task table, auto-rotate toggle
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	GET http://localhost:8000/api/bench/results ; the URL override is not inlined client-side, so in the captured run the fetch failed and the page labelled its demo data ('Demo mode — start backend to see real numbers').
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Demo (labelled)

14.9 Execution Timeline (/timeline)

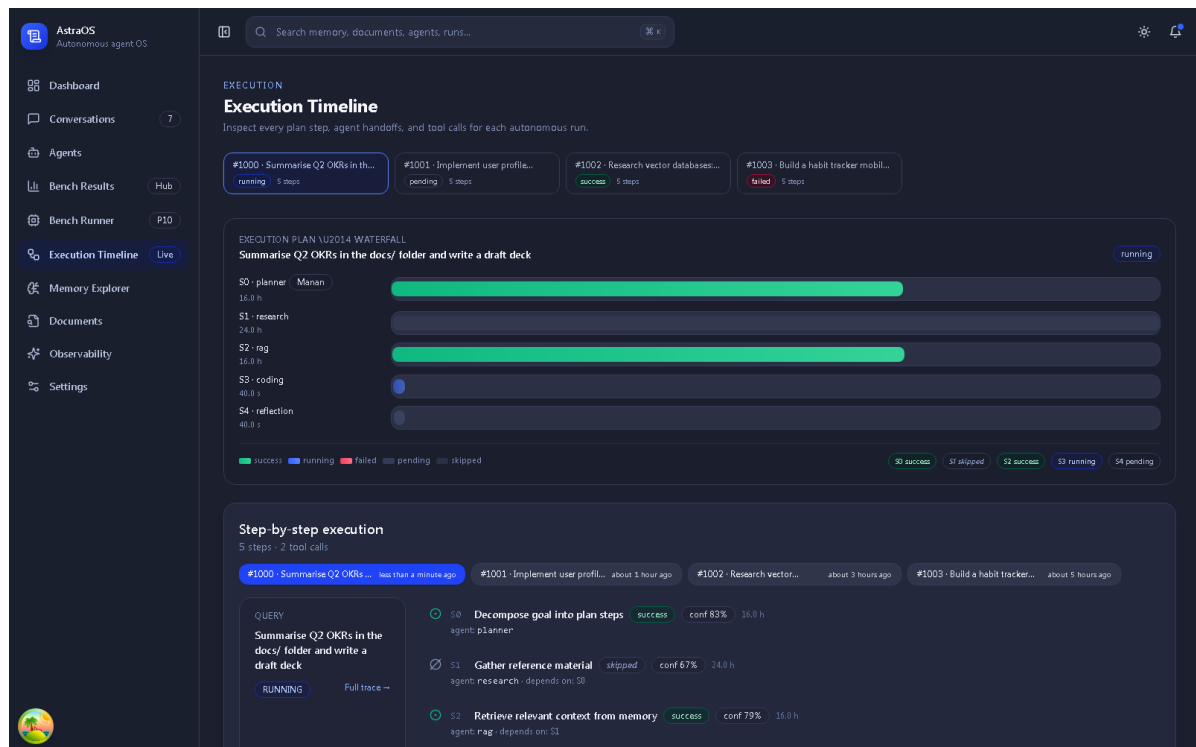


Figure 14.8: Execution Timeline (/timeline) — screenshot of the running application

Purpose	Waterfall of plan steps per run.
Components	Run selector tabs, waterfall bars per step, step-by-step list with status pills
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	listTraces (/v1/kernel/traces) returns an envelope that is not unwrapped → seeded traces.
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Placeholder

14.10 Memory (/memory)

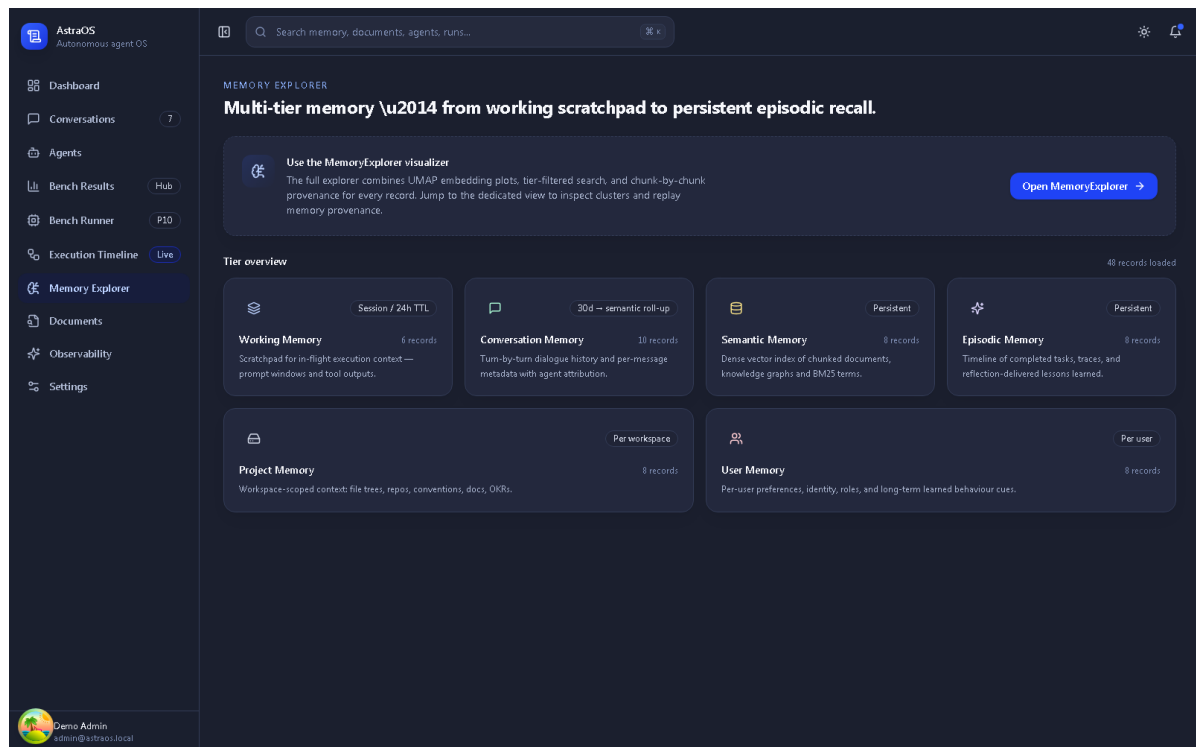


Figure 14.9: Memory (/memory) — screenshot of the running application

Purpose	Overview of memory tiers.
Components	Tier cards (working, conversation, semantic, episodic, project, user) with counts; link to explorer
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	listMemory → envelope not unwrapped → seeded records.
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Placeholder

14.11 Memory Explorer (/memory/explorer)

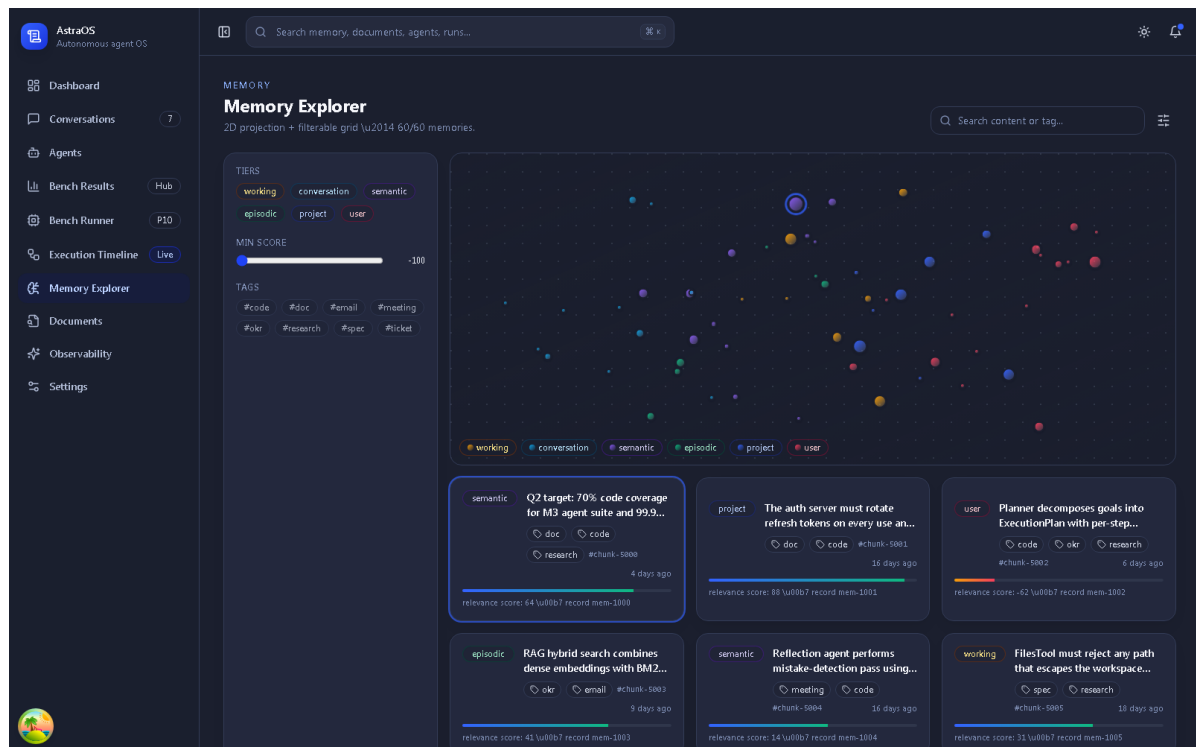


Figure 14.10: Memory Explorer (/memory/explorer) — screenshot of the running application

Purpose	2-D projection and searchable list of memories.
Components	Tier filters, minimum-score slider, tag chips, scatter projection, record cards
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	Seeded records (as above).
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Placeholder

14.12 Documents (/documents)

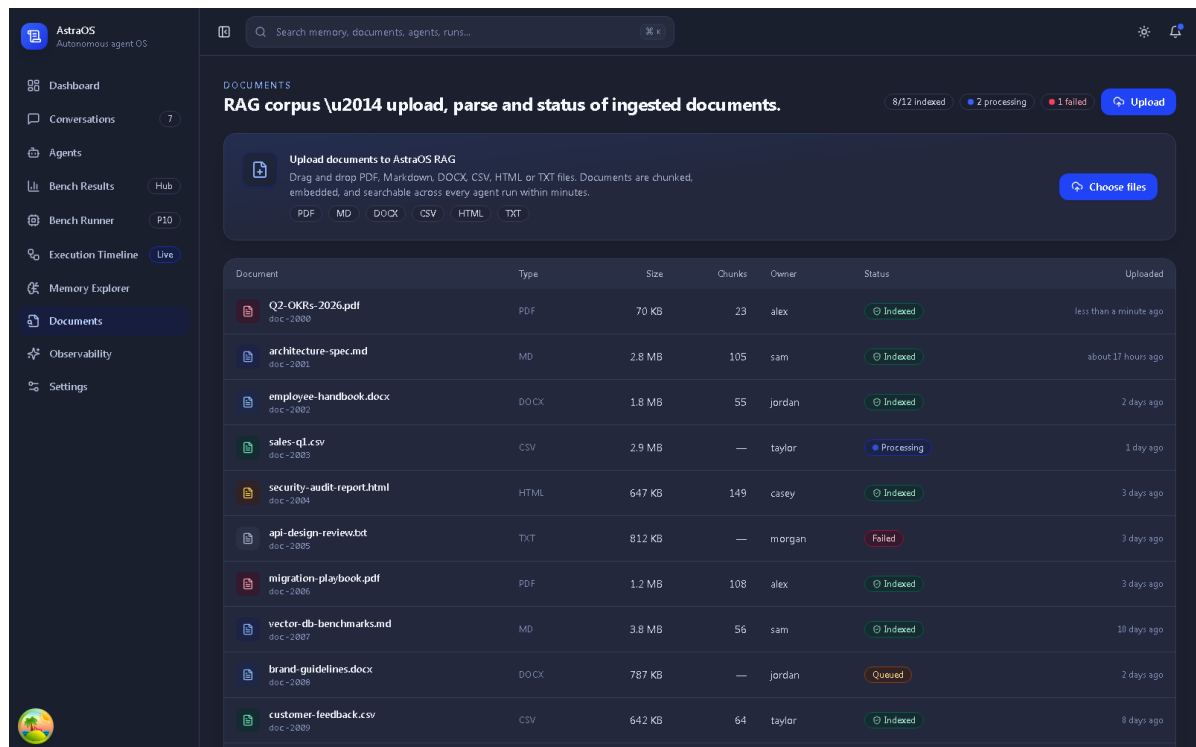


Figure 14.11: Documents (/documents) — screenshot of the running application

Purpose	Upload and track RAG documents.
Components	Drop zone with Choose-files button, status counters, document table (type, size, chunks, owner, status, uploaded)
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	listDocuments → envelope not unwrapped → seeded documents; upload posts multipart to /v1/documents/upload.
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Placeholder

14.13 Observability (/metrics)

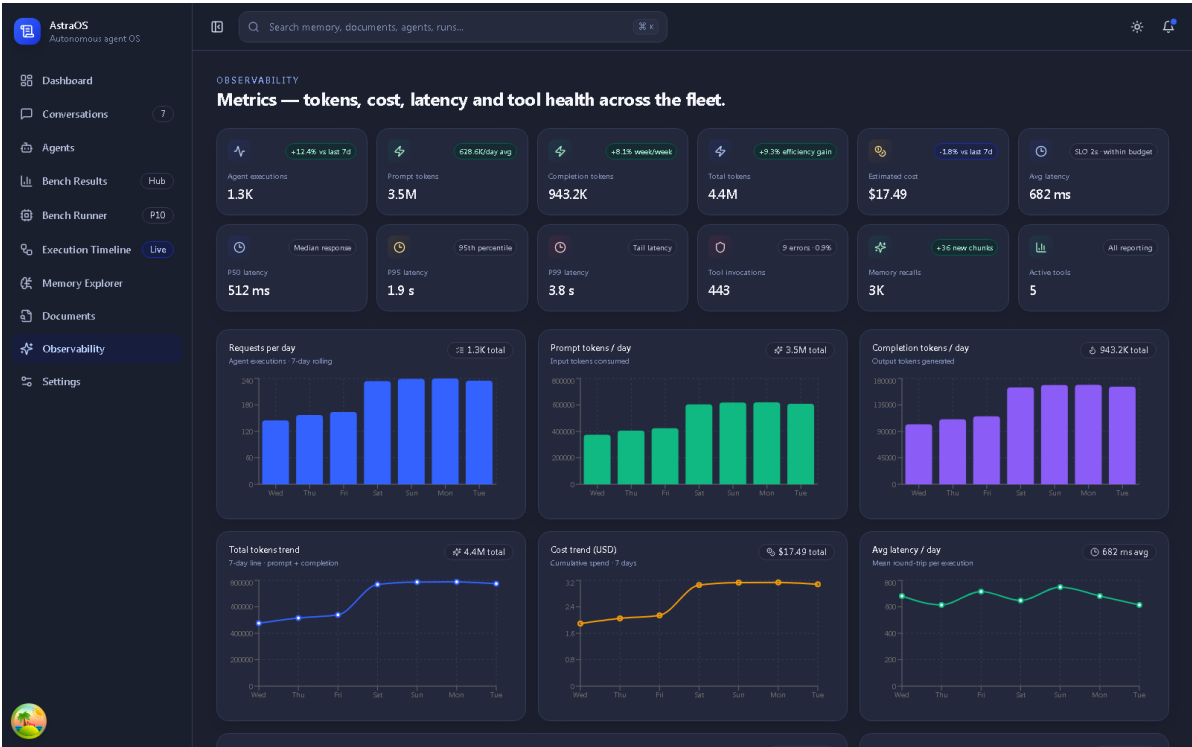


Figure 14.12: Observability (/metrics) — screenshot of the running application

Purpose	Token, cost, latency and tool health charts.
Components	KPI tiles, bar and line charts per day
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	fetchObservability returns zeros → seeded observability data.
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Placeholder

14.14 Settings (/settings)

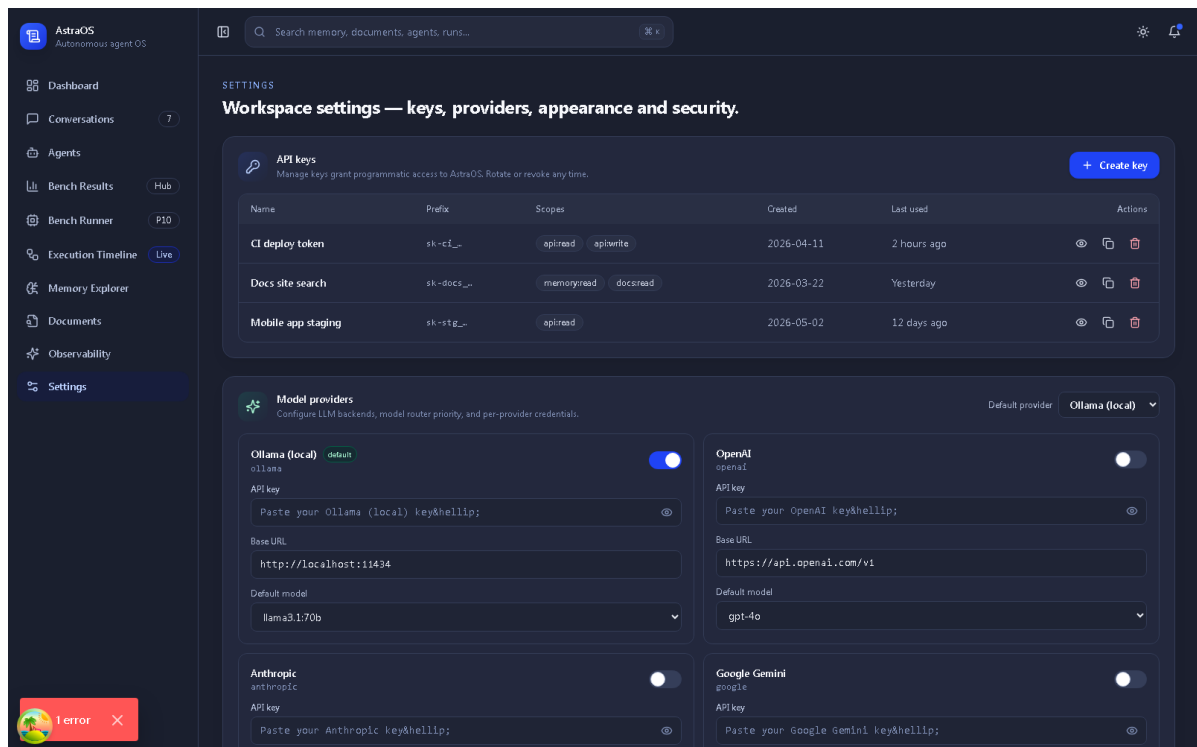


Figure 14.13: Settings (/settings) — screenshot of the running application

Purpose	API keys, model providers, appearance.
Components	API-key table, provider toggles with key/base-URL/model fields
Inputs / user actions	Navigation, filters and forms as listed under components
Data source and outputs	Browser-local state only; nothing is persisted or sent.
Validation and error states	React Query error → banner/toast; Zod parse failure → fallback data; HTTP errors → toasts (UI-02)
Live-data status	Local only

15 Security Requirements

Security is analysed from the implementation, following the protection principles of Saltzer and Schroeder [3] and the OWASP guidance for LLM applications [39]. Only mechanisms present in the code are claimed.

Table 15.1: Security mechanisms and gaps

Area	Implemented mechanism	Gaps / risks
Authentication	JWT HS256, 15-min access / 14-day refresh, iss/aud/token_use/exp/nbf checks, constant-time signature compare; secret < 32 chars is stretched with SHA-256.	Built-in user directory; admin password is the hard-coded default ('change-me-please') because Settings has no ADMIN_PASSWORD field; demo user credentials are in source; no logout or token revocation.
Password storage	bcrypt (12 rounds) via passlib, scrypt fallback ($N = 2^{15}$, $r = 8$, $p = 1$) for long passwords; dummy verification for unknown users.	Hashes are recomputed per login (≈ 180 ms).

Area	Implemented mechanism	Gaps / risks
Authorisation	Kernel capability table; AgentRoster spawn check; ToolRegistry TOOL_INVOKE gate; HMAC capability tokens on two routes with deny-mask precedence.	No authentication or ownership checks on /v1 resource routes; roles unused.
Credential handling	SecretStr settings; start-up rejects development secret in staging/production; logs redact sensitive keys and bearer values.	Default HMAC secret 'dev-only-insecure-secret-change-in-prod' is not rejected in production; the frontend ships a signed development capability token (sent only to localhost bases).
API security	CORS allow-list (wildcard refused), request validation with extra=forbid, rate limiting on login/refresh, docs disabled in production.	No rate limit on other routes; no CSRF tokens (credentials allowed with explicit origins); no request-size limit on uploads.
Input validation	Pydantic bounds; InputSanitiser for user_id; path-traversal checks; AST allow-list for Python; SQL keyword allow-list with rollback for DBTool.	LLM output stored unescaped in messages (rendered as text by React).
Tool containment	Shell is dry-run unless allow-listed; files rooted at var/workspace; web_fetch allows only http/https.	web_fetch has no private-address (SSRF) filter; sandbox timeout leaves a daemon thread running; CodingAgent may overwrite workspace files.
Data protection	SQLite file on local disk; no third-party transmission unless a hosted LLM is configured.	No encryption at rest; HTTPS left to a reverse proxy.
Secrets management	Environment variables / .env files (gitignored).	No vault integration.

15.1 Security requirements

Table 15.2: Security requirements

ID	Requirement	Status
SEC-01	All mutating /v1 routes shall require an authenticated principal.	Not met
SEC-02	Passwords shall be stored only as salted slow hashes.	Met
SEC-03	Tokens shall be verified with constant-time comparison and full claim validation.	Met
SEC-04	Login attempts shall be throttled per client and user.	Met (5/60 s)
SEC-05	Agents shall only invoke tools listed in their capability set.	Met (ToolRegistry, measured)
SEC-06	Gated routes shall deny on any of: missing, malformed, forged, expired, missing-capability or masked tokens.	Met (6/6 measured)
SEC-07	Production start-up shall reject insecure defaults.	Partially met (SECRET_KEY yes, HMAC secret no)
SEC-08	Outbound fetches shall not reach private networks.	Not met
SEC-09	Logs shall not contain secrets or bearer tokens.	Met
SEC-10	Untrusted Python shall run without imports outside an allow-list and without file access.	Met (measured)

15.2 Threat considerations

Table 15.3: Threat analysis

Threat	Vector	Mitigation present	Residual risk
Unauthorised data access	Direct calls to /v1 routes	None	High on any non-localhost deployment
Credential stuffing	POST /v1/auth/login	Rate limit + slow hash	Low
Token forgery	Crafted JWT / capability token	HMAC-SHA256 with constant-time compare	Low if secrets are changed from defaults
Prompt injection	Fetches pages or user goals reaching the LLM	LLM output is never executed or parsed into actions	Low today; rises if LLM output drives tools
SSRF	web_fetch with internal URLs	Scheme allow-list only	Medium
Sandbox escape / DoS	python_sandbox code	AST + builtins allow-list; 5 s join timeout	Medium (runaway thread continues)

Threat	Vector	Mitigation present	Residual risk
Path traversal	files tool paths	resolve() under allowed root	Low
Resource exhaustion	Large uploads; long LLM prompts	None / 120 s timeout	Medium

16 Error Handling

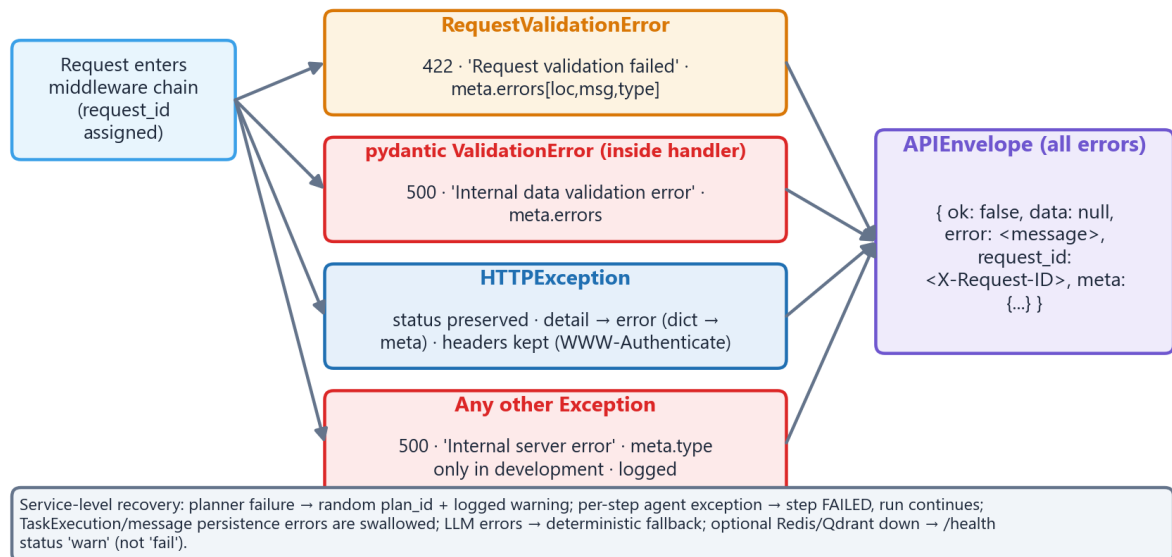


Figure 16.1: Exception handling and recovery paths

Four exception handlers convert every error into the standard envelope; service-level code recovers locally where a failure should not abort a run.

Table 16.1: Error classes and handling

Error class	Example	Behaviour (observed)	HTTP
Input error	Missing conversation title	'Request validation failed', meta.errors[{loc, msg, type}]	422
Input error inside handler	Invalid plan passed to /runs/execute	'Internal data validation error' with errors — should be 422	500
Not found	Unknown conversation/document id	'<Entity> <id> not found'	404
Authentication	Missing/tampered/expired/wrong-use JWT	'missing bearer token' / 'jwt signature mismatch' / 'jwt expired' / token_use message; WWW-Authenticate: Bearer	401
Authorisation	Capability token problems	C1_DENY_* reason codes	403
Throttling	Sixth login in 60 s	error 'HTTP 429', meta.retry_after_s	429
Unknown agent	invoke 'judge'	'Unknown agent 'judge''	400
Database error	Table missing / constraint	Generic 'Internal server error' (type shown in development); TaskExecution persistence errors swallowed	500
Model error	Ollama down, timeout, HTTP 5xx	Deterministic fallback; FailoverProvider switches provider; benchmark returns mock values	200
Network failure	Redis/Qdrant unreachable	Readiness 'warn' with error text in checks	200
Invalid state	Circular plan dependencies	ValueError → 500 envelope	500
Kernel misuse	Syscall in SHUT_DOWN state / unknown handle	SysCallResult(success=false, error=...)	n/a

16.1 Recovery behaviour

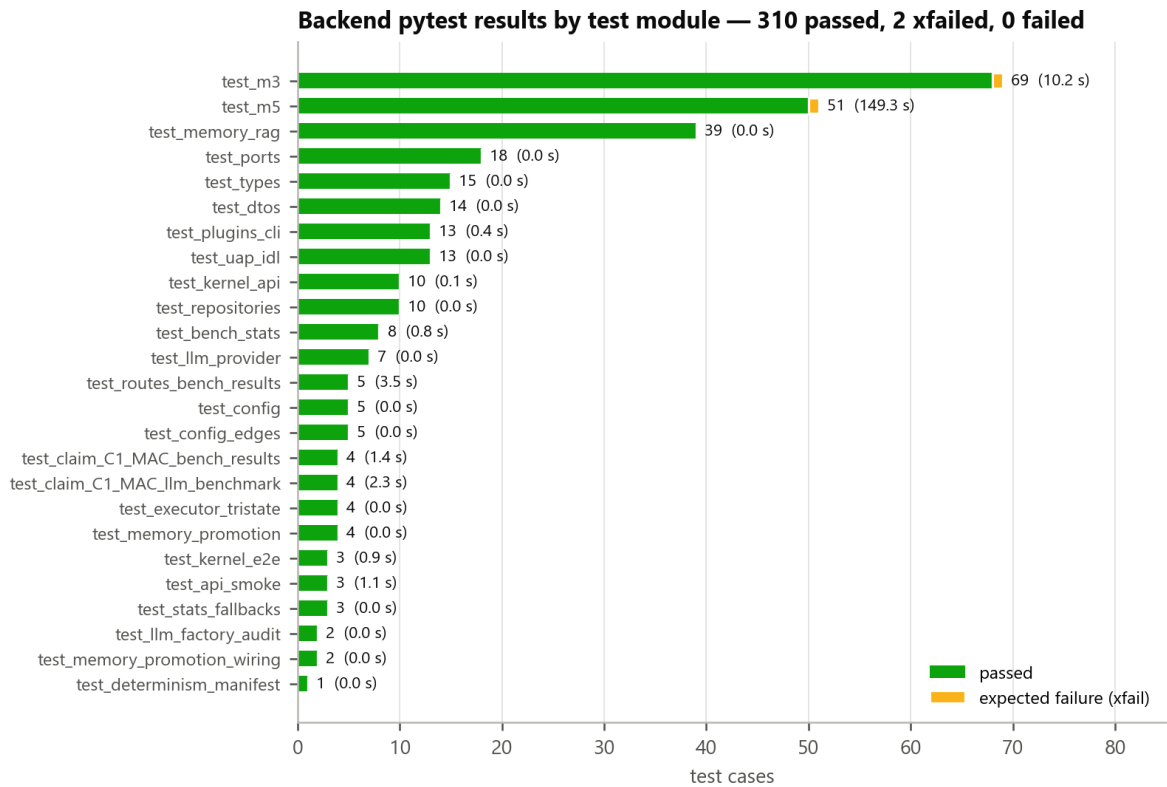
- Background run failure writes a '**Run failed**' assistant message and sets the conversation status to failed.
- Planner failure at creation time substitutes a random plan id; the background task re-plans.
- Kernel and plugin shutdown use 2-second timeouts so the process cannot hang on exit.
- Provider HTTP calls retry up to four times with jittered exponential back-off on transient status codes.

17 Testing Requirements and Traceability

17.1 Test strategy

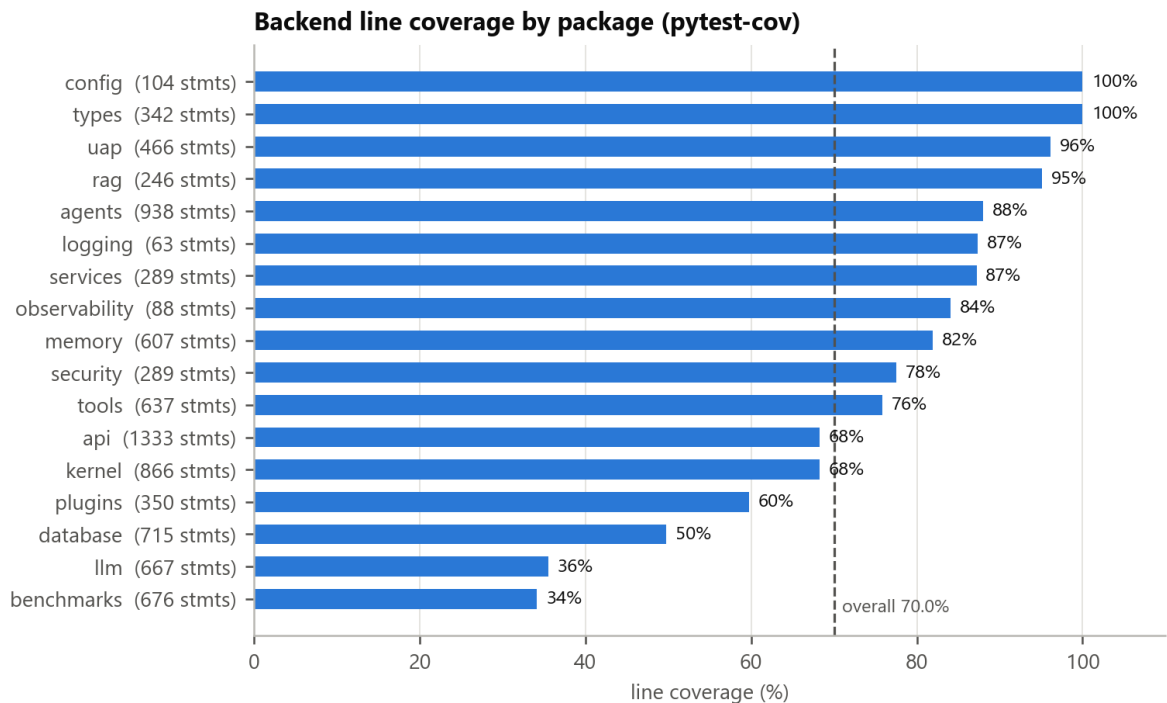
Table 17.1: Test levels and results

Level	Tooling	Scope in this project	Measured result
Unit	pytest 9, pytest-asyncio	Types, config, DTOs, kernel, agents, tools, security, memory, RAG, UAP, stats, plugins	310 passed, 2 xfail, 0 failed
Integration	FastAPI TestClient with lifespan	Kernel boot via app lifespan, stats, traces	3/3 passed
API / contract	TestClient; claim suites	Routes for conversations, runs, documents, memory, UAP, metrics; capability-gate allow/deny	Included above
Security claim suites	pytest markers C1/MAC	4 allow/deny tests per gated route	8/8 passed
Frontend unit/component	Vitest 2, Testing Library, jsdom	HTTP wrapper, schemas, mocks, dashboard components, bench pages	103/105 passed
Static analysis	tsc, ESLint, ruff	Type and lint gates	tsc 0 errors; ESLint 0/0; ruff not re-run
System / end-to-end	Manual + scripted HTTP evidence	58 request/response scenarios, latency micro-benchmarks, UI screenshots	Appendix G
Performance	TestClient timing; /lm/benchmark	Latency per endpoint; LLM throughput	Chapter 6
ML evaluation	Benchmark harness	SE-50, HumanEval/MBPP compile checks, determinism manifest	Table 12.4
Accessibility	axe-core script	CI step only	Not run



Measured 2026-10-06, Python 3.13.2, Windows 11, isolated copy of the working tree; total wall time 180.2 s (test_m5 dominates because AgentSvc tests call the local LLM).

Figure 17.1: Backend test results per module (measured)



Measured with coverage.py over noesis/ (cli.py, core/*, llm/_retry.py, llm/parse.py, llm/prompts.py omitted by pyproject configuration).

Figure 17.2: Backend line coverage per package (measured)

Overall coverage is 70.0 %. Configuration, types, protocol and RAG code are almost fully covered; LLM adapters (36 %) and the statistics module (34 %) are the least covered

because hosted providers are not called in tests and most aggregation code paths are exercised only by scripts.

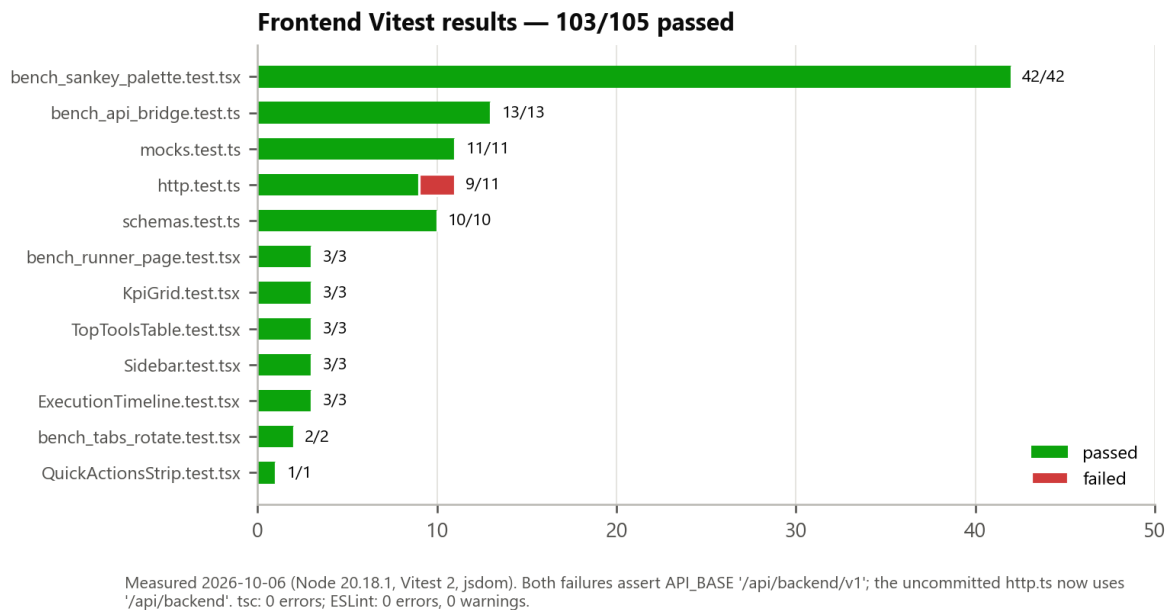


Figure 17.3: Frontend Vitest results per file (measured)

17.2 Requirements traceability matrix

Table 17.2: Requirements traceability matrix (functional requirements)

Req.	Requirement	Design / implementation	Test case(s)	Status
FR-001	User login and token issuance	api/routes/v1_m3.py: login()	test_m3::test_auth_login_demo_user_returns_token_pair, test_auth_login_bad_password_is_401...	Implemented
FR-002	Identity introspection (/auth/me)	api/routes/v1_m3.py: me(), require_auth()	test_m3::test_auth_me_returns_subject, test_auth_me_missing_bearer_returns_401, test_jwt_t...	Implemented
FR-003	Access-token refresh	api/routes/v1_m3.py: refresh()	test_m3::test_auth_refresh_issues_new_access	Implemented
FR-004	Production secret enforcement	config.py: Settings_debug_for_dev_only	test_config::test_debug_forbidden_in_production, test_config_edges::test_cors_wildcard_rej...	Implemented
FR-005	Role-based authorisation on resource routes	api/routes/v1_m3.py: require_auth()	test_m3::test_observability_summary_requires_admin_role (marked xfail: guard intentionally...)	Planned (not implemented)
FR-006	Create conversation from a goal	api/routes/v1_m5.py: create_conversation(), _bg_run_and_messages()	test_m5::test_conversations_create_and_list, test_conversations_crud_lifecycle	Implemented
FR-007	List, filter and search conversations	api/routes/v1_m5.py: list_conversations()	test_m5::test_v1_conversations_list_has_agent_field_in_envelope	Implemented
FR-008	Read a conversation thread	api/routes/v1_m5.py: get_conversation()	test_m5::test_conversations_crud_lifecycle	Implemented
FR-009	Append message to a conversation	api/routes/v1_m5.py: append_message()	test_m5::test_conversations_crud_lifecycle	Implemented
FR-010	Update conversation (title, status, tags)	api/routes/v1_m5.py: patch_conversation()	test_m5::test_conversations_crud_lifecycle	Implemented
FR-011	Delete conversation	api/routes/v1_m5.py: delete_conversation()	test_m5::test_conversations_crud_lifecycle	Implemented
FR-012	List messages of a conversation	api/routes/v1_m5.py: list_messages()	test_repositories::test_message_list_for_conversation	Implemented
FR-013	Deterministic goal decomposition	agents/core.py: PlannerAgent	test_m5::test_planner_same_seed_produces_by_identical_plan, test_planner_run_always_appe...	Implemented

Req.	Requirement	Design / implementation	Test case(s)	Status
FR-014	Topological plan execution	services/kernel_services.py: AgentSvc.run_plan(), topological_order()	test_m5::test_agentsvc_topological_linear_chain, test_agentsvc_topological_diamond, test_a...	Implemented
FR-015	Execution audit trail	services/kernel_services.py: AgentSvc.run_plan()	test_m5::test_agentsvc_run_plan_persists_task_execution_rows_and_returns_ordered	Implemented
FR-016	List executions of a run	database/sql_repos.py: SqlTaskExecutionRepository.list_for_run()	test_repositories::test_task_execution_run_filter	Partially implemented
FR-017	Background run completion messages	api/routes/v1_m5.py: _bg_run_and_messages()	test_m5::test_agentsvc_run_plan_writes_assistant_messages_for_report_types	Implemented
FR-018	Agent catalogue	api/routes/v1_m5.py: list_agents()	test_m5::test_v1_agents_list_contains_manifest_fields	Implemented
FR-019	Direct single-agent invocation	api/routes/v1_m5.py: invoke_agent()	test_m5::test_agents_list_and_invoke	Implemented
FR-020	Spawn-time capability gating of agents	agents/core.py: AgentRoster.spawn()	test_m3::test_agent_roster_spawn_gate_rejects_insufficient_token, test_agent_roster_spawn ...	Implemented
FR-021	Twelve typed agents	agents/core.py	test_m3::test_agent_roster_exactly_12_matches_enum	Implemented
FR-022	Research agent	agents/core.py: ResearchAgent	test_m3::test_research_agent_synthesises_multiple_sources, test_research_agent_plan_only_w...	Implemented
FR-023	Coding agent	agents/core.py: CodingAgent	test_m3::test_coding_agent_returns_plan_and_patches, test_coding_agent_patch_ambiguous_mat...	Implemented
FR-024	Reflection, judge and critic agents	agents/core.py	test_m3::test_reflection_reports_contradiction, test_judge_ranks_candidates_and_picks_winner ...	Implemented
FR-025	Executor tri-state decision	agents/core.py: ExecutorAgent.evaluate_decision()	test_executor_tristate (4 tests), test_m3::test_executor_signs_off_good_answer	Implemented
FR-026	Memory, RAG, tool, supervisor and orchestrator agents	agents/core.py	test_m3::test_memory_agent *, test_rag_agent *, test_tool_agent *, test_supervisor *, test...	Stub / simulated
FR-027	Register document metadata	api/routes/v1_m5.py: create_document()	test_m5::test_documents_lifecycle_create_list_get_upload_delete	Implemented
FR-028	Upload document file	api/routes/v1_m5.py: upload_document()	test_m5::test_documents_lifecycle_create_list_get_upload_delete	Stub / simulated
FR-029	List, get and delete documents	api/routes/v1_m5.py	test_m5::test_documents_lifecycle_create_list_get_upload_delete	Implemented
FR-030	List memory records	api/routes/v1_m5.py: list_memory()	test_m5::test_memory_list_query_compress	Partially implemented
FR-031	Query memory records	api/routes/v1_m5.py: query_memory()	test_m5::test_memory_list_query_compress	Partially implemented
FR-032	Compress stale memory	api/routes/v1_m5.py: compress_memory()	test_m5::test_memory_list_query_compress	Implemented
FR-033	Six-tier memory promotion with provenance	memory/promotion.py: PromotionController	test_memory_promotion (4 tests), test_memory_promotion_wiring (2 tests)	Partially implemented
FR-034	Tier persistence across restarts	api/main.py lifespan	—	Planned (not implemented)
FR-035	Multi-tier recall bus	memory/bus.py: MemoryBus	test_memory_rag::test_bus * (7 tests)	Implemented as library (not wired to API)
FR-036	Kernel lifecycle and state	kernel/kernel.py	test_kernel_api::test_kernel_state_shows_uptime_after_start, integration::test_lifespan_bo...	Implemented
FR-037	Capability-checked system calls	kernel/kernel.py: Kernel.syscall()	test_kernel_api (10 tests); scripts/audit_mac_spawn.py	Implemented
FR-038	HMAC capability gate on HTTP routes	api/middleware/capability_gate.py	claim_suites (8 tests)	Implemented
FR-039	Kernel statistics and traces	api/routes/kernel.py	test_kernel_api::test_kernel_traces *	Implemented
FR-040	LLM enrichment with deterministic fallback	services/kernel_services.py: run_with_llm_or_fallback()	test_m5::test_agentsvc_run_step_research_agent_runs_and_returns_success	Implemented
FR-041	Scheduler, allocator, router and interrupts	kernel/scheduler.py, allocators.py, model_router.py, interrupts.py	test_kernel_api::test_kernel_allocator_zones_populated_after_malloc, test_kernel_model_pro...	Implemented as library (not wired to API)
FR-042	Capability-gated tool registry	tools/_init_.py: ToolRegistry	test_m3::test_registry_capability_gate_blocks_without_permission	Implemented
FR-043	Default sandboxed tools	tools/_init_.py	test_m3::test_shell_tool *, test_python_sandbox *, test_files_tool *, test_web_fetch_tool ...	Implemented

Req.	Requirement	Design / implementation	Test case(s)	Status
FR-044	Additional tool adapters	tools/_init_.py	test_m3::test_github_tool_dry_run_repo, test_db_tool_blocks_insert, test_calendar_tool_add...	Implemented as library (not wired to API)
FR-045	Pluggable LLM providers	llm/*.py	test_llm_provider (7), test_llm_factory_audit (2)	Implemented
FR-046	Provider failover	llm/factory.py: FailoverProvider	test_llm_factory_audit::test_factory_fallback_kicks_in_when_primary_raises	Implemented
FR-047	Embedding providers	llm/openai_provider.py, ollama_provider.py	—	Implemented as library (not wired to API)
FR-048	Liveness and readiness probes	api/routes/health.py	test_api_smoke::test_livez_always_pass, test_m5::test_v1_root_envelope_ok_true_for_health	Implemented
FR-049	Prometheus and JSON metrics	observability/metrics.py	test_m5::test_metrics_prometheus_scrape_returns_text, test_metrics_json_snapshot_envelope...	Partially implemented
FR-050	Observability summary	api/routes/v1_m3.py	test_m3::test_observability_summary_admin	Stub / simulated
FR-051	Request correlation and structured logs	api/main.py	test_api_smoke	Implemented
FR-052	LLM throughput benchmark endpoint	api/routes/llm_benchmark.py	test_routes_bench_results::test_llm_benchmark_endpoint_still_works, claim_suites	Implemented
FR-053	Benchmark results API	api/routes/bench_results.py	test_routes_bench_results (5 tests)	Implemented
FR-054	Offline evaluation harness	scripts/bench_*.py, scripts/run_w7_weekend.py, benchmarks/stats.py	test_bench_stats (8), test_stats_fallbacks (3), test_determinism_manifest (1)	Partially implemented
FR-055	UAP message model and wire codecs	uap/_init_.py, uap/wire.py, schemas/uap/v1/uap.proto	test_uap_idl (13 tests)	Implemented as library (not wired to API)
FR-056	UAP HTTP endpoints	api/routes/v1_m5.py	test_m5::test_uap_transports_bind_and_envelope	Stub / simulated
FR-057	Plugin discovery and lifecycle	plugins/*.py	test_plugins_cli (13 tests)	Implemented
FR-058	Command-line interface	cli.py	test_plugins_cli::test_noesis_version	Implemented
FR-059	Dashboard shell and navigation	frontend/app/layout.tsx, src/components/layout/*	Sidebar.test.tsx (3)	Implemented
FR-060	Conversation workflow in the UI	frontend/app/conversations/*	—	Implemented
FR-061	Monitoring pages (dashboard, agents, timeline, memory, documents, observability)	frontend/src/lib/api/endpoints.ts	mocks.test.ts, schemas.test.ts	Stub / simulated
FR-062	Benchmark hub and runner pages	frontend/app/benchmarks/page.tsx, app/agents/benchmark/page.tsx, src/lib/api/bench.ts	bench_api_bridge.test.ts (13), bench_runner_page.test.tsx (3), bench_tabs_rotate.test.tsx ...	Partially implemented
FR-063	Settings page	frontend/app/settings/page.tsx	—	Stub / simulated
FR-064	Document parsing and chunking	rag/pipeline.py	test_memory_rag::test_parser_*, test_chunker_*, test_ingest_*	Implemented as library (not wired to API)
FR-065	Hybrid lexical search	rag/pipeline.py: FragmentStore, HybridSearch	test_memory_rag::test_fragment_store_bm25_single_term_hits_the_relevant_chunk, test_hybrid...	Implemented as library (not wired to API)

17.3 Acceptance test cases (selected)

Table 17.3: Acceptance test cases executed during the audit

ID	Req.	Procedure	Expected	Result
TC-01	FR-001	POST /v1/auth/login demo/Demo!12345678	200 with two JWTs	Pass
TC-02	FR-001	Same with wrong password ×6	401 ×5 then 429	Pass
TC-03	FR-002	GET /v1/auth/me with tampered token	401 jwt signature mismatch	Pass
TC-04	FR-006	POST /v1/conversations with a goal; poll	Status completed; 7 messages	Pass
TC-05	FR-013	Plan the same goal 3× with seed 42	Identical SHA-256	Pass (50/50 goals)
TC-06	FR-014	Execute plan with cycle	ValueError	Pass (unit test)
TC-07	FR-016	GET /v1/runs/{run_id}/executions	Rows for the run	Fail (returns [])
TC-08	FR-038	GET /llm/benchmark with deny mask llm.*	403 C1 DENY_MASK	Pass

ID	Req.	Procedure	Expected	Result
TC-09	FR-043	<code>python_sandbox import os</code>	exit 66, blocked	Pass
TC-10	FR-043	<code>files read ../../etc/passwd</code>	exit 13, refused	Pass
TC-11	FR-033	Controlled promotion scenario ×5	Same provenance tail	Pass
TC-12	FR-048	<code>GET /health/readyz</code> without Redis/Qdrant	status warn, ok true	Pass
TC-13	FR-061	Open <code>/agents</code> with backend up	Backend agent list	Fail (mock data)
TC-14	NFR-SEC-03	<code>GET /v1/conversations</code> without token	401	Fail (200)

18 Deployment Requirements

Table 18.1: Deployment requirements

Item	Requirement
Hardware	x86-64 or ARM laptop/server; ≥ 8 GB RAM recommended when running a 7B model locally (model file 4.68 GB); disk for SQLite and workspace.
Software	Python 3.10–3.12 (3.13 works in practice); Node.js ≥ 20.17 for the dashboard; optional Ollama; optional Docker Compose.
Runtime	<code>uvicorn noesis.api.main:app --port 8000</code> ; next dev / next start on port 3000.
Build	<code>pip install -e backend[dev]</code> ; <code>npm ci</code> && <code>npm run build</code> in frontend; wheel built by hatchling (CI job).
Configuration	Environment variables (Appendix D); backend reads <code>.env</code> ; frontend uses <code>BACKEND_BASE_URL</code> (server) and <code>NEXT_PUBLIC_*</code> variables.
External services	Ollama (optional, localhost:11434); hosted LLM APIs (optional keys); Qdrant/Redis (optional, probed only).

18.1 Build and deployment process (verified path)

1. Install backend dependencies and set `SECRET_KEY` (≥ 32 characters) and, for non-development use, `CLAIM_SUITES_HMAC_SECRET`.
2. Optionally `ollama pull qwen2.5-coder:7b-instruct-q4_K_M` and run `ollama serve`.
3. Start the backend; on first start in development the SQLite file and tables are created.
4. Install frontend dependencies and start Next.js with `BACKEND_BASE_URL` pointing to the backend.
5. Verify `GET /health/readyz` and open the dashboard.

18.2 Container deployment (not executed)

`docker-compose.yml` defines backend and frontend images with health checks and volumes; `docker-compose.laptop.yml` adds Qdrant and Redis. Configuration analysis identifies issues that would need to be resolved before use:

- `docker-compose.yml` sets `APP_ENV=production` but provides `JWT_SECRET` instead of `SECRET_KEY`; Settings would therefore reject the default secret at start-up.

- `docker-compose.laptop.yml` sets `OLLAMA_BASE_URL` to `...:11434/v1`, while `OllamaProvider` appends `/api/chat`, producing `/v1/api/chat`.
- The backend `Dockerfile` installs `LangChain/LangGraph` and vendor SDKs that the code does not import (image size only).

18.3 Continuous integration

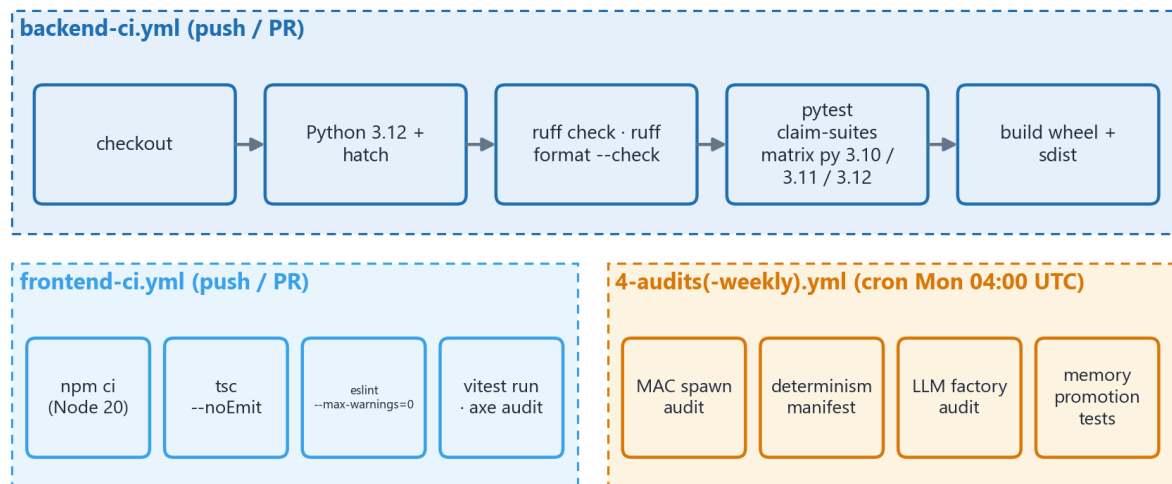


Figure 18.1: GitHub Actions workflows defined in the repository

Workflows exist for backend lint/tests/build, frontend type-check/lint/test/accessibility, and weekly audits. Their remote execution status was not verified during this audit.

19 Other Requirements

Table 19.1: Other requirements

ID	Requirement	Status
OTH-01 Backup	The SQLite file shall be backed up by the operator (file copy); the application provides no backup function.	Operator responsibility
OTH-02 Recovery	On restart, durable data (SQLite) is recovered; in-memory tiers, spans and rate limits are reset.	Partially implemented
OTH-03 Logging	One structured log line per request; warnings for degraded dependencies, fallbacks and swallowed errors.	Implemented
OTH-04 Monitoring	Prometheus counters for HTTP; readiness probe; kernel stats endpoint.	Partially implemented
OTH-05 Scalability	Single process; scale-out requires externalising in-memory state.	Planned (not implemented)
OTH-06 Documentation	SRS, thesis, presentation, README, ARCHITECTURE.md, OpenAPI.	Implemented
OTH-07 Licensing	MIT licence (LICENSE file); CITATION.cff provided.	Implemented

20 Limitations

Table 20.1: Known limitations

#	Limitation	Evidence	Impact
---	------------	----------	--------

#	Limitation	Evidence	Impact
L1	Agents are rule-based; the LLM only adds rationale text.	agents/core.py; run_with_llm_or_fallback	Task quality bounded by heuristics
L2	Resource routes have no authentication/authorisation.	No Depends(require_auth) on v1_m5 routes	Unsafe beyond localhost
L3	Run executions query filters by conversation_id.	SqlTaskExecutionRepository.list_for_run ; TC-07	Executions API unusable by run id
L4	Document upload and RAG agent are simulated.	upload_document; RAGAgent_mock_chunk	No real retrieval in runs
L5	Memory promotion is not exercised in live runs and tier persistence is not wired.	planner promotion_report T1 = 5 only; FR-034	Six-tier memory is a library capability
L6	Six dashboard pages show placeholder data.	endpoints.ts array checks; screenshots	Monitoring UI not live
L7	Observability summary and step metrics never populated.	record_request/observe_step_run not called	Limited runtime insight
L8	Benchmarks measure compile-ability and synthesised sign-off, not task correctness.	bench_humaneval/mbpp; run_full_seeded_pipeline	No evidence of coding ability
L9	Per-request Ollama probe adds latency; 1024-token prompts time out at 120 s.	Measured	Slow plan/execute when Ollama is down
L10	Single-process state (rate limiter, tiers, spans, metrics).	Code inspection	No horizontal scaling
L11	Container configuration defects; deployment not verified.	Chapter 18	Docker path may fail
L12	Frontend unit tests: 2 failures on the working tree.	http.test.ts	CI would fail until test or code is aligned
L13	Inconsistent Sanskrit codename mappings between backend prompts, frontend and harness.	Appendix F	Documentation confusion

21 Future Scope

The following items are proposals; none is implemented.

Table 21.1: Future enhancements (proposed)

Area	Proposed enhancement	Builds on
Security	Apply require_auth and ownership checks to all /v1 routes; reject default HMAC secret in production; SSRF filter for web_fetch; process-based sandbox.	v1_m3.require_auth, capability_gate
Agents	LLM-backed agents that emit the existing typed reports via structured output parsing (llm/parse.py exists) with deterministic fallback.	Report models, prompts registry
Retrieval	Wire IngestPipeline + HybridSearch into /v1/documents/upload and RAGAgent; add embeddings through EmbeddingPort and Qdrant.	rag/pipeline.py, database/qdrant.py
Memory	Register a PromotionController on the kernel so tiers persist; feed real access counts from recall.	memory/promotion.py save/load
Runs	Add a run_id column and index; stream step progress over Server-Sent Events.	task_executions
Dashboard	Unwrap envelopes in endpoints.ts so the six placeholder pages show live data.	requestJson
Evaluation	Execute unit tests for HumanEval/MBPP (functional pass@k) and evaluate on SWE-bench Lite.	bench_* scripts, stats.py
Operations	Externalise state (Redis rate limits, Prometheus multi-process), fix Compose files, publish images.	redis client, Dockerfiles
Edge deployment	Characterise the pipeline on low-power hardware (e.g. NVIDIA Jetson) as proposed in the synopsis.	Ollama provider

Appendix A: Glossary

Table A.1: Glossary

Term	Definition
Agent	A class implementing <code>run(state, ctx)</code> that performs one kind of plan step and returns a typed report.
AgentType	Enumeration of the 12 agent kinds (planner ... orchestrator).
Capability	An (operation, target-glob) pair granting one permission, e.g. <code>TOOL_INVOKE:files</code> .
Capability token	Kernel: opaque UUID identifying an agent's capability set. HTTP gate: signed JSON payload in <code>X-Noesis-Capability-Token</code> .
Syscall	A kernel operation invoked through <code>Kernel.syscall(handle, SysCall, payload)</code> ; 27 identifiers are defined.
Envelope	The standard JSON response wrapper <code>{ok, data, error, request_id, meta}</code> .
Run	One execution of a plan, identified by <code>run_id</code> .
Step	One node of a plan, assigned to an agent, with dependencies on earlier steps.
Tier	A memory store level T1–T6 in the promotion model.
Promotion	Moving a memory entry to a higher tier when its trigger condition holds, recorded in the provenance hash chain.
Provenance tail	SHA-256 value summarising all promotion events so far.
SIGNOFF / REPLAN / REJECT	The executor's tri-state decision.
Deterministic mode	Execution without an LLM; reports come only from the rule-based agents.
Adaptive mode	Benchmark mode that calls a real Ollama model.
UAP	Universal Agent Protocol — the project's message model and wire encodings.
Sanskrit codename	Display name attached to an agent type in prompts and the UI (Appendix F).

Appendix B: Analysis Models

Table B.1: Analysis models in this SRS

Model	Where	Summary
Use-case model	Chapter 8	5 human/system actors, 18 use cases
Data-flow model	§3.7	Level 0 and level 1 DFDs; 7 processes, 5 stores
Architecture model	Chapter 3	5 layers + storage; ports and adapters
Behavioural models	§3.8	Sequence diagrams (3) and state diagrams (3)
Structural models	§3.8, Chapter 11	Class diagrams (agents, kernel) and ER diagram
Algorithmic models	Chapter 13	A1–A10 with pseudocode

Appendix C: Issues List

Table C.1: Open issues identified during the audit

Issue	Description	Status
ISSUE-01	Resource routes unauthenticated (SEC-01)	Open
ISSUE-02	list for run filters by conversation id (FR-016)	Open
ISSUE-03	Invalid plan returns 500 instead of 422	Open
ISSUE-04	Frontend list helpers ignore envelopes (FR-061)	Open
ISSUE-05	Two frontend tests fail after the uncommitted http.ts change	Open
ISSUE-06	Memory tier persistence not wired (FR-034)	Open

Issue	Description	Status
ISSUE-07	Observability counters never recorded (FR-050)	Open
ISSUE-08	Per-request Ollama probe (NFR-PERF-04)	Open
ISSUE-09	Compose files: SECRET_KEY and OLLAMA_BASE_URL	Open
ISSUE-10	TaskExecution metadata 'mode' always 'deterministic' even when the LLM was used	Open
ISSUE-11	run_with_llm_or_fallback reads resp.text (absent) and falls back to str(resp)	Open
ISSUE-12	save_to_sql returns None on success	Open
ISSUE-13	README claims exceed the implementation (Appendix F)	Open

Appendix D: Configuration Reference

Table D.1: Environment variables (config.py Settings and frontend)

Variable	Meaning	Default
APP_ENV	development staging production	development
APP_NAME / APP_VERSION	Display metadata	Noesis / 0.1.0
APP_HOST / APP_PORT	Bind address (CLI serve)	0.0.0.0 / 8000
DEBUG	SQL echo; forbidden outside development	false
CORS_ORIGINS	CSV or JSON list; '*' rejected	http://localhost:3000
EVAL_ROOT	Benchmark tables root	docs/eval
SECRET_KEY	JWT signing secret (≥ 32 chars recommended)	development placeholder
JWT_ALGORITHM / JWT_EXPIRE_MINUTES	Declared; JWTService uses HS256 with fixed 15 min / 14 d	HS256 / 1440
RATE_LIMIT_PER_MINUTE	Declared; login/refresh limits are hard-coded	60
CLAIM_SUITES_HMAC_SECRET	Capability-token HMAC key	dev-only placeholder
DATABASE_URL	SQLAlchemy async URL	sqlite+aiosqlite:///./data/noesis.db
QDRANT_URL / QDRANT_API_KEY / QDRANT_COLLECTION_PREFIX	Vector store client	http://localhost:6333 / - / noesis
REDIS_URL	Cache client	redis://localhost:6379/0
DEFAULT_PROVIDER / FALLBACK_PROVIDER	openai anthropic gemini ollama openrouter llama.cpp	openai / ollama
DEFAULT_MODEL / TEMPERATURE / MAX_TOKENS / TIMEOUT	Generation defaults	gpt-4o-mini / 0.2 / 4096 / 120 s
EMBEDDING_PROVIDER / EMBEDDING_MODEL / EMBEDDING_DIMENSIONS	Embedding defaults	openai / text-embedding-3-small / 1536
OPENAI_API_KEY / OPENAI_BASE_URL / OPENAI_ORG_ID	OpenAI credentials	—
ANTHROPIC_API_KEY / ANTHROPIC_BASE_URL	Anthropic credentials	—
GOOGLE_API_KEY	Gemini key	—
OPENROUTER_API_KEY / OPENROUTER_BASE_URL	OpenRouter	— / https://openrouter.ai/api/v1
OLLAMA_BASE_URL / OLLAMA_MODEL	Ollama server and default model	http://localhost:11434 / llama3.1:8b
AGENTS_OLLAMA_MODEL	Model used for agent enrichment	qwen2.5-coder:7b-instruct-q4_K_M
LLAMACPP_BASE_URL / LLAMACPP_MODEL	llama.cpp server	http://localhost:8080 / llama-3
BACKEND_BASE_URL (frontend)	Rewrite target for /api/backend	http://localhost:8000
NEXT_PUBLIC_API_BASE (frontend)	Client API base	/api/backend

Appendix E: Repository Structure

```

ASTRAOS/
├── backend/
│   ├── noesis/          agents/ api/ benchmarks/ core/ database/ kernel/ llm/ memory/
│   │                   observability/ plugins/ rag/ security/ services/ tools/ uap/
│   │                   types.py config.py logging.py cli.py
│   ├── tests/          unit/ (21 files) integration/ api/ claim_suites/
│   ├── scripts/        bench_se50_batch.py bench_humaneval.py bench_mbpp.py run_w7_weekend.py
│   │                   determinism_manifest.py audit_mac_spawn.py audit_llm_providers.py ...
│   ├── benchmarks/     noesis_se50/ (corpus.csv/json, goals.txt) mbpp/ humaneval/ selfhost/
│   ├── alembic/         versions/0001_milestone0_schema.py 0002_milestone5_mvp.py
│   ├── schemas/uap/v1/uap.proto Dockerfile pyproject.toml dist_rc2/*.whl
│   └── frontend/
│       ├── app/         page.tsx agents/ benchmarks/ conversations/ documents/ memory/ metrics/
│       │               settings/ timeline/ layout.tsx providers.tsx
│       ├── src/         components/ lib/api/ lib/ testing/ __tests__/
│       └── package.json  next.config.mjs vitest.config.ts Dockerfile
├── docs/
│   ├── eval/ (benchmark artefacts) paper/ thesis/ synopsis/
│   ├── .github/workflows/ backend-ci.yml frontend-ci.yml 4-audits.yml 4-audits-weekly.yml
│   ├── docker-compose.yml docker-compose.laptop.yml README.md ARCHITECTURE.md AUDIT.md
│   └── final_documentation/ (this package)

```

Appendix F: Documentation-to-Implementation Reconciliation

The repository README and earlier project documents contain statements that the code does not support. Because this SRS is evidence-based, the differences are listed so that readers can reconcile the documents.

Table F.1: Claims versus implementation

Claim in earlier documents	Finding in the code / measurements
Tests '309/309 pytest' and '105/105 vitest'	312 collected: 310 passed, 2 xfailed; Vitest 103/105 on the working tree
'43 FastAPI routes'	42 operations on 36 paths + hidden GET /
'/llm/benchmark POST'	GET
Routes /v1/uap/cap-token/mint/verify, /v1/kernel/runs, /v1/kernel/scheduler, /v1/kernel/alloc, /v1/observability/spans	Do not exist
'JWT · CSRF · CORS' and auth on /v1 routes	JWT only on /v1/auth/me; no CSRF mechanism
'Kernel capability gate 5-stage HMAC on every tool call and spawn'	HMAC gate only on two HTTP routes; tools/spawns use in-process capability checks (no HMAC)
Memory tiers Indriya, Kushalata, Gyān, Ranniti, Yojanā, Tattva with Redis/Qdrant/Git backends	Code: T1 Indriya, T2 Smriti, T3 Gyān, T4 Yojanā, T5 Sāadhanā, T6 Paalak — all in-memory dicts
'Efron BCa' bootstrap	Percentile bootstrap (10 000 resamples)
SE-50/HumanEval/MBPP pass@1 for GPT-4o 1.000, Claude 0.987, Gemini 0.973	No result files for hosted models in the repository
'Ablation +18.0 pp'	/api/bench/results demo payload shows 0.22/0.31/0.40; no ablation run artefacts found
'LangGraph 0.6 StateGraph orchestration'	LangGraph declared as dependency but never imported
'9 whitelist tool adapters wired'	9 adapters exist; 4 in the default registry
'Determinism 450/450 SHA-256'	450 counts ordered pairs including self-pairs (50 goals × 3 ²); 50/50 goals identical
'12 pages + Bench Hub, 10 sidebar items'	13 routes in the working tree, 10 sidebar items
Jetson Orin Nano deployment, power/TCO figures (synopsis/PPT)	No Jetson-specific code or measurements
'42 % longer task retention' (PPT context)	No supporting data

Table F.2: Sanskrit codename mappings found in three places

AgentType	Backend system prompt	Frontend label	Seeded pipeline script
planner	Manan	Manan (Planner)	Manan
research	Anveshak	Anveshak (Researcher)	Darshak
coding	Vidya	Vidya (Coder)	Karmakarta
reflection	Darshak	— (Darshak is 'Analyst')	Vivechak
memory	Paalak	Paalak (Memory)	Paalak
rag	Vivechak-II	— (no RAG role)	Vidya
tool	Karmakarta	Karmakarta (Tooler)	Anveshak
judge	Parikshak	— (Parikshak is 'Tester')	Parikshak
critic	Vivechak	Vivechak (Critic)	Rakshak
executor	Kriyakārī	Kriyakārī (Executor)	Kriyakārī
supervisor	Nirikshak	Nirikshak (Supervisor)	Nirikshak
orchestrator	Samanyaka	— (Samanyakā is 'Synthesizer'; Rakshak is 'Security')	Samanyaka

This document uses the backend system-prompt mapping (services/kernel_services.py) whenever a codename is mentioned, because it is the mapping used at run time.

Appendix G: Audit Evidence Log

Table G.1: Evidence collected for this SRS

Evidence	Method	Artefact (final_documentation/evidence/)
Route inventory	create_app().openapi() on the working tree	openapi_live.json
Backend tests and coverage	pytest --cov on an isolated copy (Python 3.13.2)	pytest_out.txt, junit.xml, cov.json
Frontend tests	vitest run (Node 20.18.1), tsc --noEmit, eslint	vitest_results.json, eslint_out.json
API scenarios and latency	TestClient with app lifespan, SQLite scratch DB, Ollama unreachable	evidence_deterministic.json (58 scenarios)
LLM-mode behaviour	Same script with Ollama reachable	evidence_llm.json
Library measurements	Planner determinism, kernel/roster/tool gating, promotion, RAG, router, executor grid, code metrics, DB schema	evidence_lib.json
UI	next dev + uvicorn on scratch copies; headless Edge 1600×1000	screenshots/*.png
Reported results	Committed artefacts	docs/eval/w7_weekend_real_Oct06_real/, se50_determinism_3runs.csv

Environment: Windows 11 Home (10.0.26200), Python 3.13.2, Node.js 20.18.1, FastAPI 0.139.2, Pydantic 2.13.4, SQLAlchemy 2.0.51, Ollama with qwen2.5-coder:7b-instruct-q4_K_M (7.6 B parameters, Q4_K_M, 4.68 GB). The repository itself was not modified: all executions used copies in a temporary directory, and the user's own running servers (ports 3000 and 8000) were left untouched.